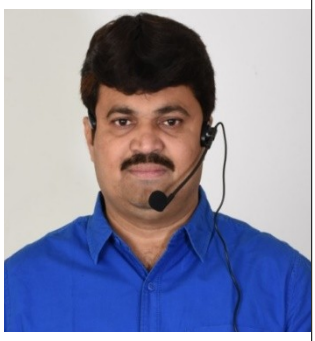


Step by Step

ASP.Net MVC

by

Mr.R.N Reddy

RN Reddy IT School

Online/Classroom Training's |Work Supports |Proxies

For more details

E-Mail: Rnreddyitschool@gmail.com

Phone : +91 9966640779, 9866183094,9885199122

Visit: www.rnreddyitschool.co

FB Page: - www.facebook.com/Reddyitschool

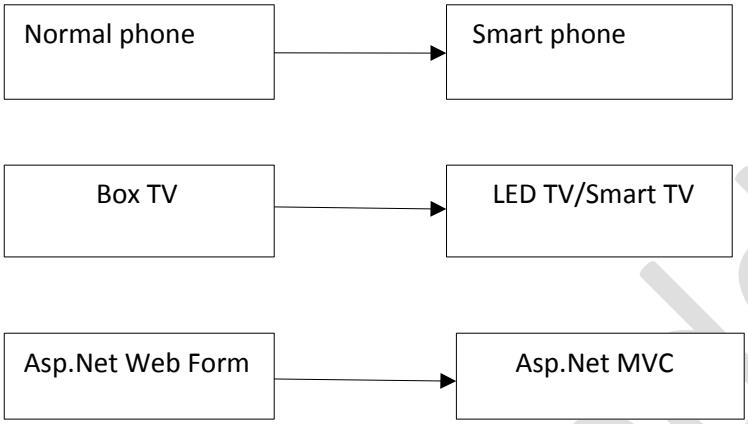
Cover's:-

---

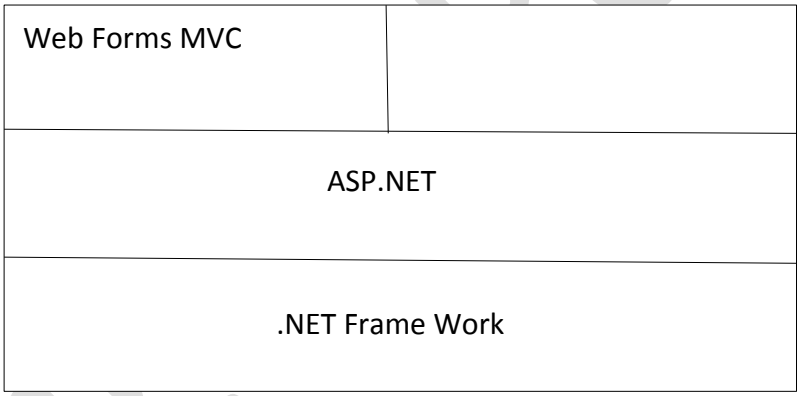
1. ASP.Net MVC
2. Linq
3. Entity Frame Work
4. Bundling In MVC
5. Routing In MVC
6. Filter In MVC

Introduction To MVC

MVC stands for Model View Controller.
MVC is a design pattern used to develop the application.
Mvc came's under architecture design pattern.
Microsoft is using MVC in Asp.Net which can be called as Asp.Net MVC.
Using Asp.Net MVC we can develop advanced web applications.
Asp.net MVC is a next generation of Asp.Net Web Forms
Like below:



Architecture:



What is Design pattern?
Design pattern is well defined solution for the problems that occur in software development in specific content(solution).Design pattern can be used by in any technology and any programming environment.
Example: Mvc - Asp.Net MVC.
Singleton – WCF.
DAO(Data Access Object) – ADO.NET.

What is MVC Design Pattern?
In MVC Design pattern application development will be splitting into three modules.
1.Module
2.View
3.Controller

Model:

- Model is responsible for database related logic.
- In Mvc application model will communicate with database and returns the result to controller.
- Model send the result/status to controller after processing data base operation.
- In Asp.Net mvc modules are developed by using Ado.Net/Entity Frame Work.

View:

- **View** is responsible for presentation related logic(UI).
- View will get the data from controller and prepares output.
- In Asp.Net Mvc, view are developed by using technologies
  1. HTML(Hyper Text Mark-up Language)
  2. CSS(Cascading style sheet)
  3. JavaScript
  4. Angular JS
  5. **C#**

Controller:

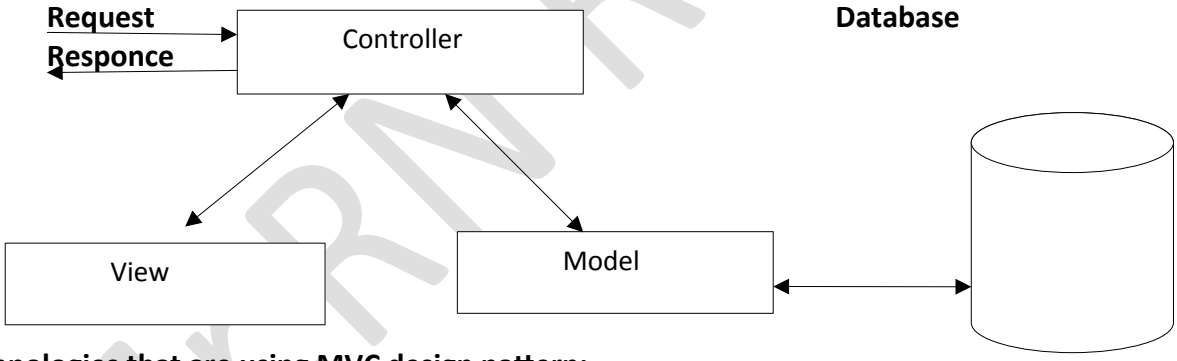
- Controller is responsible for application Execution logic.

Every request will received by controller.

- All the time of processing request, controller will communicate with model or view in order to process the request
- In Asp.Net Mvc controller are developed by using C#.Net.

i.e: **.cs** file

MVC Diagram:



Technologies that are using MVC design pattern:

- Asp.Net Mvc(.Net)
- Spring Mvc(Java)
- Struct Mvc(Java)
- Ruby on Rails Mvc
- Php Mvc
- Iphone OS

Introduction to Asp.Net MVC:

What is Asp.Net Mvc?

- Asp.Net Mvc is a framework used to develop web application by using Mvc Pattern.
- In Mvc based application development will be splitting into three modules. They are: Model - view - controller

Note: Asp.Net Mvc was designed by **scott Guthire** in the year 2007

History of Asp.Net Mvc version?

Asp.Net Mvc 1.0 - 2009
Asp.Net Mvc 2.0 - 2010
Asp.Net Mvc 3.0 - 2011
Asp.Net Mvc 4.0 - 2012
Asp.Net Mvc 5.0 - 2013
Asp.Net Mvc 5.2 - 2014
Asp.Net Mvc 6.0 - 2015

Note:

- Visual Studio 2012 and 2013 contain Asp.net Mvc 4.0 by default
- If we want to develop Asp.Net Mvc 4.0 application with visual studio 2010.we have to install following additional packages
    1. Visual studio 2010 service pack1
    2. Asp.net Mvc 4.0

Note: If we are install Visual Studio 2015 by default Asp.Net Mvc 6.0 will come.

Asp.Net Mvc Feature:-

What are the advantages of Asp.Net Mvc and why do we use Asp.net Mvc inserted of Asp.Net web form?Because of the below 10 advantages we will go for Asp.Net Mvc rather than Asp.Net Web forms.

1. Separation of code
2. Loosely coupled/less dependence
3. Parallel development
4. Easy to perform unit testing
5. Tdd support
6. Clear/clean urls
7. Improved performance
8. More controlling on html(design)
9. Added new concepts
10. Easy to learn and easy to implement

1.What is separation of code in Asp.Net Mvc?

- Mvc based applications are separated the model[database logic] from view[presentation logic].
- Due to this separation of code application can develop easily
- Single application development we can split into three parts

2.What is loosely coupled or less dependency in ASP.NET MVC?

- In MVC based application views are less dependent on model
- These two modules[view and model] are organization by controller
- Due to this feature we can easily update the required model without affecting other module.

3.What is parallel development in Asp.Net Mvc?

- In Asp.Net Mvc a team can be work on model and another team can work on views with this parallel application development process will continue.
- Due to this feature development process become faster and easy.

4.**How in Asp.net Mvc easy to perform unit testing?**

- In Asp.Net Mvc application development will be divided into three parts .
  i.e. Model , View, Controller
- Because of this we can easily perform the unit testing on particular module.

5.What is Test Driven Development (TDD) support in Asp.Net Mvc?

- TDD stands **Test Driven Development**
- It is a level development technique.
- In TDD test will provide new feature/ options for development continutationally.
- Mvc applications are more comfortable to modify at any point of the time without affecting the entire project.

6.what is clear/clean Urls in Asp.Net Mvc?

- Mvc applications are using urls according to the controller.
- All mvc are controller based not file based.
- In Asp.Net urls are file based.
- Mvc urls are called as clean/clear urls because it contain less number of characters[Query string] in url .
- Due to this feature SEO will be easy.

Example for Asp.Net Web forms urls:

/product.aspx
/product.aspx?cat=5
/product.aspx?cat=5&sub=17

Example for Asp.Net Mvc url:

/product
/product/shirts
/product/shirts/kids

Due to this feature SEO will be easy.

What is SEO?

- SEOStands for Search Engine optimization.
- SEO is a process of identity our web site url by search engine.
- Mvc urls are more comfortable for SEO.

7.How Asp.Net Mvc will improved the performance od web application?

- Compare with Asp.Net web form application.Asp.Net Mvc application will provide more performance.
- Asp.Net web form process the webpage with lot of activities like below
    1. Page event
    2. Server control object
    3. Load request data
    4. Process the values
    5. Covert to html and so on...
- Due to this behaviour Asp.Net web form execution will be time consuming and had extra burden on the server .
- Asp.Net Mvc does not perform all the above activities because it does not support server controls hence processing will be more faster.

8.**how in asp.net mvc we can have more controlling on html?**

- In asp.net Mvc view[which contain UI logic] will be developing with the help of HTML.
- Due to above reason we can easily integrate ant kind of client side libraries which mvc views.

Example for client side libraries:
j Query, Angular JS , Bootstrap and so on.....

9.What are the newly added concepts in Asp.Net Mvc?

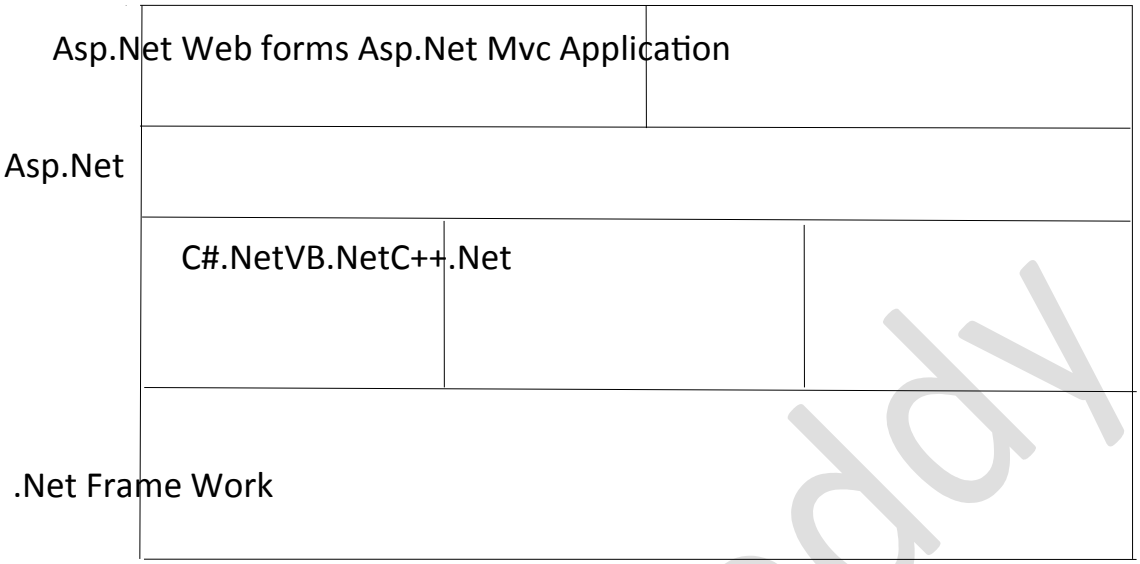
In Asp.Net Mvc new concepts are added to make web application development will be comfortable and these concepts are

1. Filters
2. Routing
3. Attributes
4. Web API

5. Scaffold templates etc...

10.How Asp.Net Mvc is easy to learn and easy to implement?

Asp.Net Mvc is development based on the existing .net while developing the Asp.Net Mvc application we can reuse the most of the C#.Net and Asp.Net web forms concept.



Note: Asp.Net Mvc is not replacement of Asp.Net ,if is on alternative option to develop web application in .net environment finally we can say Asp.Net Mvc is next generation of Asp.Net web forms.

Asp.Net Mvc application development environment:

Asp.Net Mvc provides two types of projects templates to develop the web application.

1.**Main template**:

Asp.Net Mvc4 web application.

2.**Sub template:**

1. Empty
2. Basic
3. Internet applications
4. Intranet applications
5. Mobile applications
6. Web API etc...

1.Empty:

This template is similar to Asp.Net empty web applications.
its generates empty folders. We need to develop form scratch.

2.Basic:

This template same as empty template.
It provides additional folders for client side programming.

I. Content:- this folder can contain .CSS files and Images.

Example: .CSS

II.Scripts:- this folder can contain the client side scripting like
Java Script , Angular JS, or J Query files ...

Example: .Js

3.Internet applications:

This template is same as “Asp.Net” web application.

It generate project with from authentication security.

4.Internet application :

This template is same as internet application, but it uses windows authentication as security.

5.Mobile application:

This template same as internet application.

It targeted the mobile device instead of regular browser.

6.Web API:

Web API template is used to create service based application in Mvc pattern.

Web API service is called as HTTP sevice.

Note:1 Mobile application and web Api templates are introduction in Asp.Net Mvc4.

Note:2 Basic template is more comfortable for beginners/who wants to develop the project from scratch with default styles and scripts.

How to create Asp.Net Mvc basic application?

- Open Visual Studio.Net
- Click on new project. It will open new project windows here select .NetFrameWork 4.0 from the top dropdown list.
- Select left side Visual C# and select Web
- Right side select template as Asp.Net Mvc4 Web application.
- rename it as MyMvcBasicAppication. Location as D-drive;click ok button with this process it will open new Asp.netMvc4 project window.
- Here select template as Basic click ok button. With this process it will create a new Asp.NetBasicMvcApplication will created.

Asp.Net Mvc application folder structure:

By default this basic Mvc application solution explorer window will come following important folders.

1. App_Start.
2. Content.
3. Controllers.
4. Models.
5. Scripts.
6. Views

1.**App_start:**

This folder content application start up setting related files. These files are C# files(.CS)

By default this folder will come with four C# file

1. BundleConfig.Cs
2. FilterConfig.Cs
3. RouteConfig.Cs
4. WebApiConfig.Cs

Among these 4 files important file is routeconfig.cs

In this file we will defined the default controller details.

2.**Content:**

Content folder contains all 'css' files,we may add images also in this folder if we required

3.**Controller**:

Controller folder contains all controller class files.

How to add a controller?

Select controller folder ,right click->Add->select controller.

It will open Add controller window here rename controller name as HomeController, Click Add button with this one controller is Added.
i.e. HomeController.Cs
We can add multiple controllers but every controller file will be C# class file.
Extension will be .Cs
4.Models: This folder contain database logic related C# files.
Here we will implement Ado.Net code or Entity Framework code.
5.Script: It contain client side scripting file(.Js)
For ex: .Js
By default Asp.Net Mvc project generates JQuery related files
We can also add user defined script files in this folder.
6.Views: views folders contain webpage.
In Asp.Net Mvc all web pages are HTML web pages.
In Mvc user develop with C# code and HTML tags,due to that reason these files extension will be like below
".Cshtml"
Note: Asp.Net Mvc application follows proper folder structure, you should add the file in specified folder only.
Controllers in Asp.Net Mvc :-

- In Mvc 'C' stands for controller .
- Controller is responcible for application execution logic.
- Controller is important module in Mvc which will take care execution on every request.
- In Mvc applications request or handled by controller.
- Every request in mvc application mapping with controller and corresponding action method.
- Controller is act as a mediatory between model and view. That model communicate with view throw controller.

Responsibility for controller:-
Controller will perform the following activities.

- Receiving the request.
- Process the request by execution action method. To communicate with model or to communicate with view.
- Execute the corresponding view.
- Sending the response.

Note: At the time of execution action method controller will communicate with model based on requirement.
Developing controllers :

- In Asp.Net Mvc controllers are created as class.
- Every controller is a class.
- We can create multiple controller class based on the requirement.
- Each controller class defines corresponding required action method to process the request.

Example for controller class:
HomeController.cs files

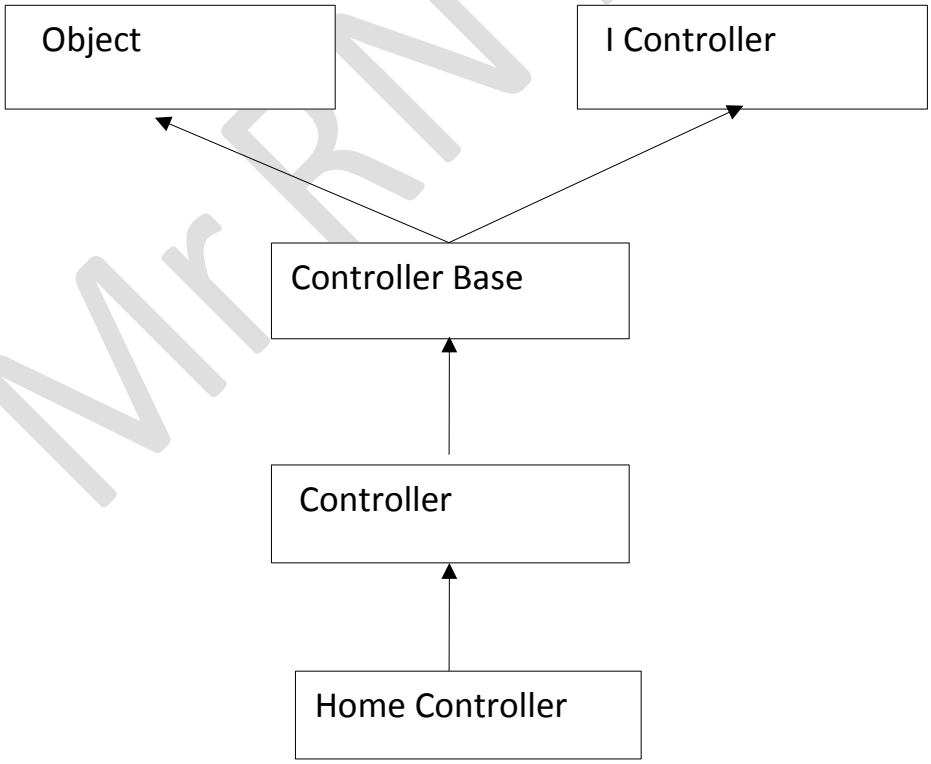
Public class HomeController:Controller
{
//here we have to define execution logic with the help of Action method.
}

Note:1 controller class should be declare with “public” access modifier or specifier.
Note:2 controller class name should be end with(postfix) “controller” word.
For exemple: HomeControlle , EmployeeController
Note:3 every controller class(user defined) class which is post of a namespace(base class library) That is **System.web.Mvc.**
This controller class is super class for all controller user defined class. like below .

1.class HomeController:Controller
{
}
2.class EmployeeController:Controller
{
}
3.class StudentController:Controller
{
}

Controller pre defined class will provided built in object and predefined methods to process the request.

Diagram of inheritance hierarchy of controller class.



Note: Here controller class and controller base class are predefined abstract class.
But I-controller is predefined interface.

This of above class are:

```
Interface IController
{
}
Abstract class Controller:ControllerBase
{
}
Class HomeController:Controller
{
}
```

Action Method**:**

- Controller will process the request with the help of action method.
- Actual action method contains request processing logic
- This logic may communicate with model to process database related task.
- Action method will prepare the result that should display in view.
- Finally action method will transfer the result to view in order to display to end user

Developing action method:

Note: action method should be public and non-static(instance).

Syntax:

```
Public ActionResult Index()
{
Return view();
}
```

viewResult. That means view() method return object of "ViewResult" class.

What is viewresult?

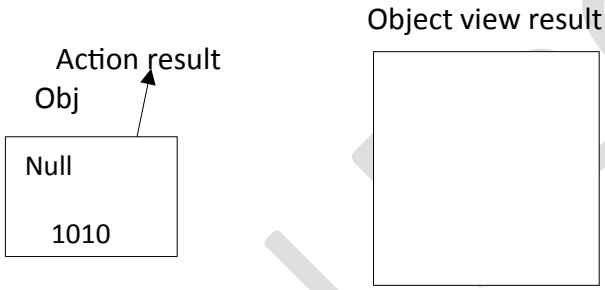
- View() is a predefined method in controller class.
- View() method return object of viewResult Class.
- View result class is a child class of "ActionResult".
- Action method may return following result class object
    1. ViewResult
    2. RedirectToRouteResult
    3. ContentResult
    4. JSonResult
    5. FileResult..

All these result class are derived class of action result class like below

```
abstract class ActionResult()
{
}
Class ViewResult:ActionResult
{
}
Class RedirectToRouteResult:ActionResult
{
```

```
}
Class contentResult:ActionResult
{
}
Class JSonResult:ActionResult
{
}
Class FileResult:ActionResult
{
}.....
```

In mvc Action()Method may return any type of above result class type:
But Action Result class variable can hold any of the above result class objects. Because action result is the super class for all above result class.
A diagram to represent how view result object is holding by the action result class variable like below.



Explanation for the above diagram with data follow:
Here action method Index() is calling the View() method like below:

```
Public class ActionResult Index()
  {
    Return view();
  }
```

View() method is returning the ViewResult.object like below:

```
 ViewResult View()
{
//it returns object of ViewResult()
}
```

View() method is retuning the ViewResult object to index() method. That object holding by the ViewResult () variable because index method1 return type is ActionResult.
Finally index method will return ViewResult object.
Note:
Most of the ActionMethod return ViewResult.
All ViewResult invalided ActionMethod should have corresponding view page.
That is: .Cshtml

Type of ActionMethods:

The method which we will define with in the controller are called ActionMethod. These ActionMethods are 3 types:

1. Action method with ViewResult.
2. Actiom method with non ViewResult.
3. Non-Action method.

Action method with ViewResult:

- ActionMethod that will return View pages output those method come under.
- Most of the actionmethod() or returnview result.
- These method return statement like below.
  - ReturnView().
- These method requided a view page with corresponding method name
  - For ex: index.cshtml.

Action method with non view result:

- Action method with non view result.
- Action method will return data instated of HTML formatted output(webpage)
- These method will return single value/object /collection of objects.
- Especially in Ajax programming we are involving these methods.
- These method having return statements like below
  1. Return JSon();
  2. Return content();
- These method doesn't required view page.

Non_ActionMethod:

- By default all method in controller class consider as action method.
- All action methods can be accessible(requested) by end user from browser.
- If we want to define a method that should access with be called as non_action method.

Note: Non_action methods are preparing by using a attribute like below.
That is: [Non Action]

Example for non-action method:

```
[Non Action]
Public int sum(int x, int y)
{
   Return X+Y;
}
```

Attributes in Asp.Net Mvc:

- Attribute in Asp.Net Mvc are user to provide a special instruction to execution engine.
- So that it will be handled separately.
- Evert attribute in Asp.Net Mvc was developed as a predefined class.
- All the attribute pre-defined class defined by the Microsoft with in a base class library called System.web.mvc.
- Attribute class name should end with attribute word.
  For example:
  - NonActionAtrribute
  - HttpGetAttribute
  - HttpPostAttribute....

- At the time of appling attribute we will write particular attribute with in[].
  For example:
  - [NonAction]
  - [HttpGet]
  - [HttpPost]...
- We can apply attributes either to define class or methods or property.
  For example:

```
[NonAction]
        Public int sum[int x, int y]
        {
                Return x+y;
        }
```

View in Asp.Net Mvc Applivation:

In mvc application,execution of views are processed by a special view engine called view enginee.

What is view enginee?

- view engine is sub system of Mvc framework.
- View engine can independent process the views.
- **If any changes in view page no need to compile entire project**.
- Views are separately process by view engine.

What are the view engines supported by Asp.Net Mvc ?

Microsoft has developed two view engines that are integrated with Asp.Net Mvc

1. Web form view engine [.aspx].
2. Razor engine.

1**.Web form view engine [.aspx]:**

- Web form view engine is introduced in Asp.Net Mvc 1.0
- It uses same style of Asp.Net web forms.

Limitations of web form view engine:

- Write server code(C# code in aspx.file is complex)
- It is not much comparable for programmers to perform development, modification , testing.
- We need to write more code to organize server logic.

2.Razor engine:

- Razor engine was introduce in Asp.Net Mvc 3.0.
- Razor engine called advanced view engine.
- Compare with Aspx view engine razor engine will provided more advantages.

Advantages of Razor engine:

- less code to implement server side logic.
- Razor engine is having automatic recognition of C# statements and Html tags because it is advance view engine.
- Easy to perform unit testing and more comfortable for development and modification.
- Which contain mixing of Html and C# code.
- Easy to read and understand.

Basic of Razor programming:

- In razor programming view page execution will be .cshtml.
- Every view page(.cshtml) contain html tags and C# code.

- By default content will be consider as by view engine.
- We need to differentiate C# code with a special symbol "@" .

We can integrate related items(C# code) in view in the following ways:

1. Inline Statements
2. singleLine Statements
3. MultipleLine statements.sssss

1.InLine Statements:

- InLine Statements to usedto involve C# variable or objects in html tags.
- Inside the html tags we can use c# variables or objects by attaching "@" symbol.
- Otherwise it will consider as Html and display the variable to the end user.

For Example:1

```
String Uname:'Rama';
<h1>where to uname</h1>   //Wrong
O/P
-------------------------
welcome to uname

<h1>welcome to @umame</h1> //correct
 O/P
-------------------------
Welcome to Rama
```

Example:2

```
<span>EmpName is: @obj.Ename</span>
O/P
--------------------------
Empname is:Ravi
```

Example:3

```
<span>Custname is: @obj.cname</span>
O/P
---------------------------
CustName is:raju
```

Example:4

```
Int i=10;
<span>@(i+1)</span>
O/P
-------------------------
   11
```

SingleLine Statement:

If we want to write one line of C#.code then we can use this option.

If may be used to declare variables/objects,assignments,statements...

Syntax:

@{C#statements}

For example 1:Declare a variable:

@{int x=10;}

Example 2: creating a object
@{emp obj=new emp();}
Example 3: increment and assignment
@{count = count +1;}

MultiLine Statements:

- If we want to write more than one Line of C# code then we can used the option.
- Multiple statements also allow the html tags.
- Razor engine automatically deviate C# statement and Html tags.

Example for MultiLine statements:

```
@{
        Int x=10;
       If(x%2==0
       {
            <h1>@x is Even</h1>
        }
       Else
        {
            <h1>@x is odd</h1>
         }
      }
```

Note:
where we will write the server side logic. that is: C# code in case of web form view engine.

A. With in .aspx file head session.

Programming by using Asp.Net Mvcwith the help of web from view engine.
Who will select the view engine?

A. Programmer

1.example to display one welcome message with the help of web form view engine in step-by-step:

step 1: Open Visual Studio .Net.
click on new project it will open new project window,here select visual C# and select web,right side select Asp.Net Mvc4 web application and rename it as: MvcEmptyWelcomeApp click OK.
It will open New Asp.Net Mvc4 project then select a template called Empty, then select view engine as aspx then click OK with this process a new Asp.Net Mvc empty Application will create.

Step 2: Adding Controller.
Open solution explorer select controller folder right click Selec → Add→Controller. with this process add controller window will open.Rename it as HellowWordController Click Addwith this process a new controller will add to the controller folder that controller name is HellowWordController.Cs .
By default HellowWordController.Cs will come with one action method.
That is: Index().
Finally .Cs file will be like below.

```
Class HellowWordController:Controller
  {
     Public ActionResult Index()
```

```
    {
      Return View();
     }
  }
```

Step 3: Adding view to Controller

Open HellowWordController.Cs
Right click on Index() method.
Select Add view, It will open AddView window
Make view name as Index
View engine as Aspx(C#)
Unselect use a layout or master page
Click Add

Note: Don't select any master page. Here because in this example we are not using any master pages.With this process within the views folder one view page will add under Helloworld subfolder i.e: Index.aspx
Step:4 Add design code within index.aspx div session.

```
<div>
        <h1>welcome to Asp.netMvc</h1>
</div>
```

Step:5 Write the server side code i.e C# code within Index.Aspx HeadSession.
Note: No code for this example.
Step:6 Initialise the default controller name within the RouteConfig.Cs file.
Expand App_Start folder,open RouteConfig.cs
With RouteConfig.Cs file update,like below
Controller:"HelloWorld".
Build the solution and Run the application finally
O/P is Index.aspx

Welcome to Asp.Net Mvc

Example to display the welcome message using aspx view engine when the page is loading.
Step:1 Gating a new empty application.
Step:2 Adding Controller.
Step:3 Adding view page to controller
Step:4 Add design code(Index.aspx) div session
Note: no design code.
Step:5 write the server side code(C# Code) in Index.aspx Headsection.
Here we need to load event due to that reason double click on Index.aspx page to generate load event .

```
<script runat="server">
 Void page_load()
{
    Response.write("HI , Welcome to Asp.Net Mvc");
}
</script>
```

Step:6 Initialize the default controller within RouteConfig.CS Build the solution and run the application, finally O/P is Index.aspx

Hi, Welcome to Asp.Net Mvc

Example to display the welcome message within the label at the time of page is loading by using Aspx View Engine.
Step:1 Create a empty Mvc application.
Setp:2 Adding a Controller.
Step:3 Adding a view page to controller.
Step:4 Adding the design code within Index.aspx div selection.

```
<div>
<asp: Label ID=”lblwelcome” runat=”server” />
</div>
```

Setp:5 Write the server side code i.e C# code in Index.aspx head session.

```
<script runat=”server” >
 Void page_load()
{
   Lblwelcome.Text=”welcome to Asp.Net Mvc...”;
}
</script>
```

Build and execute the **O/P is**
index.aspx

Welcome to Asp.Net Mvc

Example to display to messages on web page within using label when the page is loading by using aspx view Engine Application name MvcFirstApplication. Controller name is Welcome view page name Home.aspx.
step1: Creating new MvcFirstApplication.
Select view Engine.aspx
Step2: Adding the controller WelcomeController.CS
Step3: Adding viewpage to the controller Home.aspx.
Step4: Adding design code(Home.aspx) div session
Note: No designcode.
Step5: write the server code(C# Code) in Home.aspx.here we need load event due to that read on double click on the Home.aspx page to generate load event.

```
<script runat=”server”>
     Void page_load()
      {
           Response.write(“I  Like C#.Net”);
           Response.write(“I Lile Asp.Net”)l;
      }
</script>
```

Observation : Whenever we are implementing aspx view engine programming there are two precutations we should take care.
1.Action method name and view page name should be same.
2.By default Action method name is Index().

For example: If we want to be have view page as employe.aspx then ActionMethod name should be changed as Employee()
When ever ActionMethod name should be changed then we should change the RouteConfig.CS file below Attribute i.e. action="Employee"

Example to implement aspx view engine programming by changing with action method name as student() and view page name as student.aspx.

Step:1 Create asp.net Mvc application.
Step:2 Add controller rename it as myEmpController(every controller should postfix as "Controller").
Note: Here controller name is MyEmp display name is MyEmp controller
Step:3 Adding the view:
Open MyEmpController.Cs and change the action method name asstudent() like below:

```
Public ActionResult Student()
{
    return view();
}
```

Right click on action method i.e. student()
Select Add View.
View name as student
View engine as aspx.
Unselect the master page checkbox.
Click Add Button in this process a new page will as
i.e. student.aspx under

write the below code with in the student.aspx div section

```
<div>
<asp: Label ID="lblwelcome" font_Bold ="true" runat="server" font_size="large"/>
</div>
```

Step:5 Double click on student.aspx it will generate the load event. Under that load event write the below code.

```
<script runat="server">
 Void page_load()
  {
      Lblwelcome.text="welcome to Asp.Net Mvc aspx programming";
  }
```

Open App_start folder⟶ Open RouteConfig.cs fileand update the below attributes.
1.Controller="MYEmp"
2.action="submit"
Now nuild the application and run the application finally

O/P is Student.aspx

Welcome to Asp.Net Mvc aspx programming

Razor engine programming:

Example to display welcome message by using razor view engine in step by step

Step1:

- Open visual studio.Net click on new project
- Select left side web; right side Asp.Net Mvc4 web application ,rename it as MvcEMpRazorApplikcation location as D:/ drive click **ok.**
- Here select empty template; select viewEngine as Razor, click **ok** Button .

Step2: Adding Controller:

- Open solution explorer select controller folder right click add controller. Rename it as controller name as welcome controller click Add Button.

Step3: Adding Views:

- Right click on Index() or Rename ActionMethod "Index()" as "Display()" like below

```
Public ActionResult Display()
{
    Return view();
}
```

- Right click Display() → select Add view. It will open AddView window .view name as "Display" and view Engine as "Razor"(CsHtml) and uncheck the below checkbox. Use a layout of master page
  Click Add with thid process Display.Cshtml view page will Add under below

View Welcome [@]Display.Cshtml

Write the below code with in the Display.cshtml page div section Display.cshtl page

```
<html>
<body>
    <div>
    <h1>welcome to Asp.Net Mvc Razor programming</h1>
    </div>
</body>
</html>
```

step4: Open App_start folder → open routeconfig.cs file
change the controller ="Welcome " action ="display"

step5: Build the solution and Run the application finally

O/P Display.Cshtml

Welcome to Asp.Net Mvc Razor programming

Example to display welcome message with the help of single line statement and inline statement

Step1: creating the application
Step2: Adding the controller
Step3: Adding viewpage
Step4: write the below code with in Display.cshtml file

```
<html>
    <body>
        <div>
            @{ String name = “Rama”;}
        <h1> welcome to @ name</h1>
        </div>
    </body>
</html>
```

Example to implement multi line statement and inline statement

Step1: creating the application
Step2: Adding the controller
Step3: Adding viewpage
Step4: write the below code with in Display.cshtml file

```
<html>
    <body>
        <div>
            @{ String name = “Rama”;
               Byte age= 20;
               String location =”Hyderabad”
        <h1> welcome to @ name</h1>
        <h1>age is @age</h1>
<h1>location is @ location</h1>
            }
    </div>
    </body>
</html>
```

O/P

Name is : Rama Age is : 20 Location is : Hyderabad

Example to store customer information and display

Step1: creating the application

Step2: Adding the controller

Step3: Adding viewpage

Step4: write the below code with in Display.cshtml file

```
<html>
    <body>
        <div>
            @{   ulong cid=111;
 String cname = “Rama”;
                String cloc =”Hyderabad”
<h1> customer id is: @cid<h1>
            <h1> customer name is: @ cname</h1>
<h1>customer location is @ cloc</h1>
                }
        </div>
        </body>
</html>
```

O/P

customer id is:111

customer name is: Rama

customer location is : Hyderabad

Example To calculate 3 subject marks and display totmarks and avemarks

```
<html>
<body>
<div>
    @{ float m1=60;
        float m2=70;
        float m3=80;
        float totmarks=m1+m2+m3;
        float avgmarks=totmarks/3;
<h1> totmarks are :@ totmarks</h1>
<h1>average marks are: @avgmarks</h1>
</div>
</body>
```

O/P: Display.cshtml

Total marks are:210

Average marks are: 70

Example to calculate 6 product cost total bill i.e. p1 to p6 and on total bill give 10% discount values and display total bill before discount and give discount value and after discount.

Dispaly.cshtml

P1 double p2 double p3 double

1000 20000 3000

p4 double p5 double p6 double

4000 5000 6000

Total bill before is:21000

Discount is:2100

Total bill after discount:18900

double tot bill

21000 +p6

1000+2000+3000+4000+5000+6000

=21000

disc double

2100

21000*0.10

= 2100

totbill

totbill-disc

18900

21000-2100

=1890

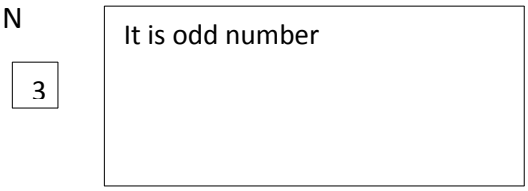
Display.cshtml

```
<html>
<div>
        @{ double p1=1000;
           Double p2=2000;
           Double p3=3000;
           Double p4=4000;
           Double p5=5000;
           Double p6=6000;
           Double totbill=P1+p2+p3+p4+p5+p6;
           Double disc=totbill*0.10;
           <h1>totbill before discount: @totbill<h1>
           Totbill=totbill-disc;
           <h1>discount is: @disc<h1>
           <h1> total bill after discount :@totbill</h1>
}
</div>
</body>
</html>
```

Control statement in razor view enginee programming

Example to take one number and check it is even or odd

Step1: Dispaly.cshtml

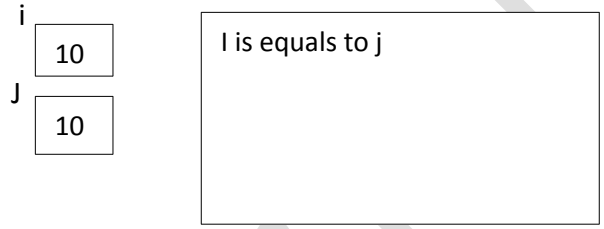


Step:2 dispaly.cshtml div session code

```
<div>
    @{  int n=3;
        If(n%2==0)
          {
          <h1>It is even number</h1>
          }
       Else
        {
<h1>It is odd number</h1>
        }
      }
</div>
```

Example to compare i and J

STEP1: display.cshtml



Step2:

```
<div>
          @{
              Int i=10;
              Int j=10;
              If(i > j)
                {
              <h1> i is greater than j<h1>
                }
              Else if(j > i)
              {
              <h1> j is greater than i</h1>
              }
             Else
              {
              <h1>i is equals to j</h1>
              }
          }
```

```
</div>
```

Example to print 1 to 10 numbers.

Display.cshtml div session code

```
<div>
   @{
          For(int i=1;i<10;i++)
          {
<h1> @i </h1>
          }}
</div>
```

o/p

```
1
2
3
4
5
6
7
8
910
```

Example to print 1 to 10 numbers like below

```
<div>
    @ {
          For(int i=1;1<=10;i++)
           {
              If(i<=5)
                  {
                  <h1> @i </h1>
                   }
                  Else
                  {
                  <span> @i </span>
            }
```

```
1
2
3
4
5
6  7  8  9  10
```

Note: In razor programming all the controller are client side controller

To implement user interaction programming or to implement above programming we have to understand type of request in Mvc and sum pre define properties and predefined html helper class and a pre defined object that is html and a pre defined html helper methods.

End user will give the request with server with the help of browser.

These request are divided into 2 types

1.Get request

2. Post Request

1. **Get Request:**

We can make get request by entering URL or by clicking hyper link.

The purpose of Get Request is to Get the webpage from the server.

2.**Post request:**

We can make the post request by click submit button. Because will submit the request based on ActionMethod attribute

For example: attribute action method type of request

<form action: “Home/Index” method= “post”>

..... controller attribute

</form>

Browser will submit the input controller values along with their names.

In mvc server code we can read the values by using request object

Request object will return the values in “string” format.

For example:

String s1=request[“t1”]

How to identify the request type:

Asp.net or Asp.net Mvc web page request are 2 types

1. First Request
2. Post Back Request

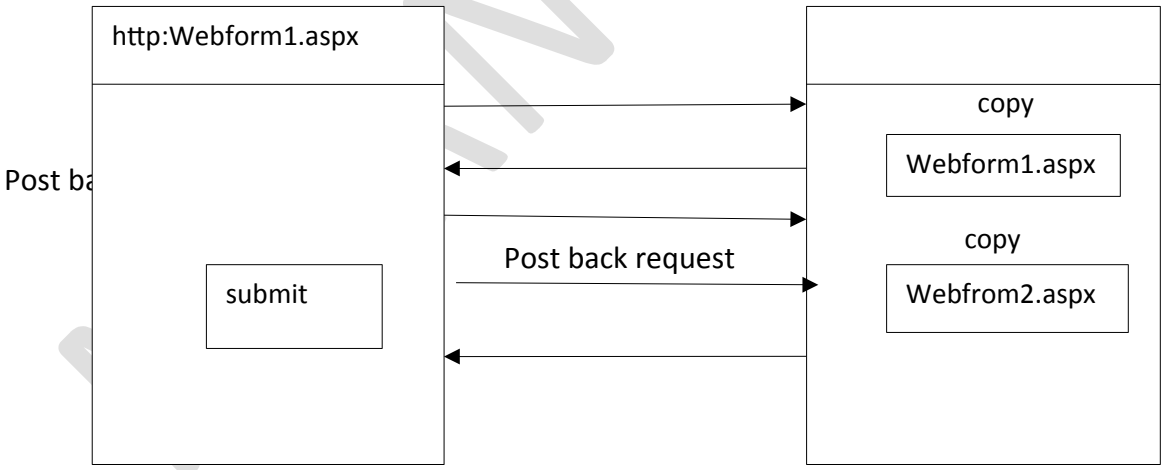
1.First Request: A request which is generate for a web page first time . is called as first request.

A web page will have only one first request.

2.post Back Request: when ever user will interact the web page with the help ofcontroller like click button or select the radio button or select the check the box...

One request generate which is called as post back request.A web page may contain multiple post back requests the number of post back request will be depending on number of user interaction diagram to under s.

Diagrams to under stand first request and post back request



How to identify the request type:

In mvc viewpage provides a property is called “IsPost” to recognize first request or post back request.

IsPost: It is a Boolean property it can contain either true or false

when ever it is first request then ispost value is “False”.

When ever it is post back request then ispost is “true”.

<syntax to access ispost property>
This.ispost
Note: Default value of the IsPost property is "False".
Program: Example to understand first request and post back request.
Step:1 index.cshtml [Design]

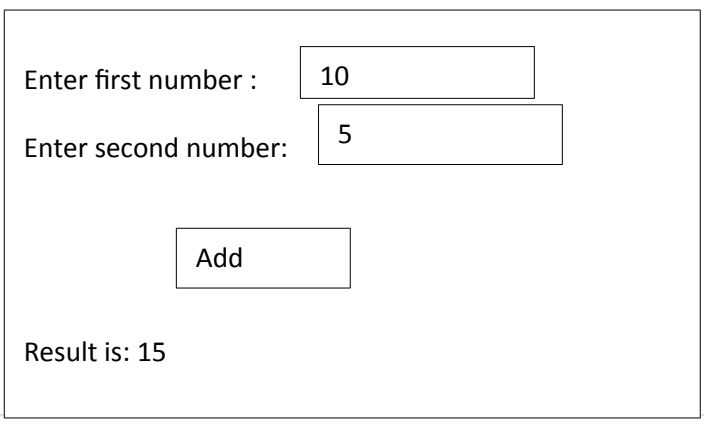
Step:2 Write the below design code with in the body section of Index.cshtml

```
<html>
<body>
<form name="myform" method="post">
<div>
<input type="Submit" name="btnsubmit" value="Submit" />
</div>
</body>
</html>
```

Step:3 Write the Razor code with in the Index.cshtml

```
@{
        Layout=null;
        If(IsPost)
           {
<h1> this is post back request </h1>
           }
  Else
           {
<h1> this is first request</h1>
           }
}
```

Example to accept two numbers and perform Addition then display the Addition request



a
10
5
15

10+5

Index.cshtml [Designcode]

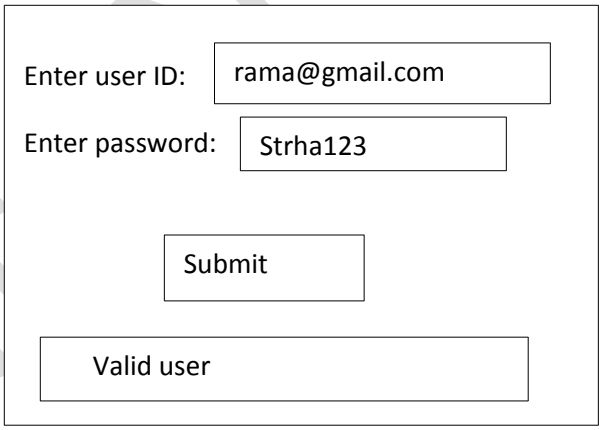
```
<div>
    Enter first Number:<input type=”text” name =”txtnum1” />
<br/></br>
 Enter second Number:<input type =”text” name=”txtnum2”/>
<br/>
<input type =”submit” name=”btnsubmit” value=”submit” />
</br>
Result is: <span>@c </span>
</div>
```

Step :3 Razor code with in index.cshtml

```
@{ layout=null;
     Int c=0;
     If(this.IsPost)
      {
        Int a=request[“txtnum1”].AsInt();
        Int b=request[“txtnum2”].AsInt();
        C=a+b;
       }
}
```

Example to validate uid and password

Index.cshtml[design]



Step 2: Index.cshtml view page[design code]

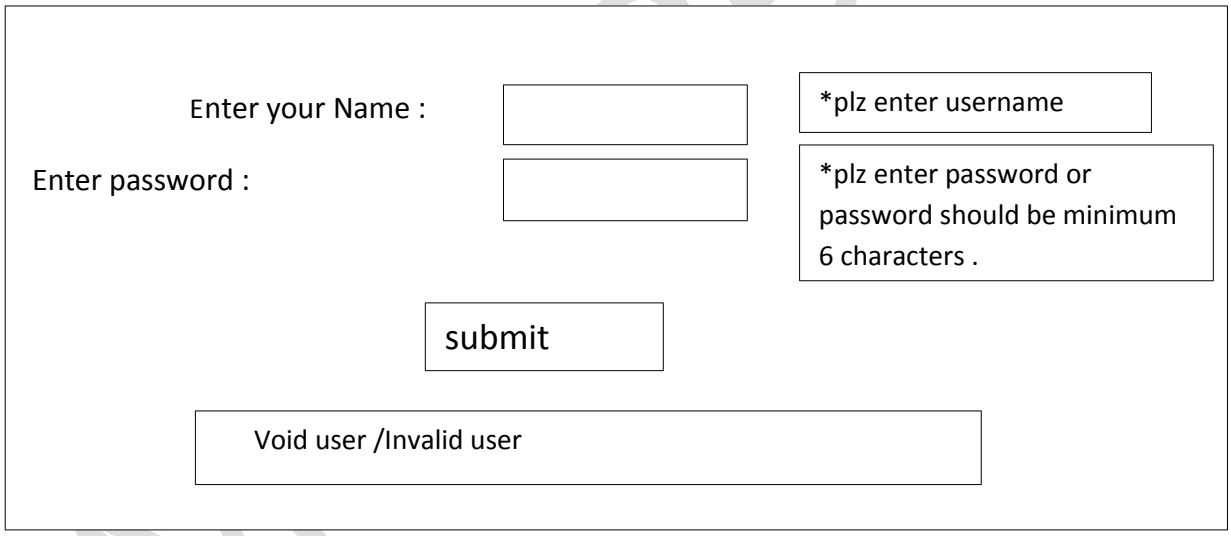
```
</head>
<body>
<form name="myform" method="post">
<div>
            Enter EmailID:<input type="email" name="txtemaild" />
<br />
           Enter password:<input type="password" name="txtpwd" />

<br />
<input type="submit" name="btnsubmit" value="submit">
<br />
<br />
<spanp>@msg</spanp>
```

```
</div>
</form>
</body>
</html>
```

Step 3: Razor code

```
@{
    Layout = null;
    string msg = Request["txtemaild"];
if(this.ispost)
 {
if (Request["txtemaild"] == "rama@gmail.com" && Request[“txtpwd”]=”sitha123”)
    {
        msg = " userId : " + msg;
    }
    else
    {
        msg = "invalide userid";
    }
  }
}
```

```
Implement above example with following validations
1.txyUname should not be empty.
2.txtpwd should not be empty.
3.txtUname should be Rama.
4.txtPwd length should be 6 characters.
5. txtPwd should be sitha123.
```



Step 2:

what is the predefined method empty

IsEmpty()

This method will return true the given filed is empty otherwise it will return false.

What is the property you check the length field

By using a property called “Length”.

Index.Cshtml[design code]

```
<body>
<form name=”myform” method=”Post”>
<div>
        Enter userid:<input type=”text” name=”txtuname”/>
        <span>@uidmsg</span>
        <br/>
```

```
            <br/>
            Enter your password :<input type="password" name="txtpwd"/>
            <span>@pwdmsg</span>
            <br/>
            <br/>
            <input type="Submit" name="btnsubmit"/>
            <br/>
            <br/>
            <span>@msg</span>
</div>
</body>
</html>
```

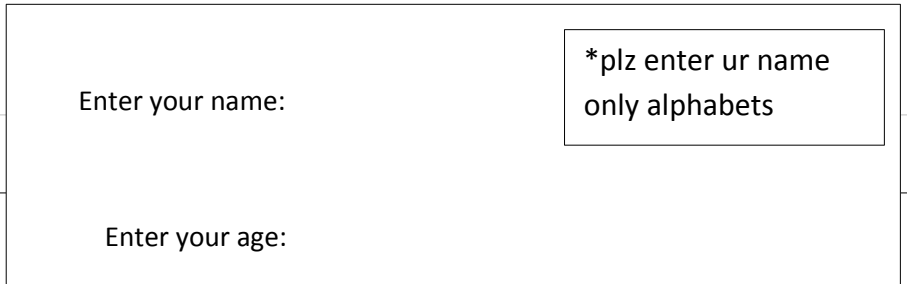
Step:4 Razor Code:

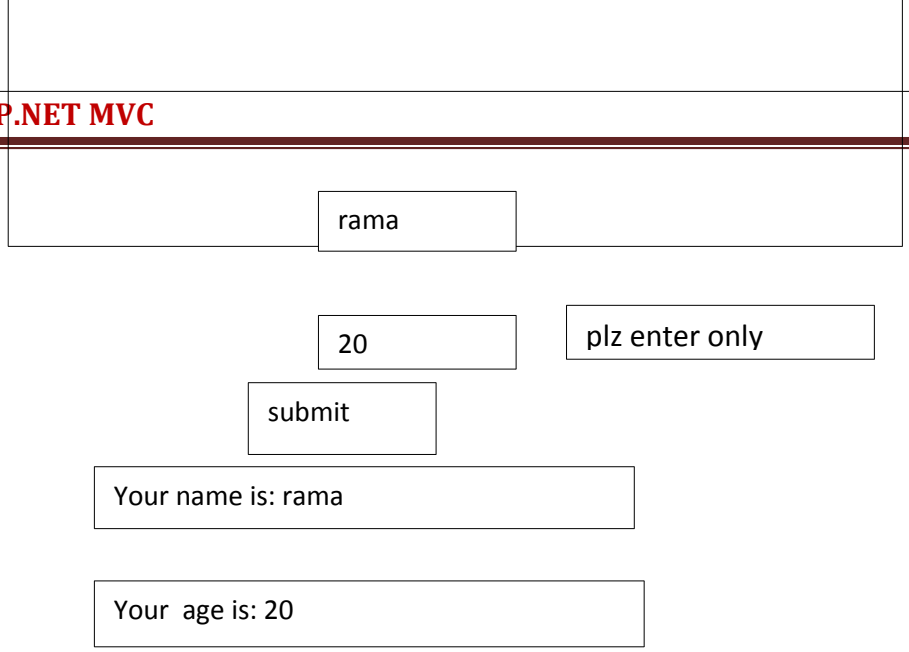
```
@{
      Layaout=null;
      String uidmsg="";
 String pwdmsg="";
If(this.post)
{
     If(Request["txtuname"].IsEmpty())
     {
       Uidmsg="*plz enter user name";
      }
    Else if(Request["txtpwd"].IsEmpty())
    {
       Pwdmsg="*plz enter password";
     }
    Else if(Request["txtpwd.length"]<6)
    {
       Pwdmsg="*password should minimum 6 characters ";
     }
  Else if(Request["txtuname"]=="Rama"&&Request["txtpwd"]=="sitha123")
    {
       Msg="valid user";
    }
   Else
   {
       Msg="invalid user";
    }
}
```

Example to accept name and age from the user and display on demand with following validations.

1.Name Should not be empty 2.name textbox should allow only letters

3.age textbox not be empty4.age textbox should allow only numbers.





Step2: Index.Cshtml [design code]

```
<html>
<body>
<form name=”Myform”  method=”post”>
<div>
 Enter your name:<input type=”text” name=”txtnmae”/>
<span>@namemsg</span>
<br/>
<br/>
Enter your age:<input type=”text” name=”txtage”/>
<span>@nagemsg</span>
<br/>
<br/>
<input type=”submit” name=”btnsubmit” value=”submit”/>
<br/>
<br/>
Your name:<span>@msg</span>
Your age: <span>@age</span>
</div>
</form>
</body>
</html>
```

Razor Code:

```
@{
      Layout=”null”;
      String namemsg=null;
      String agemsg=null;
      String name=null;
      Byte age=0;
      If(this.IsPost)
       {
          If(Request[“txtname”].IsEmpty())
            {
```

```
                Namemsg="*plz enter ur name"
              }
             Else if(Requset["txtname"].All(char.IsLetter)=false)
               {
                 Namemsg="only alphabets"
           }
         Else if(Request["txtage"].IsEmpty())
{
                  Namemsg="*plz enter ur age"
              }
             Else if(Requset["txtage"].All(char.IsDegit)=false)
               {
Agemsg="plz enter only numbers"
          }
        Else
          {
              Name=Request["txtname"];
              Age=byte.parse(Request["txtage"]);
        }
    }
}
```

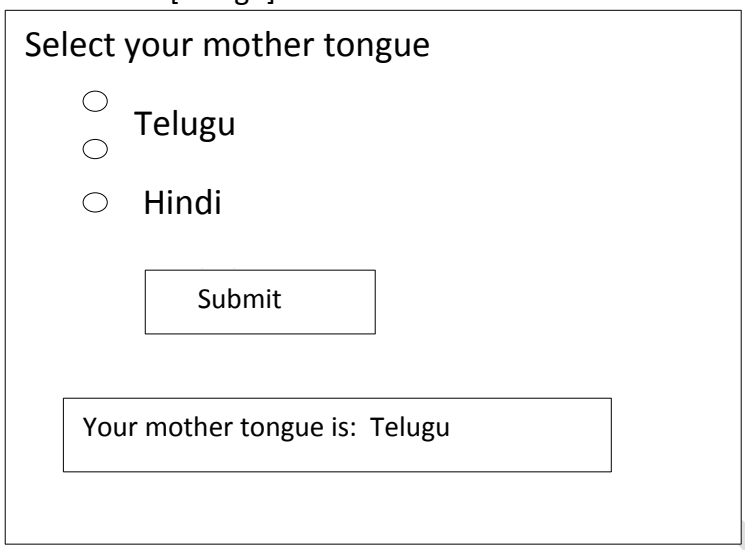
Above validations we can in following validations
with Asp.Net web forms and c#
Asp.Net web forms with JavaScript
Asp.Net Web forms with Angular Js
Implementing Radio button control in Asp.Net Mvc
How to create html Radio Button

```
<input type="radio" id="radiomale" name="gender" value="male"/>male
<br/><br/>
```

Here value property is for holding not for display

Example to display your mother tongue

Step:1 Index.cshtml[Design]



Step2: index.cshtml[code]

```
<html>
<body>
<form name=”myform” method=”post”>
<div>
<h1>select your mother tongue</h1>
<input type=”radio” name=”language” id=”radioTelugu” value=”telugu”/>TELUGU
<br/>
<input type=”radio” name=”language” id=”radioHindi” value=”Hindi”/>HINDI
<br/>
<input type=”radio” name=”language” id=”radioEnglish” value=”English”/>ENGLISH
<br/>
<input type=”Button”  id=”btnsubmit” value=”Submit”/>
</br>
<span>@msg</span>
</div>
</form>
</body>
</html>
```

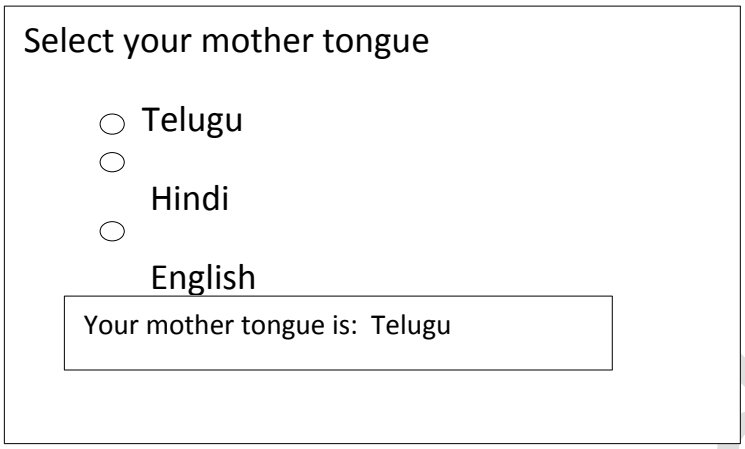
Step:3 **Razor code**

```
@{
      layout=null;
      string msg=””;
      if(this.Ispost)
     {
       If(Request[“language”].IsEmpty())
         {
           Msg=”*plz select your mother tongue”;
         }
        Else
```

```
        {
           Msg=”your mother tongue is:”+Request[“language”];
   }       }}
```

Implement above example with out submit button

Step1:index.cshtml[design]



Step2: index.cshtml[code]

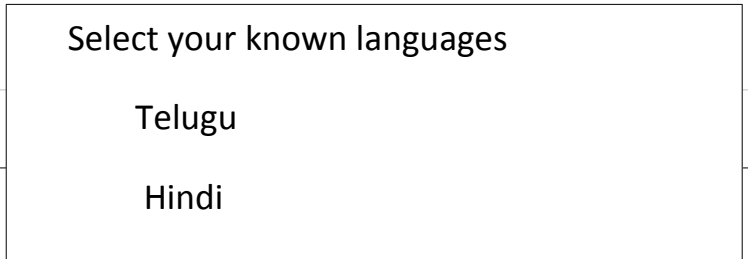
```
<html>                                    <body>
<form name=”myform” method=”post”>
<div>
<h1>select your mother tongue</h1>
<input type=”radio” name=”language” id=”radioTelugu” value=”telugu”/>TELUGU
<br/>
<input type=”radio” name=”language” id=”radioHindi” value=”Hindi”/>HINDI
<br/>
<input type=”radio” name=”language” id=”radioEnglish” value=”English”/>ENGLISH
<br/>
<span>@msg</span>
</div>
</form>
</body>
</html>
```

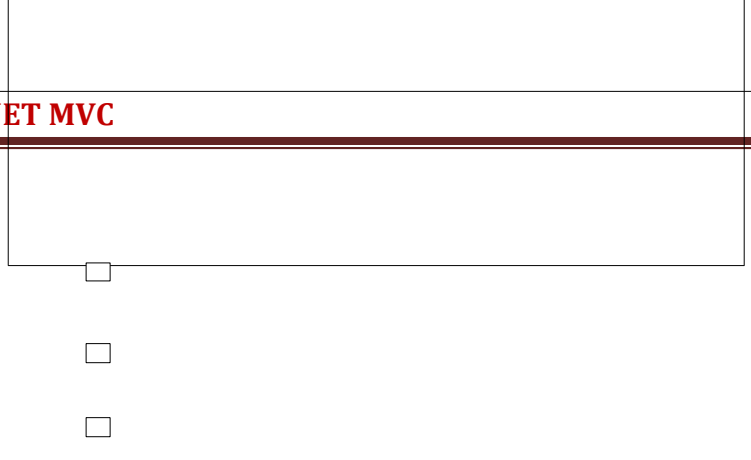
Step:3 **Razor code**

```
@{
      layout=null;
      string msg=””;
      if(this.Ispost)
       {
          Msg=”your mother tongue is:”+Request[“language”];
            }
       }
}
```

Example to select known languages by using check box control & submit button.

Step:1 Index.cshtml[Design]





Step2: index.cshtml[code]

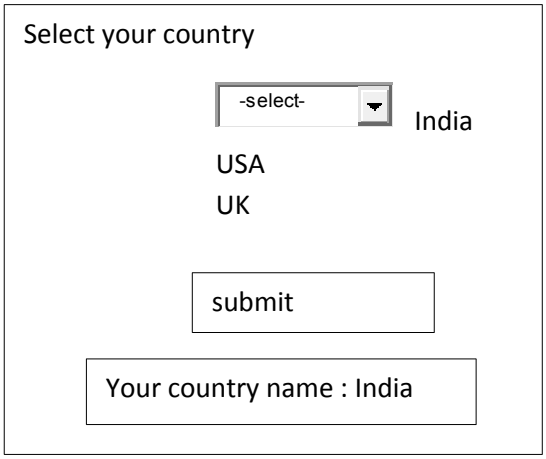
```
<html>                                    <body>
<form name=”myform” method=”post”>
<div>
<h1>select your mother tongue</h1>
<input type=”checkbox” name=”language” id=”checkTelugu” value=”telugu”/>TELUGU
<br/>
<input type=”checkbox” name=”language” id=”checkHindi” value=”Hindi”/>HINDI
<br/>
<input type=”checkbox” name=”language” id=”checkEnglish” value=”English”/>ENGLISH
<br/>
<input type=”Button”  id=”btnsubmit” value=”Submit”/>
</br>
<span>@msg</span>
</div></form></body></html>
```

Step:3 **Razor code**

```
@{
    layout=null;
    string msg=””;
    if(this.Ispost)
   {
     If(Request[“language”].IsEmpty())
      {
        Msg=”*plz select your known languages”;
      }
     Else
     {
        Msg=”your know languages are :”+Request[“languages”];
      }
    }
}
```

Example to select the country by using drop down list & submit button

Step:1 index.cshtml[design]



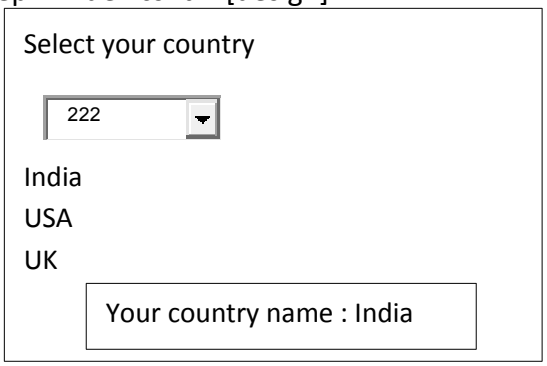
Step2: index.cshtml[design code]

```
<html>
<body>
<form type=”myform” method=”post”>
<div>
<h1> select your country<h1>
            <select name "dropdown Country list">
<option> India </option>
<option> USA<option>
<option>UK</option>
<select >
<br/>
<input type=”button” id=”btnsubmit” value=”Submit”/>
</br>
<Span> @mgs</span>
</div>
</form>
</body>
</html>
```

Step:3 Razor code

```
@{
Layout = null
       string msg="”;
if(this.Ispost)
      {
            if (Request [“ dropCountrylist”]="-select”)
            {
                  msg =" plz select your country”;
 }
Else
{
Msg=” Your County name is:” + Request [“ dropCountrylist”];
```

}
Implement above example with out submit button
Step:1 index.cshtml[design]



Step2: index.cshtml[design code]

```
<html>
<body>
<form type=”myform” method=”post”>
<div>
<h1> select your country<h1>
            <select name "dropdown Country list">
<option> India </option>
<option> USA<option>
<option>UK</option>
<select >
<br>
<Span> @mgs</span>
</div>
</form>
</body>
</html>
```

Step:3 Razor code

```
@{
Layout = null
      string msg="";
if(this.Ispost)
      {
            if (Request [“ dropCountrylist”]="-select”)
            {
                  msg =" plz select your country”;
             }
            Else
             {
                  Msg=” Your County name is:” + Request [“ dropCountrylist”];
            }
    }
```

Razor conversion Methods
1.As Bool()

2. AsDateTime()
3. AsDecimal()
4 As float()
5. As Int()

Razor comparison Methods

Razor programming will Sport followingcomparison methods

These methods will Compare the given Value, if the Value is matched then it will return "True" Value otherwise it will return false value

1. ISBool()
2. IsDateTime()
3. IsDecimal()
4. IsEmpty()
5. Isfloat()
6.IsInt()

Web Forms Vs MVC: Asp.Net webforms Vs Asp.Net Mvc.

- Here Asp.Net webforms is a Traditional web technology in .net

Asp.net Mvc is Advanced web technology in .net

In Asp.net webform we will have Serverside Controls as well as client side control but in Asp.Net MVC we well have client side control.

In Asp.netwebforms webpage Extension will be .Aspx but in Asp.Net Mvc webpage Extension will be .cshtml

A finally we can say asp.net is heavy weight programming but Asp.net mvc is light weight programming & highly testable frame work.

Asp.net mvc will support all Executing Asp.Net webform features such as master page Security & Authorization.

Data passing from Control to view

There are 2 ways to pass Data from controller to View.

1. using Viewbag
2. Using Model

1.using Viewbag:

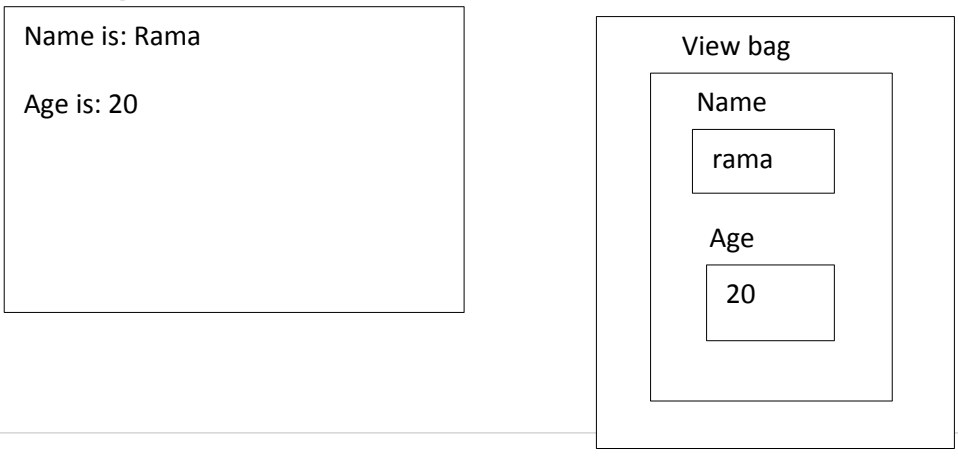
- Viewbag is a dynamic object
- using viewBag we can add any properties dynamically

Example to passing the Data from controller to view with the help of View bag

Let as assume we have 2 Values 1.name 2.age

which we will initialize with in the Control. We will pay this informationto the view

Step1: Index.cshtml view page controller



Write the below code with in home controller.cs

```
Class homecontroler
{
    Public ActionResult Index()
      {
         viewBag.name=”Rama”;
         viewBag,age=”20”;
          return view();
}
}
```

Step:3 index.cshtml [design code]

```
<Html>
<body>
<div>
        Name is: @ ViewBag.name
<br/>
<br/>
Age  is: @ ViewBag.age
</div>
</body>
</html>
```

Example to display the above controller data with the view vy using span tags.

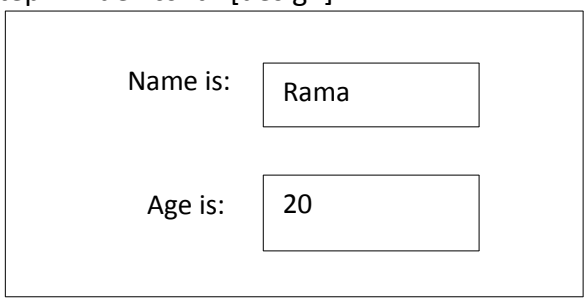
Write the below design code with the index.cshtml view page.

Page

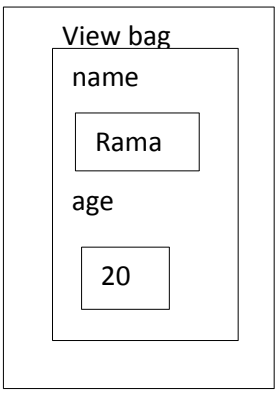
```
<html>
<body>
<div>
        name is: <span> @ ViewBag.Name </span>
Ageis: <span> @viewBag.Age </Span>
</div>
</body>
</html>
```

Example to display the above data by using Text Boxes

Step:1 index.cshtnl[design]



controller



Write the below code with in home controller.cs

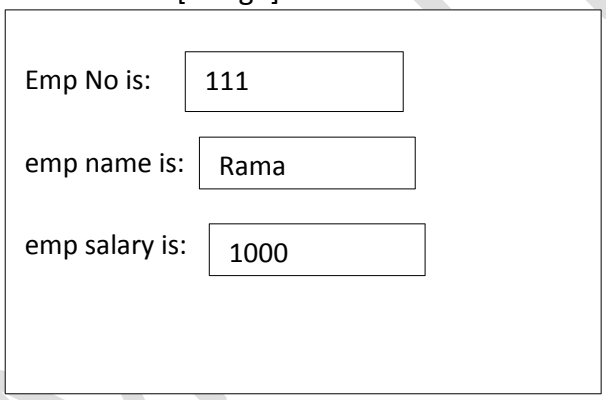
```
Class homecontroler
{
    Public ActionResult Index()
      {
         viewBag.name=”Rama”;
         viewBag,age=”20”;
          return view();
}
}
```

Step:3 index.cshtml [design code]

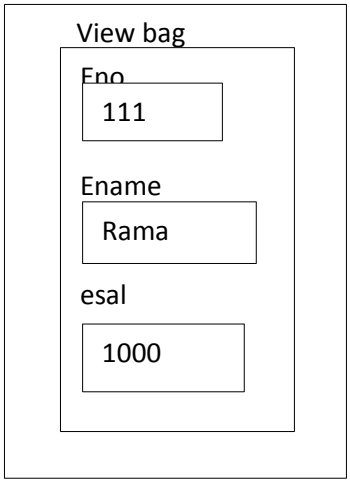
```
<Html>
<body>
<div>
        Name is: <input type=”text” id=”txtname” value=” @ViewBag.name”/>
<br/>
Age  is:<input type=”text” id=”txtage” value=”@ViewBag.age” />
</div>
</body>
</html>
```

Example to define the employee information i.e. eno=111;ename=rama ; esal= 1000 forword from the controller to view page with the help of view bag and display with in the view page by using textbox in step by step.

Step:1 index.cshtml[design]



controller



Step 2:homeController.cs

```
Class HomeController
{
   Public ActionResult Index()
{
Viewbag.eno=111;
ViewBag. Name="rama";
ViewBag.esal= 100;
return View()
}
}
```

Step3: Index.cshtml [design code]

```
<html>
<body>
<div>
Emp No: <input type="text" id= "txteno"Value="@ ViewBag.eno" />
<br/>
Emp Name: <input type="text" id= "txtename"Value=”@ Viewbag.name"/>
<br/>
Emp Salary:< input type=”text” id=”txtesal” vaue=”@viewBag.esal”/>
</div>
</body>
</html>
```

In the same way we can forword the data from Controller to View with the help of View data

Note: Viewbag & viewData Concepts will be all most all Similar.

ViewData

ViewData is an object of ViewDataDictionary" class

ViewData Syntax:

Viewdata[“id”] =10

field name

here ViewData is pre defined property which is member of Controller Base class.

This property will return “ViewDataDictionary “ object

ViewDataDictionary is a per defined class which is part of system.web.Mvc

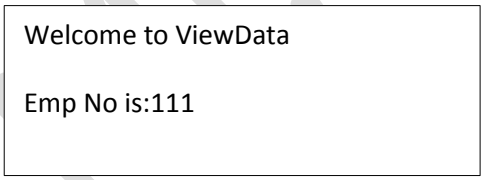
ViewData Stores the Data in key value pairs,

key is string type ,

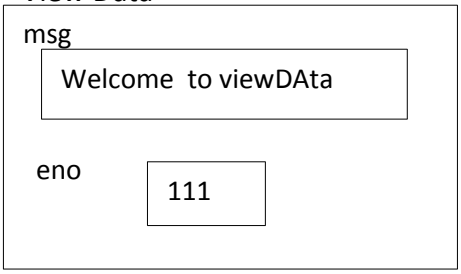
Value is Object.

Example to initialize the one string Value &numericalValue with in the ViewData (controller] & Display the Data With in the view. Finally we can Say forwarding the Datacontrol to view page with the help of ViewData.

Step :1 Index.cshtml[design]



View Data



Step:2 HomeController.cs[code]

```
class HomeController
{
Public ActionResult Index()
      {
         ViewData[“msg”]= "welcome to ViewData";
         ViewData["eno”]=111;
         return View();
       }
```

Step3: Index.csHtml[Razor code]

```
@{
Layout=null;
        String str1=Viewdata[“msg”]. Tostring();
int eid=(int)ViewData[ “eno"];
    }
```

Step:4 Index.cshtml[design Code].

```
<html>
<body>
<div>
<span> @Str</sapn>
            Empno is: <span> @eid</span>
</div>
</body>
</html>
```

Example to initialize the student information i.e. sid,sname,sloc with in the controller with the help of ViewData and forward this data to view page to display.

Step 2: HomeController.cs

```
 Class HomeController
  {
    Public ActionResult Index()
      {
        ViewData[“sid”]=”111”;
        ViewData[“sname”]=”rama”;
        ViewData[“sloc”]=”hyderabad”;
         Return view();
       }
}
```

Step:3 Index.Cshtml[Razor code]

```
@{
      Layout=null;
      Int sid=(int)viewdata[“sid”];
      String name=viewdata[“sname”].ToString();
      String loc=viewdata[“sloc”].ToString();
}
```

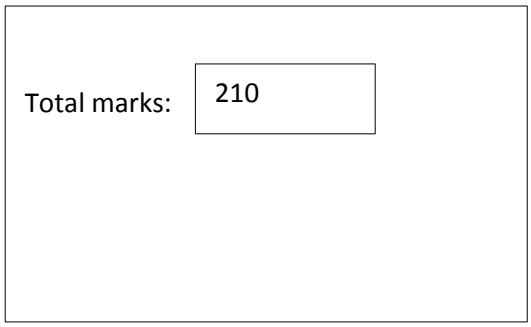
Step :4 index.cshtml[design code]

```
<html>
<body>
<div>
          Student id<span>@sid</span>
</br>
        Student name:<sapn>@sname</span>
</br>
         Student location<sapn>@sloc</span>
</div>
</body>
```

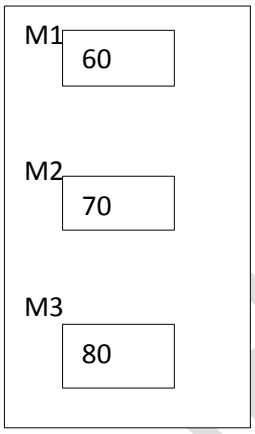
</html>

Example to initialize student 3 subject marks m1,m2,m3 with the help of view data with in the controller and calculate total marks with in the razor finally display totmarks with in the view with in the text box.

Step:1 Index.cshtml[design]



view data



Step 2:HomeController.cs[design code]

```
Class HomeController
 {
    ViewData[“m1”]=60;
    ViewData[“m2”]=70;
    ViewData[“m3”]=80;
    Return view();
}
```

Step 3: Index.Cshtml[Razor code]

```
@{
       Layout=null;
        Float m1marks=convert.ToString(ViewData[“m1”]);
       Float m2marks=convert.ToString(ViewData[“m2”]);
       Float m3marks=convert.ToString(ViewData[“m3”]);
       Float totmarks=m1marks+m2marks+m3marks;
  }
```

Step 4: Index.cshtml[view page code]

```
<html>
<body>
<div>
          Totmarks: <input type=”text” id=”totmarks” value=”@totmarks” />
</div>
</body>
</html>
```

TempData:using tempdata we can forward the data from controller to view. Tempdata stores the data as a pair, which contains Key - value

Key type is staring and value type is object.

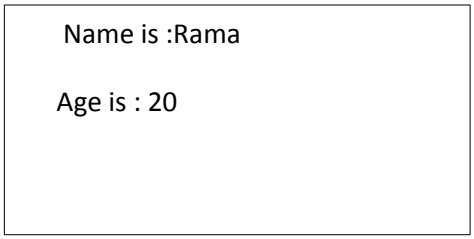
What is the difference between view Data/ViewBag and TempData.

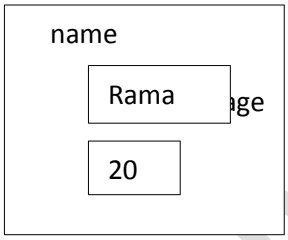
Viewdata /viewData is available only action method of current controller class and corresponding view.

But tempdata is available in current and redirect action class.

Example to forward the data from controller to view with the help of tempdata.

Step:1 Index.cshtml[Design]





Step:2 write the below code with in HomeController.cs file

```
Class HomeController:
{
Public ActionResult Index()
   {
      TempData[“name”]= ”rama”;
      TempData[“age”]= 20;
       return view();
  }
}
```

Step:3 Razor Code with in Index.cshtml

```
  @{
        Layout=null;
        String myname=(string)TempData[“name”];
        byte myage=(byte)TempData[“age”];
}
```

Step:4 Write the below view page code with in index.cshtml div section

```
<html>
<body>
<div>
          Name is: <span>@myname</span>
<br/><br/>
          Age is: <span> @myage </span>
</div>
</body>
</html>
```

TempData[“name”]=”rama”;

Here name is key, rama is value.

Here TempData is a predefined property which will return TempDataDictionary object.

Here TempDataDictionary is a predefined class which is part of system.web.MVC base class library.

Using Model:

In this technique we can forward the data from model to Controller and Controller to view

How to forward data from model to controller and controller to view

We can implement this by using following steps.

Step:1 Adding model class.

Step:2 Defining required property with in the model class.

Step:3 Adding Controller.

Step:4 Importing the model with in the controller

Step:5 Creating object of model class and pass same object to view

Step:6 With in the view Dispaly the Data

Example to forward the data from model to controller and controller to view

Step:1 Adding model class

- Select models folder→Right click →Add→Class

Here select a template called class rename it as contactInfo.cs click Add

With this process under models folder one file will add i.e. Controller.cs

Step:2 Defining the properties with in the contactInfo.cs file

```
Public class ContactInfo
{
    Public string name{ get; set; }
    Public string age{ get; set; }
}
```

Step:3 Adding Controller

Step:4 Importing the model with in the controller

Step:5 Creating object of model class

write the below code with in the HomeController.cs file

```
using UsingModelExample.Models;
name HomeController
{
 Public ActionResult Index()
  {
    //creating object for model class
    ContactInfo obj = new contactInfo
    {
       name = “rama”,
      age =20
     };
   //forwarding model object to view with the help of ViewBag.
   ViewBag.message=obj;
   Return view();
  }
}
```

Step:6 Display the data with in the view Razor Code

```
  @{
        Layout=null;
        Var data=ViewBag.message;
    }
```

Viewpage code:

```
<html>
```

output is:

Name is : Rama

Age is : 20

```
<body>
<div>
        Name is: <span>@data.name</span>
</br></br>
        Age is: <span>@data.age </span>
</div></body></html>
```

Properties with view page class:-

View page is nothing but user interface.
Every viewis Representing as a user define class.
Every view page class we have an supper class i.e. Webview Page
which is part of a base class library System.web.MVC with in the view page we can access all the following property which are defining by the micro soft with in the WebviewPage class
They are 1.Ajax
2. HTML
3. Model
4. TempData
5. URL
6.ViewBag
7. ViewContext
8.ViewData

1.Ajax:
used to set and get Ajax helper Classes
2.**Html**:
used to Set and get html controls helper class
3.**Model:**
used to Set model property of the associated view Data Dictionary object
4.TempData:
To Setand get the Tempdatawhich is required to pass to the view.
5.URL:
Used to set and get the URL for the render page
6.ViewBag :
To set or get the template required to the pass to the view
7.ViewContext:
Used to Set or get the information i.e. used to display with in the view. The information like from tempdata or viewData.
8.ViewData:
Used ta set or get the key values pairs in the form of dictionary i.e. used to pass from controller to view.
In previous example to pass the data from controller to view we are implementing 3 techniques 1.ViewData
2.ViewBag
3.TempData
1.ViewData
View Data Variables arestore with in the object.
To access view data Variables with in the Viewpage we can usede 3 ways Like bellow.
1. @ViewData[“keyname”]

2.@ViewContext.Viewdata[“keyname”]
3@ViewContext.Controller.Viewdata[“keyname”]

- Lifetime of view Data Variable is only one request i.e. Current request.
- After current request viewdata Variables are destroyed Automatically

Example to understand view Data Variable Life time.
Step 1 HomeController. Cs [code]
Class HomeController:

```
{
    Public ActionResult Index()
     {
        ViewData[“a”]=10;
        ViewData[“b”]=10.5;
        ViewData[“c”]=”rama”;
        return view();     }
[HttpPost]
  Public ActionResult Index(String x)
   {
    ViewData[“x”]=”Welcome to View Data”;
     return view();
    }
 }
```

Step 2: Write the below code with in the index.cshtml [view page] div session

```
<html>
<body>
<form id=“MyForm” method=” post”>
<p> a value is: @viewData[”a”]/p>
<p> b Vale is:@viewData[”b”]</p>
<p>c value is: @viewData[”c”]</p>
<p>x value is: @viewData[”x”]</p>
< input type="Submit" id="btnSubmit” value=”submit” />
</div>
</form>
</body>
</html>
```

When we run the abovepage theoutputwill be like below

a value is: 10

b value is: 10.5

c value is: rama

x value is:

SUBMIT

a value is:

b value is:

c value is:

x value is: welcome to view data

SUBMIT

Observation: a,b,care the view data variable are generated the first request. But when we click the Submit button post backrequest is generated a, b, c variables are destroyed because view data variables life time is only up to one request.

Implement the above example to access the view data by using 2nd syntax

@ViewContext.Viewdata[“keyname”]

Implement the above example to access the view data by using 3rdsyntax

@ViewContext.Controller.Viewdata[“keyname”]

Temp Data:

- Tempdata is almost all Similar to viewdata. But lifetime of Tempdata variable is one usage i.e. once we use Tempdata variable, then it is not available for next request or response.
- A template variable will be maintained among multiple requests if it is not used
- TO Create the tempdata variable, we will use the following mechanism
- <syntax>
  Tempdata[“keyname “]=Value
- we can access the Template variable from the view in following ways
  - ✓ @empdata["Keyname"]
  - ✓ @ ViewContext.TempDate['Keyname"]
  - ✓ @viewcontext.Controller.TempData["Keyname"]

Example for TempData:

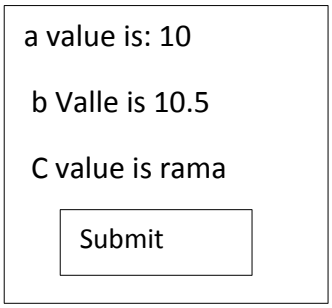
Step 1: Home Controller.cs

```
class Homecontroller.cs: conboller
{
 public ActionResult Index( )
{
    TampData[“a”]=10;
TempDate[“b"]=10.5;
TempData [“c”]=”rama”;
     return View();
}
 [Http Post]
 Public ActionResult Index(string s)
{
return view();}
```

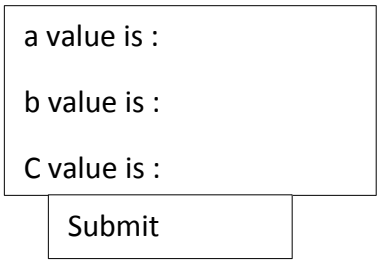
Step 2: write the below code within the Index.cshtml div section

```
<html>
<head></head>
<body> form name=”myform” method="Post”>
<div>
<p> a value is :@TempData[“a”]< /P>
<p>b value is : @TempData[“ b”] </p>
<P>c Value is : @TempData[ "c”]</p>
<input type=”submit” name=” btnSubmit” Value="submit" />
</div>
</form>
</body></html>
```

when we run the above application, the output will be like below,



when user click the Submit button, a,b, C values are not available and this outputwill be like below



Because, a, b, c variables are used accessed in first request immediately which are destroyed. Due to that reason, those variables are not available for second request (postBack request)

Example to see the value of TempData will be available for all consequence request, until its first use.

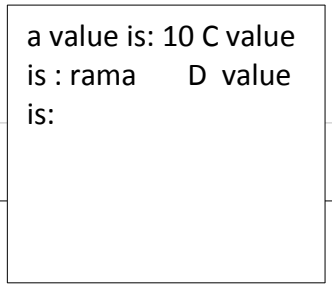
Update the below Code within the Home Controller.cs file

```
[httppost]
Public ActionResult Index[string s]
{
  viewData["d"]= TempData[“b"];
  return view();
}
```

Update the belowcode whithin Index.cshtml

```
<html>
<body>
<form Name= "myform " method="post”>
<div>
< P>a value is :@TempData[“a”]</p>
<P>c value is :@ Temppata [“c”]</p>
<P>d value is :@viewData[“d”]</p>
<input type=”submit” name=”submit" value="submit"/>
</div>
</form>
</body>
</html>
```

when we run the above page,the output is like below



Submint

Here it is not printing the d value, because d variable is not available forfirstrequest.
Observation : in this request, b Variable is not accessed which has a value called 10.5.so, it will be available for next request.
In the above problem, within the postBack method, we assigned b value into another variable Called d like below
Viewdata["d"]=TempData["b"];
Due to that reason b value will be exist for second request[Postback request].

Now click the Submit button, it will display belowoutput:

A value is :

C value is:

d value is : 10.5

Submit

HTML Property: This HTML Property of a viewpage is helpful to create HTML controls

- HTML helper controls are to similar HTML controls or asp.net webform Controls
- Here, HTML helper Controls are capable of binding with data from the model directory
- HTML Controls are light weight controls Compared with asp.net web form control
- Html helper controls do not support event handling
- HTML helper Controls do not Support any view state (or) controlstate.
- In Most of the cases,HTML Controls are just method which return stings
- we Can create custom Html helpers by using the built io html helpers.

List of built in time helpers:

- @Html. Text Box()
- @ Html. Password()
- Html.TextArea
- @Html.CheckBox()
- @Html. Radio Button()
- @Html. Hidden()
- @Html.Label()
- @Html. Actionlink()
- @Html. DropDownList()
- @Html. listBox()

- @ Html.Beginform() and so on...

Example to work with html helper lable control and-textbox Control

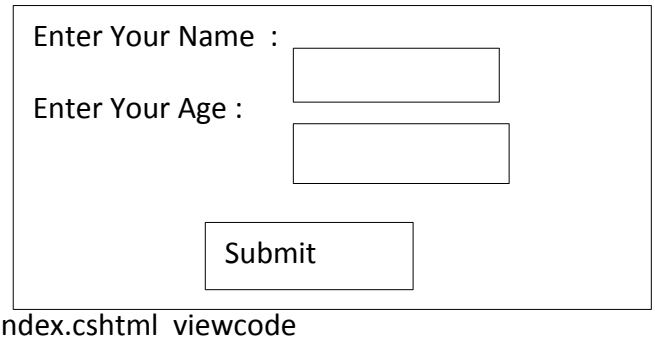
< Syntax for Label>

@Html.lable("<expression>","<lable text>");

<Syntax for html helper textbox Control>

@ Html.Textbox("name")

Step:1 Index.cshtml



Step :2 Index.cshtml viewcode

```
< htm>
<body>
<form id="myform" method="post">
<div>
 @Html.Label("lblname", "Enter you name);
 @Html.TextBox("textName");
<br/><br/>
@Html.Label("lblage","enter your age")
@Html.TextBox("txtage")
<br/><br/>
<input type="submit" id="btnsubmit" value="submit"/>
</div></form></body></html>
```

Example to select your gender

step:1

Step:2 Index .cshtml view page code

```
<html>
<body>
<form name=”myform” method=”post”>
@Html.Label(“lblgender”,”select ur Gender” )
<br/></br>
@Html.RadioButton(“gender”,”Male”)Male
<br/></br>
@Html.RadioButton(“gender”,”FeMale”)FeMale
<br/></br>
<input type=”submit” id=”btnSubmit” value=”Submit” />
</div>
</form>
</body>
</html>
```

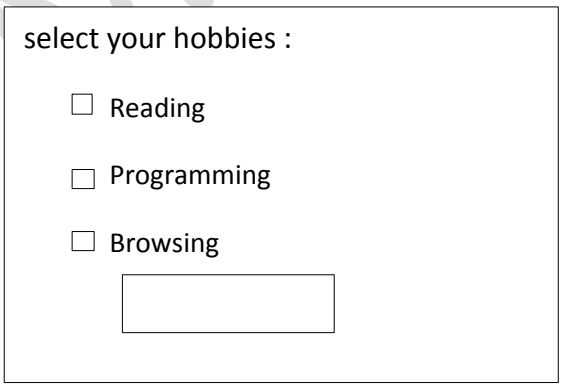
Example for Checkbox

<Syntax >@ Html.checkBox(<name>)

example for Html helper checkbox

Step1:

Index.cshtml [Design]



Step 2: index.htm [view page Code]

```
<html>
<body>
```

```
<form name= "myform" method="post">
<div>
 @Html.label("lblHobbies"," select your hobbies")
<br/><br/>
 @Html.checkBox("hobbies","Reading") Reading<br/><br/>
@Html.checkBox("hobbies ","Browsing") Browsing<br/><br/>
@Html.checkBox("hobbies","programming") programming<br/><br/>
< Input type="submit" id="btnsubmit" value="submit"/><br/><br/>
@Html.Lable("lblmsg",@msg)
</div>
</form>
</body>
</html>
```

Example for dropdownlist control:

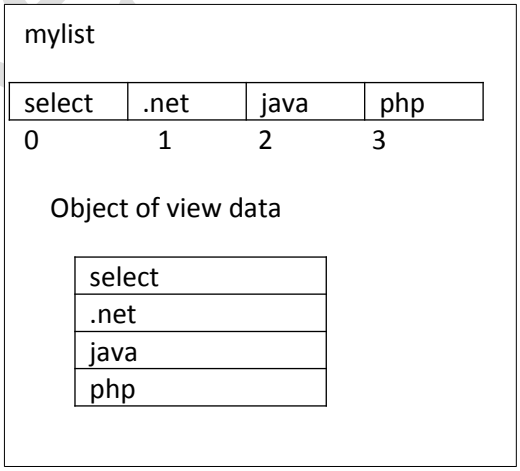
<syntax> for Dropdownlist:-

```
@Html.DropdownList("d1")
```

Example to forward the items from controller to view and within the view, bind to the dropdownlist

Step1:- Index.cshtml[Design]

HomeControler



Step2:- Index.cshtmlviewpage code

```
<html>
<body>
<form id="my form" method="post">
<div>
@Html.label ("lblcourse","select your coursename")
@Html.DropDownList("d1")
<br/><br/>
<input type = "submit" id="b+nsubmit" value="submit"/>
</div>
</form>
</body>
</html>
```

Step3:-HomeController.cs code

```
Class HomeController
{
    Public ActionResultIndex()
    {
```

```
                    List<string>mylist = new list <string>
                    {select, ".Net","Java", "Php" };
                    ViewData["d1"] = new SelectList(mylist);
                    return view();
                    }
                }
```

Syntax for dropdownlist helper control

```
    @Html.DropdownList("name",new List<SelectListItem>)
    {
        newSelectListItem {Text = "Display value", value="Holding Value"};
        newSelectListItem {Text = "Display value", value="Holding Value"};
}
```

Example to bind the dept Names to the dropdownlist and holding the deptNo's within the viepage.

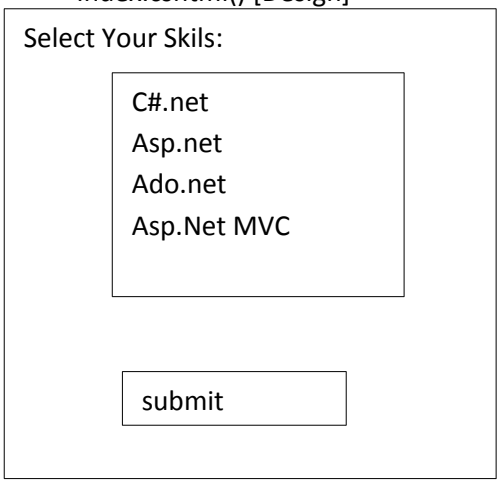
Step1:- Index.cshtml[DEsign]

Step2:- Index.cshtmlviewpage code

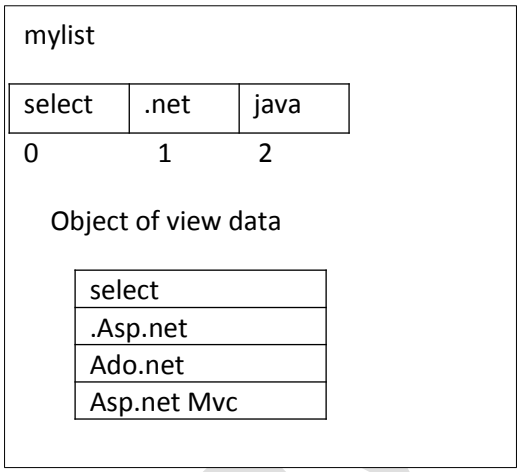
```
    <html>
    <body>
    <form name = "myform" method = "post">
    <div>
@Html.label("lbldept","select your Dept Name")
@Html.dropdownlist ("d1", new List <SelectListItem>)
    {
    newSelectListItem{Text = "HR", value = "10" };
    newSelectListItem{Text = "Technical", value = "20" };
    newSelectListItem{Text = "Marketing", value = "30" };
    }, "-select-")
    <br/><br/>
    <input type = "submit" id = "btnsubmit" value = "submit"/>
    </div>
    </form>
    </body></html>
```

Example for html helper ListBox control by getting the values from controller to view page.

Step1:- Index.cshtml() [Design]



Home.Controller



Step2:- Index.cshtml code

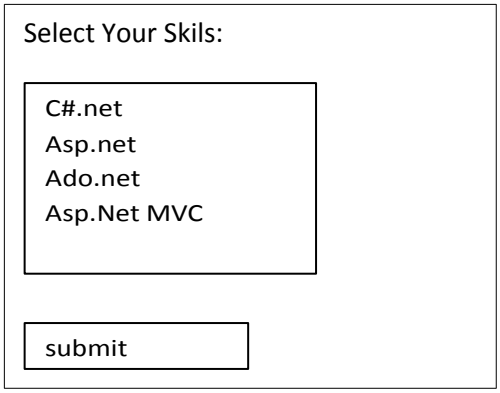
```
<html>
<body>
<from name = “myform” method = “post”>
<div>
@Html.Label (“lblskill”, “select or skillset”)
@Html.ListBox (“li”)
<br/><br/>
< input type = “submit” id = "btnsubmit" Value=”submit/>”
</div>
</form>
</body>
</html>
```

Home controller.cs:-

```
class Home Controller
{
publicActionResultIndex()
{
 List <string>mylist = new list <string>{ “c#.net”, “Asp.Net”, “ADO.Net”, “Asp.net MVC” };
viewData[“l1”] = new selectList(mylist);
return view();
}
}
```

Implement the above example to bind the items to the List box within the viewpge.

Step1: Index.cshtml [Design]



Step2: Index.cshtml view page code

```
<html>
<body>
<form name=”myform” method=”post”>
<div>
```

Syntax for label

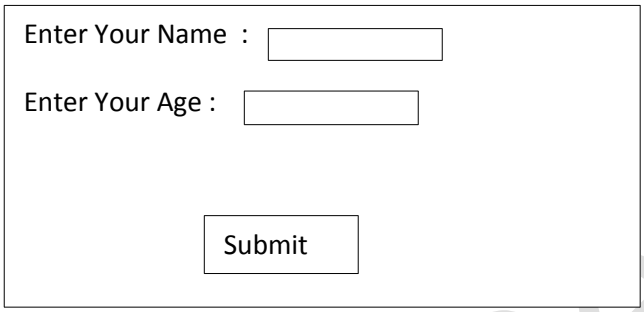
@Html.Label(“<expression>”,”<labelText>”)

Syntax for Html helper TextBox Controller:

@html.TextBox(“name”)

Example to work with helper label control and textbox controller.

Step:1 Index.cshtml[Design]



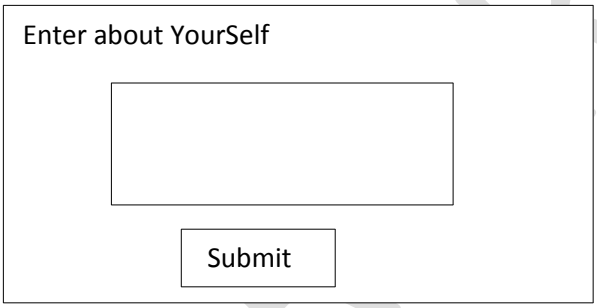
Step: 2 Razor Code:

```
@{
   Layout = null;
   string Name = "";
   int age =0;
   string msg= "";
   string age1 = "";
   if (this.IsPost)
   {
      if (Request["txtName"].IsEmpty())
      {
         msg= "*plz enter ur name";
      }
      else if (Request["txtage"].IsEmpty())
      {
         age1 = "plz enter ur age";
      }
      else if(Request["txtName"].All(char.IsLetter)==false)
      {
         msg = "plz ente only alphabets";
      }
      else if(Request["txtage"].All(char.IsDigit)==false)
      {
         msg = "plz ente only  numbers";
      }
      else
      {
```

```
        Name = Request["txtName"];
        age = Request["txtAge"].AsInt();
      }
    }
  }
<!DOCTYPE html>
<html>
<body>
<form name="myform" method="post">
<div>
  Enter ur Name: <input type="text" name="txtName" /><span>@msg</span><br /><br />
Enter ur Age: <input type="number" name="txtage"/><span>@age1</span><br /><br
/><input type="submit" name="btnsubmit" value="submit"/><br /><br />
<span>@Name </span><br /><br />
<span>@age</span>
</div>
</form>
</body>
</html>
```

Example for Html Helper text area controls[multi Line Text Box]

Step:1 Index.Cshtml:

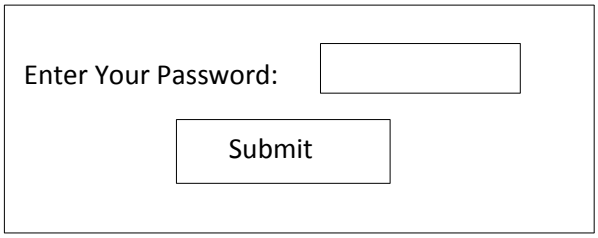


Index.cshtml [View Page Code]

```
<html>
<body>
<form id=”myform” method=”post”>
<div>
Enter About YourSelf : @Html.TextArea(“txtArea”,val s,15,null)<br/><br/>
<input type=”Submit” value=”Submit” id=”btnSubmit”/>
</div>
</form>
</body>
</html>
```

Example for html helper password textbox

Step :1 Index.cshtml[Design]

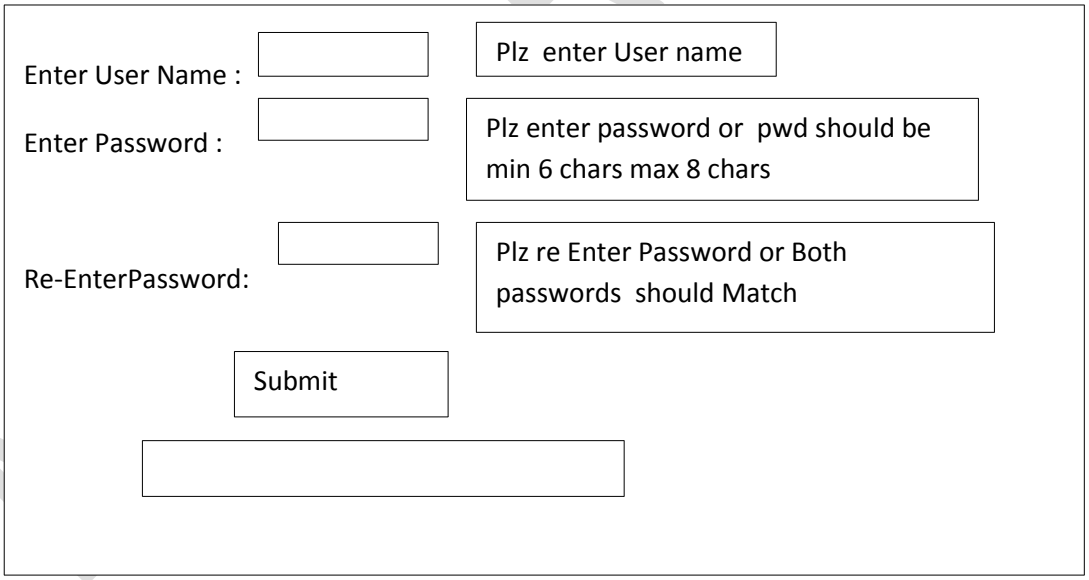


Step 2: Index.cs[View Page code]

```
<html>
<body>
<form id=”myform” method=”post”>
<div>
Enter Your Password : @Html.password(“txtpwd”) </br></br>
<input type=”Submit” id=”btnSubmit” value=”Submit” />
</div>
</form>
</body>
</html>
```

Example to implement list of validations for the following requirements.

Step1: Index.cshtml[Design]



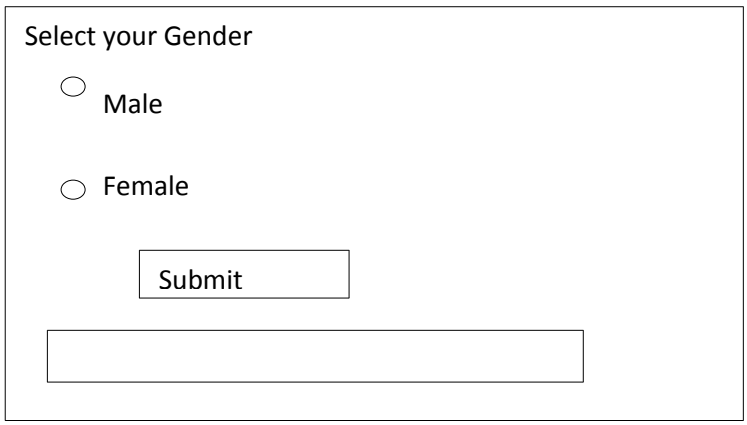
Step 2: Index.cshtml[view page code]

```
@{
   Layout = null;
   string Uname = "";
   string pwd = "";
   string repwd = "";
   string msg = "";
   if(this.IsPost)
```

```
  {
     if(Request["txtName"].IsEmpty())
     {
        Uname = "*plz enter user name";
     }
     else if(Request["txtpwd"].IsEmpty())
     {
        pwd = "*plz enter password";
     }
     else if(Request["txtpwd"].Length>=6 && Request["txtpwd"].Length <= 8)
     {
        pwd = "*password should be max 6 chars and min 8 chars";
     }
     else if(Request["txtRepwd"].IsEmpty())
     {
        repwd = "plz Re-enter password";
     }
     else if (Request["txtpwd"]==Request["txtRepwd"])
     {
        repwd = "* both passwords should match";
     }
     else
     {
        msg = "valid user";
     }
  }
  }
<!DOCTYPE html>
<html>
<body>
<form id="myform" method="post">
<div>
        Enter User Name: @Html.TextBox("txtName")
  @Html.Label("lblName",Uname)</br><br/>
Enter password: @Html.Password("txtpwd")
    @Html.Label("lblpwd",pwd)   <br /><br />
        Enter ReEnter Password: @Html.Password("txtRepwd")
        @Html.Label("lblrepwd",repwd)   <br /><br />
<input type="button" id="btnsubmit" value="submit" /><br /><br />
@Html.Label("lblmsg",msg)
</div>
</form>
</body>
</html>
```

Example to validate the html helper radio button

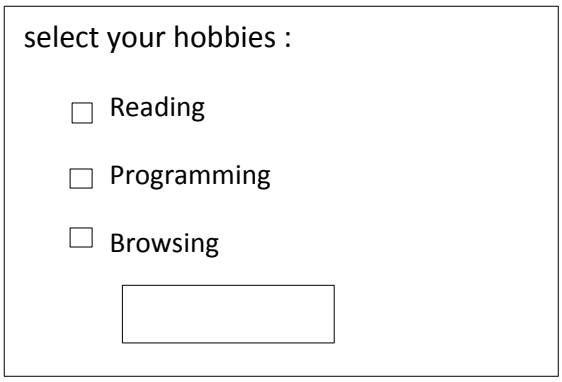
Step:1 Index.cshtml[Design]




```
@{
    Layout = null;
    string msg = "";
    if (this.IsPost)
    {
        if (Request["gender"].IsEmpty())
        {
            msg = "plz select gender";
        }
        else
        {
            msg = "your gender is: " + Request["gender"];
        }
    }
}
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form id="myform" method="post">
<div>
        @Html.RadioButton("gender", "male")Male</br></br>
        @Html.RadioButton("gender", "female")Female</br></br>
<input type="submit" value="submit" id="bynsubmit" /></br></br>
@Html.Label("lblmsg", @msg)
</div>
</form>
</body>
</html>
```

example for Html helper checkbox

Step1:Index.cshtml [Design]



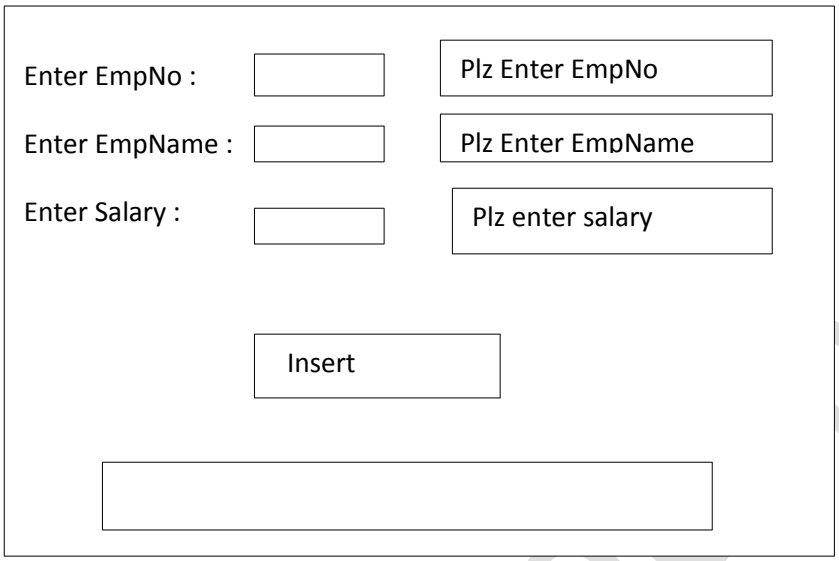
Step 2: index.htm [view page Code]

```
@{
   Layout = null;
   string msg = "";
   if(this.IsPost)
   {
      if(Request["hobbies"].IsEmpty())
      {
         msg = "Plz select your hobbies";
      }
      else
      {
         msg = "your hobbies are : " + Request["hobbies"];
      }
   }
}
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form id="myform" method="post">
<div>
<h1>select Your Hobbies</h1>
        @Html.CheckBox("hobbies", "reading")Reading<br /><br />
        @Html.CheckBox("hobbies", "coding")Coding<br /><br />
        @Html.CheckBox("hobbies", "browsing")Browsing<br /><br />
<input type="submit" id="btnsubmit" value="Submit" /><br /><br />
        @Html.Label("lblmsg", @msg)
</div>
</form>
</body>
</html>
```

Ado.Net in Asp.Net MVC

Example to insert a record in to Emp table with the help of Ado.Net

Step :1 Index.cshtml [Design]



Step 2: implement Model

- Open solution expolerer , select model folder right click Add class rename it as Employee-model.cs click Add button
- Write the below code with in the EmployeeModel.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Data.SqlClient;
using System.Data;
namespace Adonet_insert.Models
{
   public class insert
   {
      public int Eno{ get; set; }
      public string Ename { get; set; }
      public double Esal { get; set; }
      public int InsertIntoEmp()
      {
         SqlConnection conn = new
SqlConnection("server=.;database=mydbmvc;uid=sa;pwd=1234");
         SqlCommand cmd = new SqlCommand("insert into Emp
values(@eno,@ename,@esal)",conn);
         cmd.Parameters.AddWithValue("@eno",Eno);
         cmd.Parameters.AddWithValue("@ename", Ename);
         cmd.Parameters.AddWithValue("@esal", Esal);
         conn.Open();
         int i = cmd.ExecuteNonQuery();
```

```
                conn.Close();
                return i;
            }
        }
    }
```

Implement Controller :

Open solution explorer select controller folder right click Add controller rename it as EmpController click add button.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using Adonet_insert.Models;
using System.Data;
namespace InsertEmployee.Controllers
{
   public class EmpController : Controller
   {
      // GET: Default
      public ActionResult Index()
      {
         return View();
      }
      [HttpPost]
      public ActionResult Index(string s)
      {
         if(Request["txtEno"]=="" || Request["txtEName"]==""||Request["txtEsal"]=="")
         {
            return View();
         }
         else
         {
            insert empobj = new insert
            {
               Eno = Convert.ToInt32(Request["txtEno"]),
               Ename = Request["txtEname"],
               Esal = Convert.ToDouble(Request["txtEsal"])
            };
            ViewData["i"] = empobj.InsertIntoEmp();
            return View();
         }
      }
   }
}
```

Implement View Code:
Right click on second Index method Add view ,view name it as Index Unselect the below check Box click Add with this process index.cshtml Add
viewcode:

```
@{
   Layout = null;
   string enomsg;
   string enamemsg;
   string esalmsg;
   string msg;
   if(this.IsPost)
   {
      if (Request["txteno"].IsEmpty())
      {
         enomsg = "plz enter EmpNo";
      }
      else if(Request["txtename"].IsEmpty())
      {
         enamemsg = "plz enter EmpName";
      }
      else if(Request["txtesal"].IsEmpty())
      {
         esalmsg = "plz Enter Emp Salary";
      }
      else
      {
         int j = (int)ViewData["i"];
         if (j== 1)
         {
            msg = "Record is Inserted";
         }
         else
         {
            msg = "Record is Not Inserted";
         }
      }
   }
}
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form name="myform" method="post">
<div>
```

```
        Enter EmpNo:@Html.TextBox("txtEno")
        @Html.Label("lbleno",@enomsg) <br/><br />
        Enter EmpName:  @Html.TextBox("txtEname")
        @Html.Label("lblename", @enamemsg)<br/><br />
        Enter Salary:  @Html.TextBox("txtEsal")
        @Html.Label("lblsal", @esalmsg)<br/><br />
<input type="submit" id="btnsubmit" value="Insert" /><br/><br />
        @Html.Label("lblmsg", msg)
</div>
</form>
</body>
</html>
```

Update the controller name

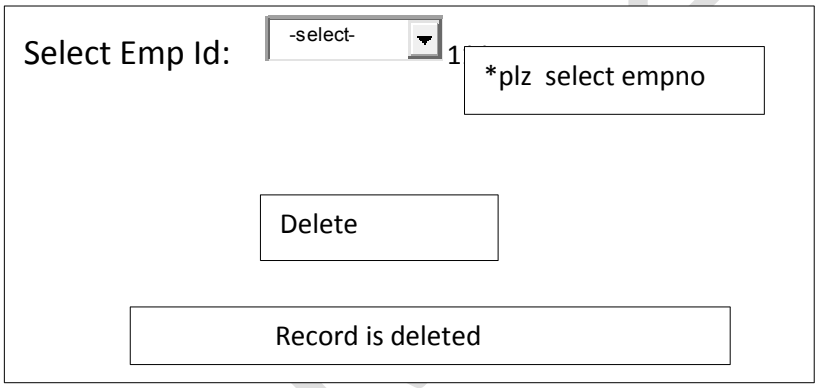
Open solution expolerer open App-start folder update the below code with in RouteConfig.cs fill

Controller=”Emp”

Example to delete a record from Emp table on selected empNo

Step :1 Design Front End and Back End

Index.cshtml[design]



Step2: the above requirements we implement into 2 tasks

Task1: Binding the Empno from EmpTable th the dropdownlist

Task2: when user click the delete vutton delete the select record from Emp table

Step 3: Model code

```
using System.Data;
namespace InsertDeleteExample1.Models
{
   public class employee
   {
      public int Eno { get; set; }
      public string Ename { get; set; }
      public double Esal { get; set; }
      SqlConnection conn = new
SqlConnection("server=.;uid=sa;pwd=1234;database=mydb");
      SqlCommand cmd;
      public DataTable GetEmp()
      {
```

```
        cmd = new SqlCommand("select empid from employee",conn);
        SqlDataAdapter da = new SqlDataAdapter(cmd);
        DataTable dt = new DataTable();
        da.Fill(dt);
        return dt;
    }
public int DeleteEmp()
    {
        cmd = new SqlCommand("delete Employee where empid=@eid",conn);
        cmd.Parameters.AddWithValue("@eid", Eno);
        conn.Open();
        int i = cmd.ExecuteNonQuery();
        conn.Close();
        return i;
    }
  }
}
```

Controller Code:

```
using System.Data;
namespace InsertDeleteExample1.Controllers
{
  public class EmpController : Controller
  {
    // GET: Emp
    internal void ForwordEmp()
    {
      employee obj = new employee();
      DataTable mydt = obj.GetEmp();
      List<int> mylist = new List<int>();
      for(int i = 0; i < mydt.Rows.Count; i++)
      {
        int no = (int)mydt.Rows[i]["empid"];
        mylist.Add(no);
      }
      ViewData["dropemplist"] = new SelectList(mylist);
    }
    public ActionResult Index()
    {
      ForwordEmp();
      return View();
    }
    [HttpPost]
public ActionResult delete()
    {
      if(Request["dropemplist"]=="")
      {
        ForwordEmp();
```

```
            return View("Index");
        }
        else
        {
            employee objdelete = new employee
            {
                Eno = int.Parse(Request["dropemplist"])
            };
            ViewData["delete"] = objdelete.DeleteEmp();
            ForwordEmp();
            return View("Index");
        }
    }
```

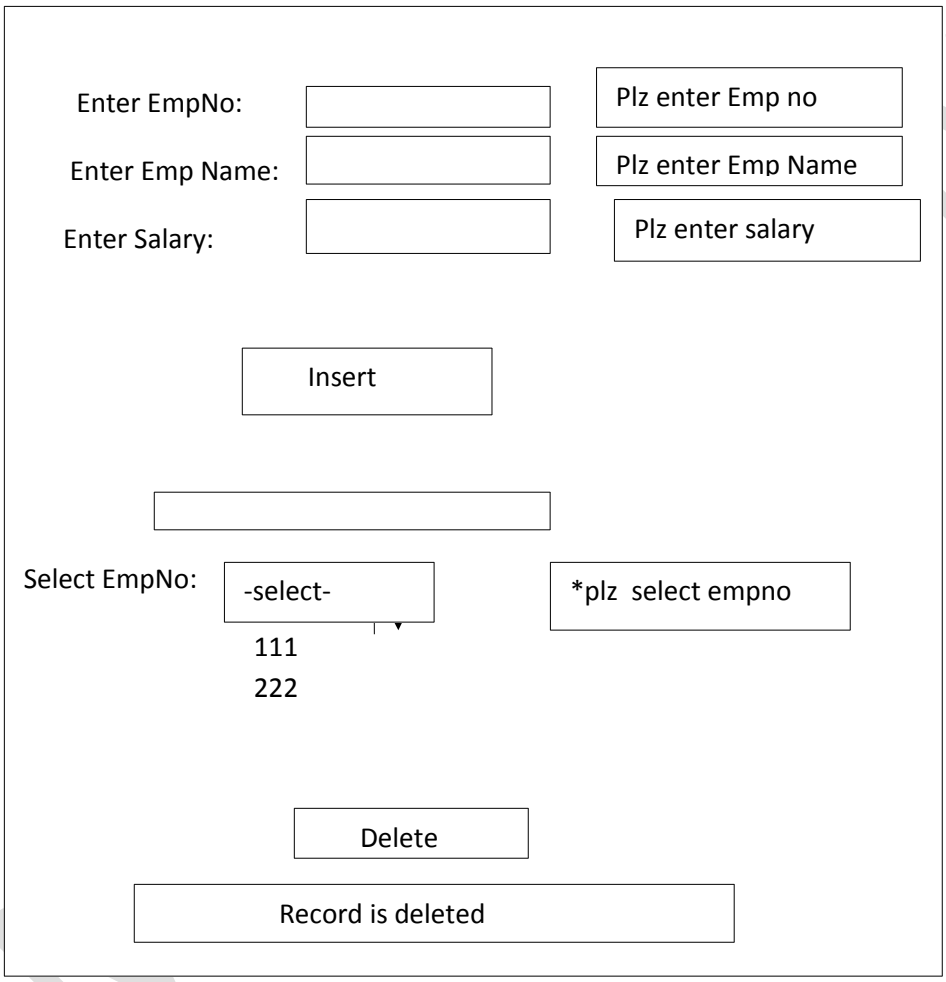
View Code:

```
@{
  string deleteeno = "";
   if (this.IsPost)
   {
       if (Request["dropemplist"] == "")
      {
         deleteeno = "* plz select Emp No...";
      }
         if (ViewData["delete"] != null)
         {
            int j = Convert.ToInt32(ViewData["delete"]);
            if (j == 1)
            {
               deletemsg = "Record is deleted";
            }
            else
            {
               deletemsg = "Record is not deleted";
            }
         }
    }
}
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form id="myform"method="post" action="Delete">
<div>
        Select Emp No: @Html.DropDownList("dropemplist", "-select-")
        @Html.Label("lblenomsg",deleteeno)<br /><br/>
```

```
<input type="submit" value="Delete" id="btndelete" /><br /><br/>
@Html.Label("lbldeletemsg", deletemsg)
</div>
</form>
</body>
</html>
```

Example to insert and delete a operations on Emp table by using single view

Step :1 Design Front End and Back End



Model Code:

```
using System.Data;
namespace InsertDeleteExample1.Models
{
   public class employee
   {
      public int Eno { get; set; }
      public string Ename { get; set; }
      public double Esal { get; set; }
      SqlConnection conn = new
SqlConnection("server=.;uid=sa;pwd=1234;database=mydb");
```

```
        SqlCommand cmd;
        public DataTable GetEmp()
        {
            cmd = new SqlCommand("select empid from employee",conn);
            SqlDataAdapter da = new SqlDataAdapter(cmd);
            DataTable dt = new DataTable();
            da.Fill(dt);
            return dt;
        }
        public int InsertEmp()
        {
            cmd = new SqlCommand("insert into employee
values(@eno,@ename,@esal)",conn);
            cmd.Parameters.AddWithValue("@eno",Eno);
            cmd.Parameters.AddWithValue("@ename",Ename);
            cmd.Parameters.AddWithValue("@esal", Esal);
            conn.Open();
            int i = cmd.ExecuteNonQuery();
            conn.Close();
            return i;
        }
        public int DeleteEmp()
        {
            cmd = new SqlCommand("delete Employee where empid=@eid",conn);
            cmd.Parameters.AddWithValue("@eid", Eno);
            conn.Open();
            int i = cmd.ExecuteNonQuery();
            conn.Close();
            return i;
        }
    }
}
```

Controller Code:

```
using System.Data;
namespace InsertDeleteExample1.Controllers
{
    public class EmpController : Controller
    {
        // GET: Emp
        internal void ForwordEmp()
        {
            employee obj = new employee();
            DataTable mydt = obj.GetEmp();
            List<int> mylist = new List<int>();
            for(int i = 0; i < mydt.Rows.Count; i++)
            {
                int no = (int)mydt.Rows[i]["empid"];
```

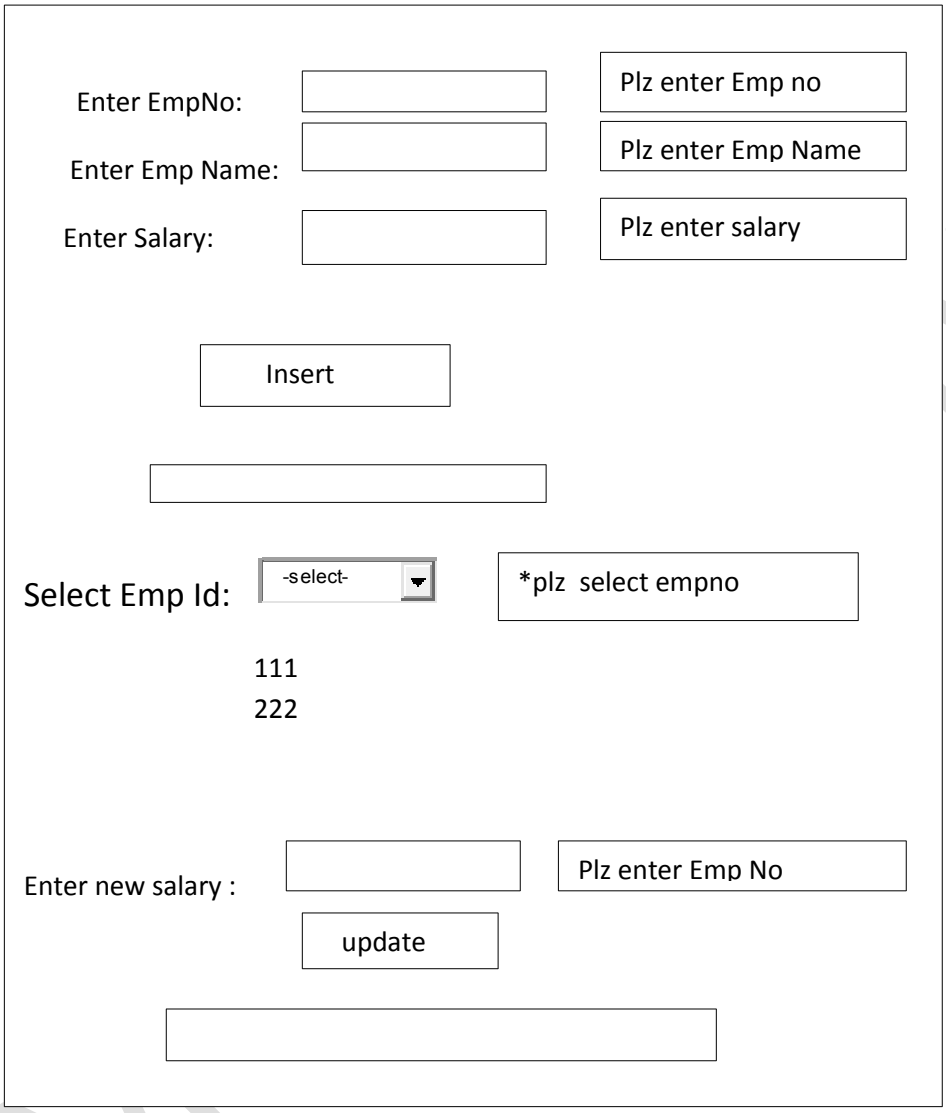
```
        mylist.Add(no);
    }
    ViewData["dropemplist"] = new SelectList(mylist);
}
public ActionResult Index()
{
    ForwordEmp();
    return View();
}
[HttpPost]
public ActionResult Insert()
{
    if (Request["txteno"] == "" || Request["txtename"] == "" || Request["txtesal"] == "")
    {
        ForwordEmp();
        return View("Index");
    }
    else
    {
        employee objInsert = new employee()
        {
            Eno = int.Parse(Request["txteno"]),
            Ename = Request["txtename"],
            Esal = double.Parse(Request["txtesal"])
        };
        ViewData["insert"] = objInsert.InsertEmp();
        ForwordEmp();
        return View("Index");
    }
}
[HttpPost]
public ActionResult delete()
{
    if(Request["dropemplist"]=="")
    {
        ForwordEmp();
        return View("Index");
    }
    else
    {
        employee objdelete = new employee
        {
            Eno = int.Parse(Request["dropemplist"])
        };
        ViewData["delete"] = objdelete.DeleteEmp();
        ForwordEmp();
   return View("Index")      } }
```

View Code

```
@{
   Layout = null;
   string enomsg = "";
   string enamemsg = "";
   string esalmsg = "";
   string deletemsg = "";
   string insertmsg = "";
   string deleteeno = "";
   if (this.IsPost)
   {
      if (Request["txteno"] == "")
      {
         enomsg = "* plz enter Emp no...";
      }
      else if (Request["txtename"] == "")
      {
         enamemsg = "*plz enter Emp Name...";
      }
      else if (Request["txtesal"] == "")
      {
         esalmsg = "*plz enter salary";
      }
      else if (Request["dropemplist"] == "")
      {
         deleteeno = "* plz select Emp No...";
      }
      else
      {
         if (ViewData["insert"] != null)
         {
            int i = Convert.ToInt32(ViewData["insert"]);
            if (i == 1)
            {
               insertmsg = "record is inserted";
            }
            else
            {
               insertmsg = "record is not inserted";
            }
         }
         if (ViewData["delete"] != null)
         {
            int j = Convert.ToInt32(ViewData["delete"]);
            if (j == 1)
            {
               deletemsg = "Record is deleted";
```

```
            }
            else
            {
               deletemsg = "Record is not deleted";
            }
         }
      }
}
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form id="myform" method="post" action="Insert">
<div>
         Enter Emp No: @Html.TextBox("txteno")
         @Html.Label("lblenomsg", enomsg)<br /><br/>
      Enter Emp Name: @Html.TextBox("txtename")
         @Html.Label("lblenamemsg", enamemsg)<br /><br/>
 Enter Salary: @Html.TextBox("txtesal")
         @Html.Label("lblenomsg", esalmsg)<br /><br/>
<input type="submit" id="btnsubmit" value="Submit" /><br /><br/>
@Html.Label("lblinsertmsg", insertmsg)
</div>
</form><br /><br/>
<form id="myform2"method="post" action="Delete">
<div>
         Select Emp No: @Html.DropDownList("dropemplist", "-select-")
         @Html.Label("lblenomsg",deleteeno)<br /><br/>
<input type="submit" value="Delete" id="btndelete" /><br /><br/>
       @Html.Label("lbldeletemsg", deletemsg)
</div>
</form>
</body>
</html>
```

Eg: Example to insert and update operations on Emp table by using single view
Step :1 Design Front End and Back End



Model Code:

```
using System.Data;
namespace InsertDeleteExample1.Models
{
   public class employee
   {
      public int Eno { get; set; }
      public string Ename { get; set; }
      public double Esal { get; set; }
      SqlConnection conn = new
SqlConnection("server=.;uid=sa;pwd=1234;database=mydb");
      SqlCommand cmd;
      public DataTable GetEmp()
```

```
      {
         cmd = new SqlCommand("select empid from employee",conn);
         SqlDataAdapter da = new SqlDataAdapter(cmd);
         DataTable dt = new DataTable();
         da.Fill(dt);
         return dt;
      }
      public int InsertEmp()
      {
         cmd = new SqlCommand("insert into employee
values(@eno,@ename,@esal)",conn);
         cmd.Parameters.AddWithValue("@eno",Eno);
         cmd.Parameters.AddWithValue("@ename",Ename);
         cmd.Parameters.AddWithValue("@esal", Esal);
         conn.Open();
      int i = cmd.ExecuteNonQuery();
         conn.Close();
         return i;
      }
      public int UpdateEmp()
      {
         cmd = new SqlCommand("update emp set salary=@esalwhere empid=@eid",conn);
         cmd.Parameters.AddWithValue("@eid", Eno);
cmd.Parameters.AddWithValue("@esal", Esal);
         conn.Open();
         int i = cmd.ExecuteNonQuery();
         conn.Close();
         return i;
      }
   }
}
Controller Code:
using System.Data;
namespace InsertDeleteExample1.Controllers
{
   public class EmpController : Controller
   {
      // GET: Emp
      internal void ForwordEmp()
      {
         employee obj = new employee();
         DataTable mydt = obj.GetEmp();
         List<int> mylist = new List<int>();
         for(int i = 0; i < mydt.Rows.Count; i++)
         {
            int no = (int)mydt.Rows[i]["empid"];
            mylist.Add(no);
```

```
    }
    ViewData["dropemplist"] = new SelectList(mylist);
}
public ActionResult Index()
{
    ForwordEmp();
    return View();
}
[HttpPost]
public ActionResult Insert()
{
    if (Request["txteno"] == "" || Request["txtename"] == "" || Request["txtesal"] == "")
    {
        ForwordEmp();
        return View("Index");
    }
    else
    {
        employee objInsert = new employee()
        {
            Eno = int.Parse(Request["txteno"]),
            Ename = Request["txtename"],
            Esal = double.Parse(Request["txtesal"])
        };
        ViewData["insert"] = objInsert.InsertEmp();
        ForwordEmp();
        return View("Index");
    }
}
[HttpPost]
public ActionResult update()
{
    if(Request["dropemplist"]=="" || Request[“txtNewSal”]=””)
    {
        ForwordEmp();
        return View("Index");
    }
    else
    {
        employee objupdate = new employee
        {
            Eno = int.Parse(Request["dropemplist"]),
          Esal=double.parse(Request[“txtnewsal”])
        };
        ViewData["update"] = objupdate.UpdateEmp();
        ForwordEmp();
   return View("Index")
```

```
}
 }
View Code
@{
   Layout = null;
   string enomsg = "";
   string enamemsg = "";
   string esalmsg = "";
   string enoupdatemsg = "";
   string insertmsg = "";
   string updatemsg = "";
   if (this.IsPost)
   {
      if (Request["txteno"] == "")
      {
         enomsg = "* plz enter Emp no...";
      }
      else if (Request["txtename"] == "")
      {
         enamemsg = "*plz enter Emp Name...";
      }
      else if (Request["txtesal"] == "")
      {
         esalmsg = "*plz enter salary";
      }
      else if (Request["updateemplist"] == "")
      {
enoupdatemsg= "* plz select Emp No...";
      }
      else
      {
         if (ViewData["insert"] != null)
         {
            int i = Convert.ToInt32(ViewData["insert"]);
            if (i == 1)
            {
               insertmsg = "record is inserted";
            }
            else
            {
               insertmsg = "record is not inserted";
            }
         }
    if (ViewData["updatete"] != null)
         {
            int j = Convert.ToInt32(ViewData["update"]);
            if (j == 1)
```

```
            {
               updatemsg = "Record is updated";
            }
            else
            {
               updatemsg = "Record is not updated";
            }
         }
      }
}
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form id="myform" method="post" action="Insert">
<div>
        Enter Emp No: @Html.TextBox("txteno")
        @Html.Label("lblenomsg", enomsg)<br /><br/>
        Enter Emp Name: @Html.TextBox("txtename")
        @Html.Label("lblenamemsg", enamemsg)<br /><br/>
        Enter Salary: @Html.TextBox("txtesal")
        @Html.Label("lblenomsg", esalmsg)<br /><br/>
<input type="submit" id="btnsubmit" value="Submit" /><br /><br/>
@Html.Label("lblinsertmsg", insertmsg)
</div>
</form><br /><br/>
<form id="myform2"method="post" action="Update">
<div>
        Select Emp No: @Html.DropDownList("dropemplist", "-select-")
   @Html.Label("lblenomsg",updateeno)<br /><br/>
<input type="submit" value="Update" id="btnupdate" /><br /><br/>
  @Html.Label("lblupdatemsg", updatemsg)<br /><br/>
</div>
</form>
</body>
</html>
```

Eg: Example to display emp table with in html table by using Asp.netmvc
Step 1: Design of Index.cs

Employee No	Employee Name	Employee Salary
111	Rama	1000.0000
222	Sitha	2000.0000
333	ravi	3000.0000

Display

Model code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Data;
using System.Data.SqlClient;
namespace MVCHtmlTableExample1.Models
{
   public class Employee
   {
      SqlConnection conn = new
SqlConnection("server=.;database=mydbMvc;uid=sa;pwd=1234");
      SqlCommand cmd;
      public DataTable GetEmp()
      {
         cmd = new SqlCommand("select * from emp", conn);
         SqlDataAdapter da = new SqlDataAdapter(cmd);
         DataTable dt = new DataTable();
         da.Fill(dt);
         return dt;
      }
   }
}
```

Controller code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using MVCHtmlTableExample1.Models;
using System.Data;
namespace MVCHtmlTableExample1.Controllers
{
```

```
  public class DefaultController : Controller
  {
     public void ForwordEmp()
     {
        Employee empobj = new Employee();
        ViewBag.emp = empobj.GetEmp();
     }
     // GET: Default
     public ActionResult Index()
     {
        ForwordEmp();
        return View();
     }
     [HttpPost]
     public ActionResult PostData()
     {
        ForwordEmp();
        return View();
     }
  }
}
```

View code:

```
@{
    Layout = null;
}

<!DOCTYPE html>

<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form>
<div>
<table border="1">
<tr>
<th>Employee No</th>
<th>Employee Name</th>
<th>Employee Salary</th>
</tr>
                @{
                    foreach(System.Data.DataRow myrow in ViewBag.emp.Rows)
                    {

<tr>
<td>@myrow["empno"]
</td>
<td>@myrow["empname"]</td>
<td>@myrow["salary"]</td>
</tr>
                    }
                }
</table>
```

```
<input type="submit" id="btnDisplay" value="Display" />,

</div>
</form>
</body>
</html>
```

Eg: Example to delete record on emp table by using Html Table

Step 1: Index.cshtml[Design]

Employee No	Employee Name	Employee Salary	delate
111	rama	1000	Delete
222	sitha	2000	Delete
333	ravi	3000	Delete

Record is Deleted

Step 2: The above we can implement in following tasks:

Task 1: Binding the EmpTable to the Html Table.

Task 2: When user click on delete link button concern record has to deleted from the data base.

Model code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Data;
using System.Data.SqlClient;
namespace MVCDeleteExampleOnEmp.Models
{
   public class Employee
   {
      SqlConnection conn = new
SqlConnection("server=.;uid=sa;pwd=1234;database=mydbmvc");
      SqlCommand cmd;
      public int Eno{get; set; }
      public DataTable GetEmp()
      {
         cmd = new SqlCommand("select * from emp",conn);
         DataTable dt = new DataTable();
         SqlDataAdapter da = new SqlDataAdapter(cmd);
         da.Fill(dt);
         return dt;
      }
      public int deleteEmp()
      {
```

```
            cmd = new SqlCommand("delete emp where empno=@eno");
            cmd.Parameters.AddWithValue("eno", Eno);
            conn.Open();
            int i = cmd.ExecuteNonQuery();
            conn.Close();
            return i;
        }
    }
}
```

Controller code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using MVCDeleteExampleOnEmp.Models;
using System.Data;
namespace MVCDeleteExampleOnEmp.Controllers
{
    public class DefaultController : Controller
    {
        public void ForwordEmp()
        {
            Employee objemp = new Employee
            {
            };
            ViewBag.emp = objemp.GetEmp();
        }
        // GET: Default
        public ActionResult Index()
        {
            ForwordEmp();
            return View();
        }
        [HttpPost]
        public ActionResult Delete()
        {
            Employee objemp = new Employee();
            ForwordEmp();
            return View("Index");
        }
    }
}
```

View code:

```
@{
    Layout = null;
}
```

```
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form name="myform" method="post">
<div>
<table border="1">
<tr>
<th>Employee No</th>
<th>Employee Name</th>
<th>Employee Salary</th>
</tr>
          @{
            foreach(System.Data.DataRow myrow in ViewBag.Emp.Rows)
            {
<tr>
<td>@myrow["empno"]</td>
<td>@myrow["empname"]</td>
<td>@myrow["salary"]</td>
<td>
                    @Html.ActionLink("Delete", "Delete",new { }
                   )
</td>
</tr>
            }
          }
</table>
</div>
</form>
</body>
</html>
```

Eg: Example to update the Employee Salary by Using Html table UpdatingIndex.cshtml

EmployeeNO	EmployeeName	Employeesalary	Edit
111	Rama	1000.0000	Edit
222	Sitha	2000.0000	Edit
333	Ravi	3000.0000	Edit

EmoNo : 333

EmpNewsalary : 3500

Update GoBack

Salary is updated

The above requirement implement following tasks.

Task :1 Display EmpTable with in the Html Table of index.cshtml view.
Task :2 When user click edit button get the concern employee,employee details from the Database Display with in the textbox of updateindex.cshtml view the values are EmpNo And Existing salary.
Task :3 After user will enter new salary and user click update button concern employee salary has to update with in the database and display /update success message .
Task :4 when user click go back button redirect to the Index.cshtml view.

Model Code

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Data.SqlClient;
using System.Data;

namespace MvcEditEmpinAdo.Models
{
    public class Employee
    {
    SqlConnection conn = new
SqlConnection("server=.;uid=sa;pwd=1234;database=mydbmvc");
        SqlCommand cmd;
        public int Eno { get; set; }
        public string Ename { get; set; }
        public double Esal { get; set; }
        public DataTable GetEmp()
        {
            cmd = new SqlCommand("select * from Emp", conn);
            DataTable dt = new DataTable();
            SqlDataAdapter da = new SqlDataAdapter(cmd);
            da.Fill(dt);
            return dt;
        }
        public DataTable GetCurrentEmpDetails()
        {
            cmd = new SqlCommand("select empno,salary from Emp where empno=@eno",
conn);
            cmd.Parameters.AddWithValue("@eno",Eno);
            SqlDataAdapter da = new SqlDataAdapter(cmd);
            DataTable dt = new DataTable();
            da.Fill(dt);
            return dt;
        }
        public double updateSal()
        {
            cmd = new SqlCommand("update emp set salary=@sal where empno=@eno
",conn);
            cmd.Parameters.AddWithValue("@sal", Esal);
            cmd.Parameters.AddWithValue("@eno", Eno);
            conn.Open();
            int i = cmd.ExecuteNonQuery();
            conn.Close();
            return i;

        }
    }
```

```
}
```

Controller Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using System.Data;
using MvcEditEmpinAdo.Models;

namespace MvcEditEmpinAdo.Controllers
{
    public class DefaultController : Controller
    {
        internal void Forwordemp()
        {
            Employee objemp = new Employee();
            ViewBag.emp = objemp.GetEmp();
        }
        // GET: Default
        public ActionResult Index()
        {
            Forwordemp();
            return View();
        }
        public ActionResult EditIndex()
        {
            Employee ObjEmp = new Employee
            {
                Eno = int.Parse(Request.QueryString["eno"])
            };
            ViewBag.emp = ObjEmp.GetCurrentEmpDetails();
            return View("UpdateIndex");
        }
        public  ActionResult UpdateIndex()
        {
            Employee objemp = new Employee
                {
                    Eno=int.Parse(Request["txtEno"]),
                    Esal=double.Parse(Request["txtEsal"])
                };
                ViewBag.update = objemp.updateSal();
                Forwordemp();
                return View();
        }
    }
}
Note : Here we add  two views
View 1: right click on Index Action Method Add View click Add button with this
Index.Cshtml will Add,
View 2: right click on UpdateIndex Action Method Add View click Add button with
this UpdateIndex.Cshtml will Add
```

View Code: Index.Cshtml

```
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
```

```
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form name="myform" method="post">
<div>
<table border="1">
<tr>
<th>Employee No</th>
<th>employee Name</th>
<th>Salary</th>
<th>Edit</th>
</tr>
@{
                    foreach (System.Data.DataRow myrow in ViewBag.emp.Rows)
                    {
<tr>
<td>@myrow["empno"]</td>
<td>@myrow["empname"]</td>
<td>@myrow["salary"]</td>
<td>@Html.ActionLink("Edit","EditIndex",new { eno=myrow["empno"]})
</td>
</tr>
                    }
}

</table>
</div>
</form>
</body>
</html>
```

UpdateIndex.cshtml [Code]

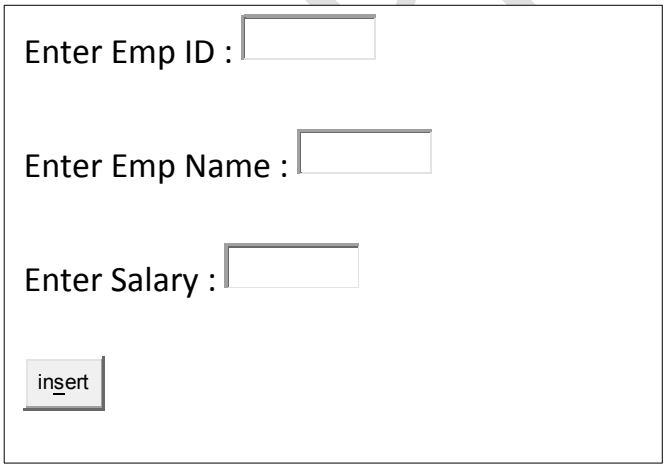
```
@using System.Data;
@{
    Layout = null;
    string eno = "";
    string esal = "";
    string esalmsg = "";
    string msg = "";
    foreach (DataRow myrow in ViewBag.emp.Rows)
    {
        eno = myrow["empno"].ToString();
        esal = myrow["salary"].ToString();
    }
    if (this.IsPost)
    {
        if (Request["txtEsal"].IsEmpty())
        {
            esalmsg = "plz Enter Emp Salary";
        }
        else
        {
            int i =ViewBag.update;
            if (i == 1)
            {
                msg = "Salary is Updated";
            }
            else
```

```
            {
                msg = "Salary is Not Updated";
            }
        }
    }
}
```

```
<!DOCTYPE html>

<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>UpdateIndex</title>
</head>
<body>
<form id="myform" method="post" action="~/Default/UpdateIndex">
<div>
            Employee No : <input type="text" value="@eno" id="txtEno" readonly />
<br /><br />
            Enter New Employee Salary : <input type="text" id="txtEsal"
       value="@esal"/>
@Html.Label("lblmsg",esalmsg)
<br /><br />
<input type="submit" name="btnupdate" value="Update" />
<input type="submit" name="btnGoBack" value="GoBack"
formaction="~/Default/Index" />
<br /><br />
@Html.Label("lblUpdate",msg)
</div>
</form>
</body>
</html>
```

Example to Insert a record into Emp Table:

Step:1 Design of front end and back end



Model Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Data;
using System.Data.SqlClient;
namespace INSERTARECORDBYUSINGHTMLTABLE.Models
```

```
{
    public class Employee
    {
        public int Eno{get;set; }
        public string Ename{get; set; }
        public double Esal{get; set; }
        SqlConnection conn = new
SqlConnection("server=.;database=mydbmvc;uid=sa;pwd=1234");
        SqlCommand cmd;
        public int insertemp()
        {
            cmd = new SqlCommand("insert into emp
values(@eno,@ename,@esal)",conn);
            cmd.Parameters.AddWithValue("@eno", Eno);
            cmd.Parameters.AddWithValue("@ename", Ename);
            cmd.Parameters.AddWithValue("@esal", Esal);
            conn.Open();
            int i = cmd.ExecuteNonQuery();
            conn.Close();
            return i;
        }
    }
}
```

Controller Code :

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using INSERTARECORDBYUSINGHTMLTABLE.Models;
namespace INSERTARECORDBYUSINGHTMLTABLE.Controllers
{
    public class EmpController : Controller
    {
        // GET: Emp
        public ActionResult Index()
        {

            return View();
        }
        public ActionResult Insert()
        {
            if (Request["txtempno"] == "" || Request["txtempname"] == ""
|| Request["txtempsal"] == "")
            {
                return View("Index");
            }
            else
            {
                Employee obj = new Employee
                {
```

```
                    Eno = int.Parse(Request["txtempno"]),
                    Ename = Request["txtempname"],
                    Esal = double.Parse(Request["txtempsal"])
                };
                ViewBag.insert = obj.insertemp();
                return View("Index");
            }
        }
    }
}
```

View Code:

```
@using System.Data;
@{
    Layout = null;
    string enomsg = "";
    string enamemsg = "";
    string esalmsg = "";
    string insertmsg = "";
    if (this.IsPost)
    {
        if (Request["txtempno"].IsEmpty())
        {
            enomsg = "pls enter empno";
        }
        else if (Request["txtempname"].IsEmpty())
        {
            enamemsg = "pls enter empname";
        }
        else if (Request["txtempsal"].IsEmpty())
        {
            esalmsg = "pls enter new salary";
        }
        else
        {
            if (ViewBag.insert != null)
            {
                int j = ViewBag.insert;
                if (j == 1)
                {
                    insertmsg = "record is inserted";
                }
                else
                {
                    insertmsg = "record is not insertred";
                }
            }
        }
    }
}
<!DOCTYPE html>
```

```
<html>
<head>
<meta name="viewport" content="width=device-width" />
</head>
<body>
<form id="myform" method="post">
<div>
Enter Emp ID : @Html.TextBox("txtempno")@Html.Label("lbleno", enomsg)
<br /><br />
Enter Emp Name :
@Html.TextBox("txtempname")@Html.Label("lblename",enamemsg)
<br /><br />
Enter Salary : @Html.TextBox("txtempsal")@Html.Label("lblesal", esalmsg)
<br /><br /><input type="submit" value="insert" name="btninsert"
formaction="Insert" />
<br /><br />
@Html.Label("lblmsg", insertmsg)
</div>
</form>
</body>
</html>
```

Example to Curd Operations (Inser,Delete,Update) by using Html Table:

Step:1 Design of Front End And Back End

Employee no	Employee name	Employee salary	DELETE	EDIT
111	Rama	1000.0000	delete	edit
222	Sitha	2000.0000	delete	edit
333	Ravi	3000.0000	delete	edit
			insert	

EmpNo	EmpName	Salary
111	Rama	1000
222	Sitha	2000
333	Ravi	3000

model:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Data;
using System.Data.SqlClient;
namespace CurdOperationsONHTMLTable.Models
{
   public class Employee
   {
      public int Eno{get;  set;}
      public string Ename{get;set; }
      public double Esal{get;set; }
```

```
    SqlConnection conn = new SqlConnection("server=.;database=mydb;uid=sa;pwd=1234");
    SqlCommand cmd;
    public DataTable GetEmp()
    {
       cmd = new SqlCommand("select *from emp", conn);
       SqlDataAdapter da = new SqlDataAdapter(cmd);
       DataTable dt = new DataTable();
       da.Fill(dt);
       return dt;
    }
    public int DeleteEmp()
    {
       cmd = new SqlCommand("delete emp where empno=@eno", conn);
       cmd.Parameters.AddWithValue("@eno", Eno);
       conn.Open();
       int i = cmd.ExecuteNonQuery();
       conn.Close();
       return i;
    }
    public DataTable GetEmpDetails()
    {
       cmd = new SqlCommand("select empno,salary from emp where empno=@eno", conn);
       cmd.Parameters.AddWithValue("@eno", Eno);
       SqlDataAdapter da = new SqlDataAdapter(cmd);
       DataTable dt = new DataTable();
       da.Fill(dt);
       return dt;
    }
    public int UpdateEmp()
    {
       cmd = new SqlCommand("update emp set salary=@newsal where empno=@eno", conn);
       cmd.Parameters.AddWithValue("@newsal", Esal);
       cmd.Parameters.AddWithValue("@eno", Eno);
       conn.Open();
       int i = cmd.ExecuteNonQuery();
       conn.Close();
       return i;
    }
    public int InsertEmp()
    {
       cmd = new SqlCommand("insert into emp values(@eno,@ename,@esal)", conn);
       cmd.Parameters.AddWithValue("@eno", Eno);
       cmd.Parameters.AddWithValue("@ename", Ename);
       cmd.Parameters.AddWithValue("@esal", Esal);
       conn.Open();
       int i = cmd.ExecuteNonQuery();
       conn.Close();
       return i;
    }
  }
}
```

controller:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using System.Data;
using CurdOperationsONHTMLTable.Models;
namespace CurdOperationsONHTMLTable.Controllers
{
   public class EmpController : Controller
   {
      internal void forwardemp()
      {
         Employee obj = new Employee();
         ViewBag.emp = obj.GetEmp();
      }
      // GET: Emp
      public ActionResult Index()
      {
         forwardemp();
         return View();
      }
      public ActionResult  DeleteCntrlEmp()
      {
         Employee obj = new Employee
         {
            Eno = int.Parse(Request.QueryString["eno"])
         };
   ViewBag.delete = obj.DeleteEmp();
         forwardemp();
         return View("Index");
      }
     public ActionResult EditEmp()
      {
         Employee obj = new Employee
         {
            Eno = int.Parse(Request.QueryString["eno"])
         };
         ViewBag.emp = obj.GetEmpDetails();
         forwardemp();
         return View("EditIndex");
      }
      public ActionResult EditIndex()
      {
         if (Request["txtnewsal"] == "")
         {
            return View();
         }
         else
```

```
        {
            Employee obj = new Employee
            {
                Eno = int.Parse(Request["txteno"]),
                Esal = double.Parse(Request["txtnewsal"])
            };
        ViewBag.update = obj.UpdateEmp();
            forwardemp();
            return View();
        }
    }
    public ActionResult  Insert()
    {
        if(Request["txtempno"]==""||Request["txtempname"]==""||Request["txtempsal"]=="")
        {
            forwardemp();
            return View("Index");
        }
        else
        {
            Employee obj = new Employee
            {
                Eno = int.Parse(Request["txtempno"]),
                Ename = Request["txtempname"],
                Esal =double.Parse( Request["txtempsal"])
            };
        ViewBag.insert = obj.InsertEmp();
            forwardemp();
            return View("Index");
        }
    }
  }
}
```

VIEW1

```
@using System.Data;
@{
   Layout = null;
   string enomsg = "";
   string enamemsg = "";
   string esalmsg = "";
   string insertmsg = "";
   if(this.IsPost)
   {
      if(Request["txtempno"].IsEmpty())
      {
         enomsg = "pls enter empno";
      }
      else if(Request["txtempname"].IsEmpty())
      {
```

```
        enamemsg = "pls enter empname";
      }
      else if(Request["txtempsal"].IsEmpty())
      {
        esalmsg ="pls enter new salary";
      }
      else
      {
        if (ViewBag.insert!=null)
        {
          int j = ViewBag.insert;
          if(j==1)
          {
            insertmsg = "record is inserted";
          }
          else
          {
            insertmsg = "record is not insertred";
          }
        }
      }
    }
}

<!DOCTYPE html>

<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form id="myform" method="post">
<div>
<table border="1">
<tr>
<th>Employee no</th>
<th>Employee name</th>
<th>Employee salary</th>
<th>DELETE</th>
<th>EDIT</th>
</tr>
          @{
            foreach(DataRow myrow in ViewBag.emp.Rows)
            {
<tr>
<td>@myrow[0]</td>
<td>@myrow[1]</td>
<td>@myrow[2]</td>
<td>@Html.ActionLink("delete","DeleteCntrlEmp",new {eno=myrow["empno"]})</td>
<td>@Html.ActionLink("edit", "EditEmp", new {eno=myrow["empno"]})</td>
```

```
</tr>
            }
          }
<tr>
<td>
                @Html.TextBox("txtempno")
</td>
<td>@Html.TextBox("txtempname")</td>
<td>@Html.TextBox("txtempsal")</td>
<td><input type="submit" value="insert" name="btninsert" formaction="Insert" /></td>
</tr>
<tr>
<td>@Html.Label("lbleno",enomsg)
</td>
<td>@Html.Label("lblename",enamemsg)</td>
<td>@Html.Label("lblesal",esalmsg)</td>
</tr>
</table>
        @Html.Label("lblmsg",insertmsg)
</div>
</form>
</body>
</html>
```

VIEW 2: EditIndex:

```
@using System.Data;
@{
   Layout = null;
   string enomsg = "";
   string esalmsg = "";
   string newsalmsg = "";
   string msg = "";
   foreach(DataRow myrow in ViewBag.emp.Rows)
   {
      enomsg = myrow["empno"].ToString();
      esalmsg = myrow["salary"].ToString();
   }
   if(this.IsPost)
   {
      if(Request["txtesal"]=="")
      {
         newsalmsg = "pls enter new salary";
      }
      else
      {
         int i = ViewBag.update;
         if(i==1)
         {
            msg = "salary is updated";
         }
```

```
        else
        {
           msg = "salary is not updated";
        }
     }
  }
}
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>editindex</title>
</head>
<body>
<form id="myform" method="post" action="~/Emp/EditIndex">
<div>
        empno is:<input type="text" value="@enomsg" name="txteno" readonly /><br /><br />
        enter new salary:<input type="text" value="@esalmsg" name="txtnewsal" />
        @Html.Label("lblmsg",newsalmsg)
<br /><br />
<input type="submit" value="update" name="btnupdate" />
<input type="submit" value="GoBack" name="btngoback" formaction="~/Emp/Index" />
<br /><br />
        @Html.Label("lblmsg",msg)
</div>
</form>
</body>
</html>
```

Linq

What is linq?

Linq stands for Language Integrated Query , Which was introduced by the Micro Soft,with >net Framework 3.5

Linq is advanced data access object in .net,Ado.Net is traditional data access object in .Net

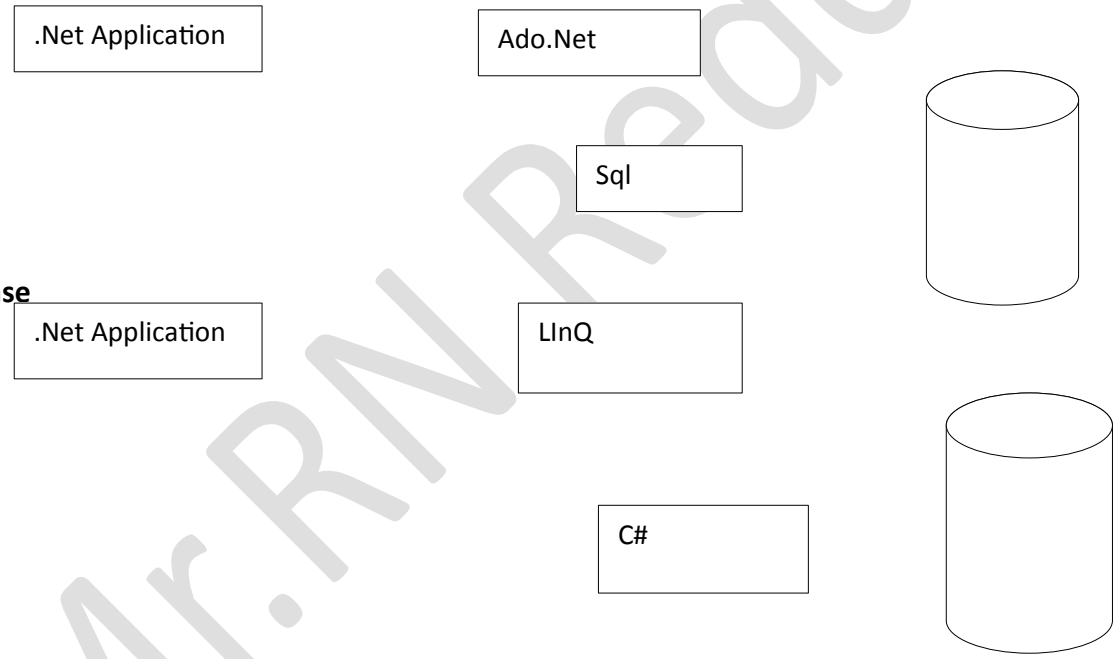
What we can do by using Linq?

Using Ado.Net a .Net application can communicate the databases , similarly using also a .Net application can communicate the database.

What is the difference between Ado.Net and linq?

Using Ado.Net to communicate the data bases with in the .Net application we will write the Sql Commands like select,Insert,Update,Delete,Create &...

But using linq to communicate the databases with in the .Net application we don't required to write SqlCommands these Sqlcommands we can implement by using any one of the .Net language like C#.Net or Vb.Net Like below.



Note : Linq is providing predefined method for insert,update,delete&...

But retrieving data from database linq is provided one Query it is called as linq query.

Syntax for linq Query :

Var v1=from <variableName>in<datasourse>[clauses]select<varibleName>;

What is OR tool:

To communicate the data Sources by using linq Microsoft is providing one predefing tool called OR tool(object relational)

What will provided by OR tool?

OR tool will be providing one predefined class that predefining class will having various predefined methods to implement remaining operations on databases like insert,update,delete and so on...

When ever we want to implement insert,update,delete and so on operations with in the datasourses we have to create an object for concern like predefined class and we have to access the concern linq class predefined method and predefined properties.
Note: OR tool treating table as a class and column are properties.
Using linq to acess the datatable we have to create an object for table class and using that object we can acesss the columns.
Using linq we can communicate the data sources like below
1.collections
2.databases
3.XML

Programming by using Linq:

Linq to Collections:

Example to retrieve the data from list collections by using Linq

```
class Program
 {
 static void Main(string[] args)
   {
     List<int> emplist = new List<int> { 30, 20, 50, 40, 10 };
     var v1 = from eno in emplist select eno;
     foreach(int i in v1)
      {
        Console.WriteLine(i);
      }
            Console.ReadLine();
    }
```

OutPut

30 20

50
40
10

Example to retrieve the data from list collections by using Linq which are garter than 20

```
class Program
 {
 static void Main(string[] args)
   {
     List<int> emplist = new List<int> { 30, 20, 50, 40, 10 };
     var v1 = from eno in emplist where eno>20 select eno;
     foreach(int i in v1)
      {
        Console.WriteLine(i);
      }
            Console.ReadLine();
    }
```

OutPut

30
50
40

Example to retrieve the data from list collections by using Linq which are garter than 20 in Order

```
class Program
 {
 static void Main(string[] args)
   {
     List<int> emplist = new List<int> { 30, 20, 50, 40, 10 };
     var v1 = from eno in emplist where eno>20 OrderBy select eno;

     foreach(int i in v1)
      {
        Console.WriteLine(i);
      }
            Console.ReadLine();
    }
```

OutPut

30
40
50

LINQ To SQL:

Example to display emp table by using Linq In Asp.Net

Step:1 Design front end and back End:

WebForm1.aspx[Design]

EmployeeNo	EmployeeName	EmployeeSalary
111	Rama	1000
222	Sitha	2000
333	Ravi	3000

Dispaly

DataBase

Empno	EmpName	Salary
111	Rama	1000
222	Sitha	2000
333	Ravi	3000

Step :2 Creating Asp.Net WebApplication.

Step :3 Add WebForm1.aspx Design like above set it as start page.

Step:4 view Server Explorer

Select dataconnection Right click Add connection it will Open , choose data source window select 'microsoft Sql' click continue.

It will open Add connection Window Here server name as 'satya-pc', here select (any authentication) Sql server authentication user name 'sa',password 'abc' in bottom select 'Mydatabase' click 'ok' Button.

Step :5 Adding OR tool

Open solution Explorer select project Name Right click Add select NewItem It will open Add new Item window select left side 'Data', right side Select Templete called ' Linq to SQL calss' rename it as LinqToSql.Dbml(or)tool designer window will Add.

LinqtoSql.dbml will have two supporting files:

1.dbml layout it is designer window.

2.linqtosql.disigner.cs it is a OR tool class file.

This class file will have a class like below

```
Class linqToSqlDataContext:System.Data.Linq.DataContext
{
}
```

Step :4 Adding Emp table to OR tool designer window

Enable OR tool Designer window

Open Serevr explorer go to mydatabase select emp table image will add.

Webform1.aspx.cs:

```
{
linqToSqlDataContext objlinq=new linqToSqlDataContext();
var v1=from empnew in objlinq.emps select empnew;
GridEmp.DataSource=v1;
GridEmp.DataBind();}
```

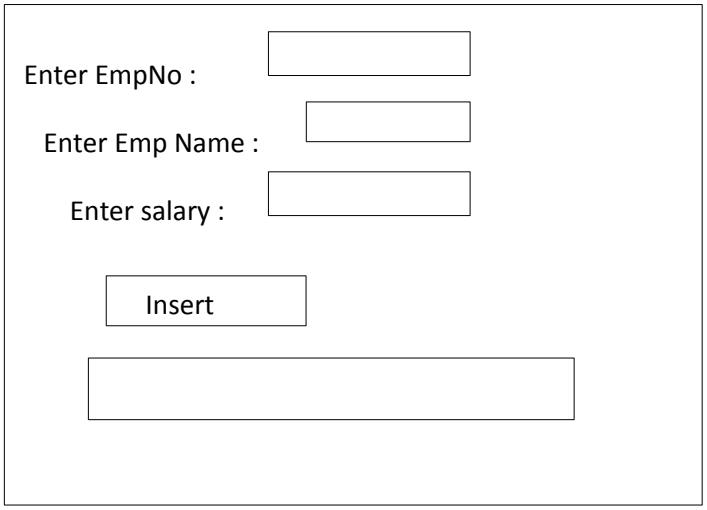
Note:OR tool class file will have the below auto generated code LinqToSql.designer.cs

Note:EMp table as class,columns as properties

```
Class Emp
{
    Empno{get; set;}
    EmpName{get; set; }
    Salary{get; set; }
}
```

Example to Insert into emp table by using Linq In Asp.Net
Step:1
Step:1 Design front end and back End:

WebForm1.aspx[Design]



DataBase

Empno	EmpName	Salary
111	Rama	1000
222	Sitha	2000
333	Ravi	3000

Step :2 Creating Asp.Net WebApplication.
Step :3 Using Server Explorer create the emp table using Server Explorer connect Emp table.
Step :4 Adding OR Tool rename it as LinqToSql.Dbml.
Step:5 Add WebForm1.aspx Design like above set it as start page.

```
Class webform1:
{
btnInsert_Click()
{
    linqToSqlDataContext objLinq=new linqToSqlDataContext();
    Emp objEmp=new Emp();
   objEmp.EmpNo=int.parse(txtEno.Text);
  objEmp.EmpName=txtEname.Text;
 objEmp.Salary=Decimal.parse(txtESal.Text);
 objLinq.Emps.InsertOnSubmit(objEmp);
 objLinq.SubmitChanges();
  txtEno.Text="";
 txtEname.Text="";
 txtEsal.Text="";
 txtEno.Focus();
  lblmsg.Text="Record is Inserted";
   }
 }
```

Why we have to create object for Emp class

When ever we will add Emp table to OR tool it will create one class with in the OR tool class file i.e. LinqToSql.Designer.csWith in the Emp Class it will create property for every column like below

```
Class Emp
{
```

EmpNo{ get; set;}
EmpName { get; Set; }
Salary{ get; set;} }

What is Emps

Emps is a property is create with in the linq class on the be of Emp table like below

```
Class LinqToSqlDataContext
{
      Emps { get; set;}
}
```

What is InsertOnSubmit(objEmp):

InsertOnSubmit() is a predefine member method of table class. This method will insert the data into targeted table to access this table we are using 'Emps' property because Emps property return type is table

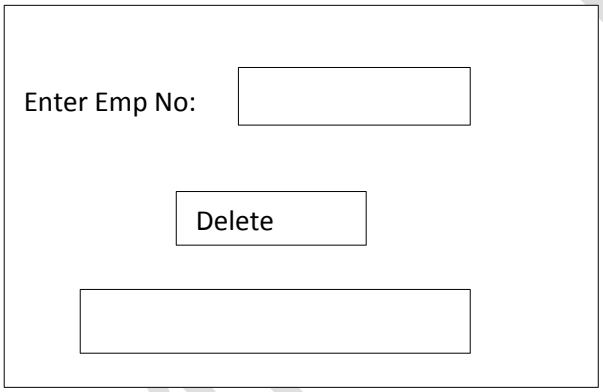
What is SubmitChanges()

It is a predefined member method od DataContext predefined class this method will conform the above operations like 'commit'.

Example to delete a record from Asp.Net in Linq

Step:1 Design of frontEnd and BackEnd

Webform1.aspx[design]



database

Emp

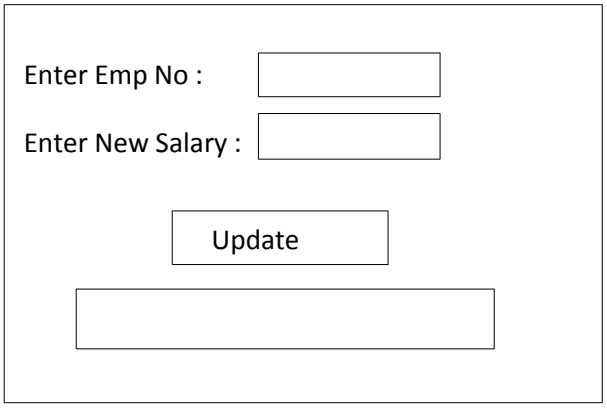
EmpNo	EmpName	Salary
111	Rama	1000
222	Sitha	2000
333	Ravi	3000

Step :2Adding OR Tool ,Adding EMP table by using server Explorer

webForm1.aspx.cs:

```
 class webform1:
{
Btndelete_click()
{
    linqToSqlDataContext objLinq=new linqToSqlDataContext();
   int Eno=int.parse(txtEno.Text);
    Emp objEmp=objLinq.Emps.Single(empnew=>empnew.EmpNo==Eno);
   objLinq.Emps.DeleteOnSubmit(objEmp);
    objLinq.SubmitChanges();
   lblmsg.Text="Record is Deleted";
 }
}
```

Example For Updating the salary in Emp table by using Asp.Net
Step:1 Design FrontEnd and BackEnd
WebForm1.aspx



DataBase

EmpNo	EmpName	Salary
111	Rama	1000
222	Sitha	2000
333	Ravi	3000

Step:2 Adding OR Tool
Step :3 Adding Emp table to the OR tool
Step :4 WebForm1.aspx.cs

```
Class WebForm1
{
btnUpdate_click()
{
    linqToSqlDataContext objLinq=new linqToSqlDataContext();
    Emp objEmp=objLinq.Emps.Single(empnew=>empnew.EmpNo==Eno);
    objEmp.Salary=decimal.parse(txtNewSal.Text);
    objLinq.SubmitChanges();
    lblmsg.Text=”Salary is Updated”;
 }
}
```

Linq in Asp.Net MVC

Example to display Emp Table with HTML table.
Step :1 Design of Display.cshtml
WebForm1.aspx

EmployeeNo	EmployeeName	EmployeeSalary
111	Rama	1000
222	Sitha	2000
333	Ravi	3000

DataBase

Emp

EmpNo	EmpName	Salary
111	Rama	1000
222	Sitha	2000
333	Ravi	3000

```
Public class Employee
 {
LinqToSqlDataContext linqObj=new LinqToSqlDataContext();
```

```
     Public IQueriable GetEmp()
    {
 Var V1=from empnew in linqobj.Emps select empnew
      Return e1;
   }
}
```

EmpControllerCode:

```
Using MvcDisplayExample.Models
namespace  MvcDisplayExample
{
    Public class EmpController
    {
      Public void ForwordEmp()
      {
         ViewBag.Emp=obj.GetEMp();
      }
      Public ActionResut Display()
      {
         forwordEmp();
        return view();
      }
   }
}
```

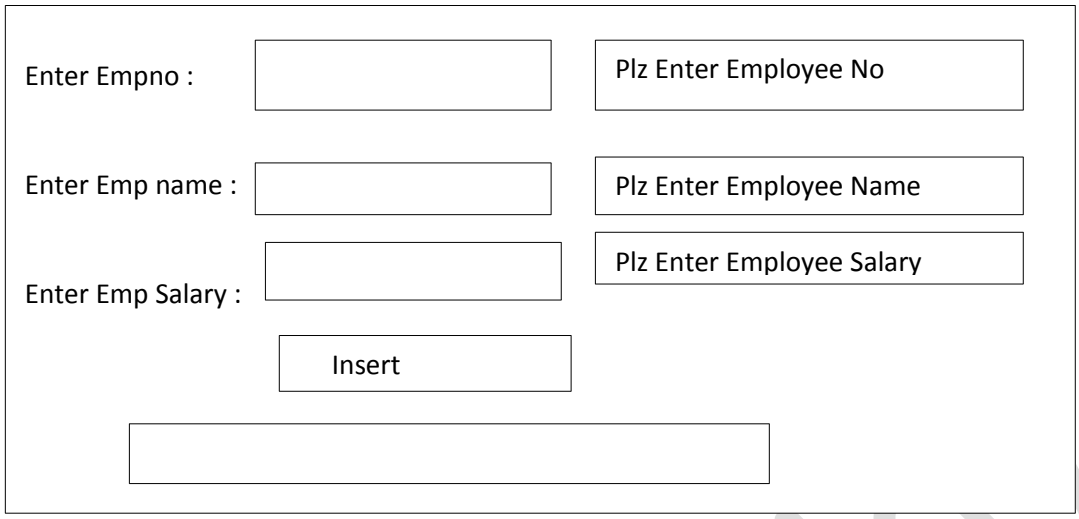
View[Display.cshtml]

```
<html>
<body>
<div>
<table border="1">
<tr>
<th>Employee No</th>
<th>Employee Name</th>
<th>Employee salary</th>
</tr>
@{
                    foreach (var v in ViewBag.emp)
                    {
<tr>
<td>@v.empno</td>
<td>@v.empname</td>
<td>@v.salary</td>
</tr>
                    }
}

</table>
</div>
</form>
</body>
</html>
```

Update the Routeconfig.cs file

Controller=”Emp” action=”Display”

Insert A record in to Emp table:




```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
namespace MvcLinqInsertEmployeeExample.Models
{
    public class Employee
    {
        public int Eid { get; set; }
        public string Ename { get; set; }
public double Esal { get; set; }

        LinqToSqlDataContext linqobj = new LinqToSqlDataContext();
Emp objstu = new Emp();
        public int insertintoEmp()
        {

            objEmp.eid = Eid;
            objEmp.ename = Ename;
            objEmp.esal=Esal;
            try
            {
                linqobj.Emps.InsertOnSubmit(objEmp);
                linqobj.SubmitChanges();
                return 1;
            }
            catch
            {
                return 0;
            }

        }
    }
}
```

Controller code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
```

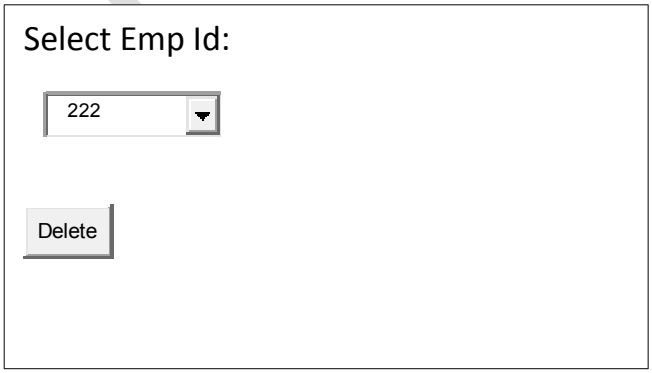
```
using MvcLinqInsertEmployeeExample.Models;
namespace MvcLinqInsertEmployeeExample.Controllers
{
    public class EmpController : Controller
    {
        // GET: Stu
        public ActionResult Insert()
        {
            return View();
        }
        [HttpPost]
        public ActionResult Insert(string s)
        {


            if(Request["txtEid"] == "" || Request["txtEName"] == ""||
Request[“txtEsal”])
            {
                return View();
            }
            else
            {
                Employee objEmp = new Employee
                {
Eid = int.Parse(Request["txtEid"]),
                    Ename = Request["txtEName"],
                    Esal=decimal.parse(Request[“txtEsal”])

                };
                ViewBag.i = objstu.insertintoEmp();
                      return View();
            }


        }
    }
}
```

View Code:

```
@{
    Layout = null;
    string eidmsg = "";
    string enamemsg = "";
    string esalmsg=””;
    string msg = "";
    if (IsPost)
    {
        if (Request["txtEid"] == "")
        {
            eidmsg = "*plz enter Emp ID";
        }
        else if(Request["txtEName"]=="")
        {
enamemsg = "*plz enter Emp Name";
        }
        else if(Request["txtEsal"]=="")
        {
            esalmsg = "*plz enter Emp salary";
        }
```

```
        else
        {
            int j = (int)ViewBag.i;
            if(j==1)
            {
                msg = "Record is Inserted ";
            }
            else
            {
                msg = "Record is Not Inserted";
            }
        }
    }
}

<!DOCTYPE html>

<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Insert</title>
</head>
<body>
<form id="Myform" method="post">
<div>
            Enter Employee ID: @Html.TextBox("txtEid")
@Html.Label("lblEid",eidmsg)
<br /><br />
            ENter Employee Name @Html.TextBox("txtEname")
@Html.Label("lblEname",enamemsg)
<br /><br />
             ENter Employee salary @Html.TextBox("txtEsal")
@Html.Label("lblESal",esalmsg)
<br/><br/>
<input type="submit" id="btnInsert" value="Insert" />
<br /><br />
@Html.Label("lblmsg",msg)
</div>
</form>
</body>
</html>
```

Example Delete a record from Emp table:



```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
namespace MvcLinqDeleteEmployeeExample.Models
{
    public class Employee
    {

        public int Eid { get; set; }
        LnqToSqlDataContext obj = new LnqToSqlDataContext();
        public IQueryable GetEmpid()
        {
            var v1 = from empnew in obj.employees select empnew.empid;
            return v1;
        }
        public int DeleteEmp()
        {
            employee objemp = obj.employees.Single(emp => emp.empid == Eid);
            obj.employees.DeleteOnSubmit(objemp);
            obj.SubmitChanges();
            return 1;

        }


    }
}
```

Controller code:

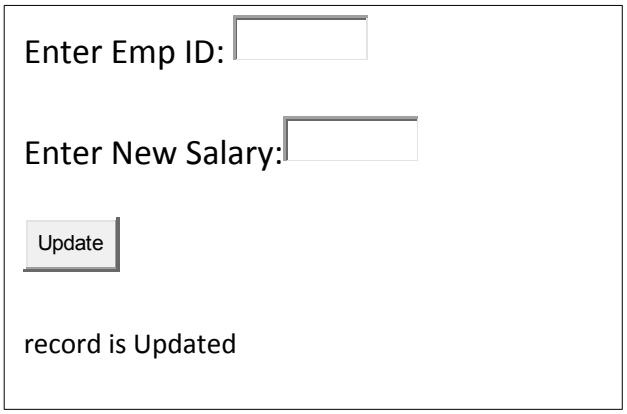
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using MvcLinqDeleteEmployeeExample.Models;
namespace MvcLinqDeleteEmployeeExample.Controllers
{
    public class EmpController : Controller
    {
        internal void Emp()
        {
            Employee objemp = new Employee();
            ViewData["d1"] = new SelectList(objemp.GetEmpid());
        }
        // GET: Emp
        public ActionResult Delete()
        {
            Emp();
            return View();
        }
        [HttpPost]
        public ActionResult Delete(string s)
        {
            Emp();
            if (Request["d1"] == "")
            {
                return View();
            }
            else
```

```
            {
                Employee obj = new Employee()
                {
                    Eid = int.Parse(Request["d1"])
                };
                ViewData["d"] = obj.DeleteEmp();
                return View();
            }
        }
    }
}
```

View code:

```
@{
    Layout = null;
    string eidmsg = "";
    string msg = "";
    if (IsPost)
    {
        if (Request["d1"] == "")
        {
            eidmsg = "*plz select Valid Emp id";
        }
        else
        {
            int j = (int)ViewData["d"];
            if (j == 1)
            {
                msg = "record is Deleted";
            }
            else
            {
                msg = "record is not Delted";
            }
        }
    }
}
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Delete</title>
</head>
<body>
<form id="myfrom" method="post">
<div>
            Select Emp  Id: @Html.DropDownList("d1","-select-")
@Html.Label("lbleid", eidmsg)
<br/><br />
<input type="submit" id="btnDelete" value="Delete" />
@Html.Label("lblmsg",msg)
</div>
</form>
</body>
</html>
```

Example Update a record in Emp table



ModelCode:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;

namespace MvcLinqUpdateEmployeeExample.Models
{
    public class Employe
    {
        LinqToSqlDataContext obj = new LinqToSqlDataContext();
        public int Eid { get; set; }
        public double Esal { get; set; }
        public int UpdateEmp()
        {
            employee objemp = obj.employees.Single(emp => emp.empid == Eid);
            objemp.salary =(decimal)Esal;
            obj.SubmitChanges();
            return 1;
        }
    }
}
```

Controller Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using MvcLinqUpdateEmployeeExample.Models;

namespace MvcLinqUpdateEmployeeExample.Controllers
{
    public class EmpController : Controller
    {
        // GET: Emp
        public ActionResult Update()
        {
            return View();
        }
        [HttpPost]
        public ActionResult Update(string s)
        {
            if (Request["txtEid"] == "" || Request["txtNewSal"] == "")
            {
                return View();
            }
```

```
            else
            {
                Employe objemp = new Employe
                {
                    Eid = int.Parse(Request["txtEid"]),
                    Esal = double.Parse(Request["txtNewSal"])
                };
                ViewBag.v = objemp.UpdateEmp();
                return View();
            }
        }
    }
}
```

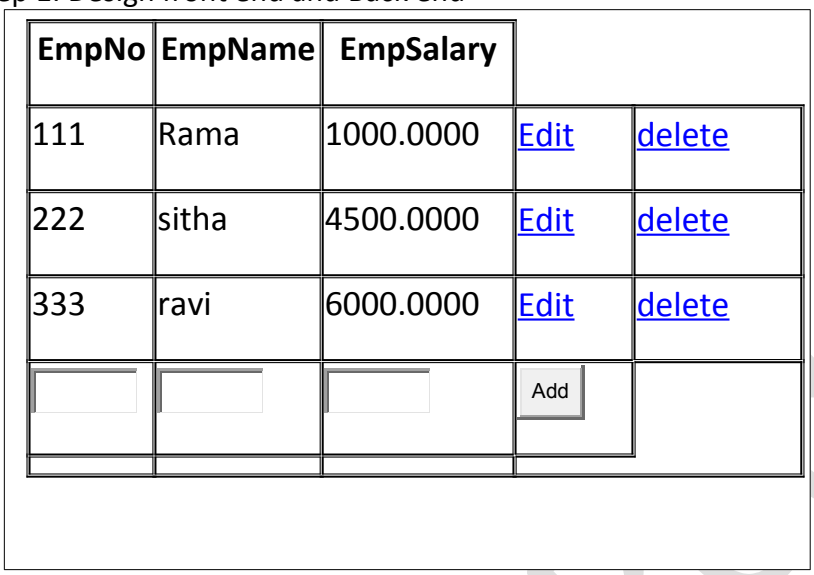
View Code:

```
@{
    Layout = null;
    string eidmsg = "";
    string esalmsg = "";
    string msg = "";
    if (IsPost)
    {
        if(Request["txtEid"]=="")
        {
            eidmsg = "* plz enter Empid";
        }
        else if(Request["txtNewSal"]=="")
        {
            esalmsg = "* plz enter New salary";
        }
        else
        {
            int j = ViewBag.v;
            if(j==1)
            {
                msg = "record is Updated";
            }
            else
            {
                msg = "record is not Updated";
            }
        }
    }
}

<!DOCTYPE html>

<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Update</title>
</head>
<body>
<form id="myform" method="post">
<div>
            Enter Emp ID: @Html.TextBox("txtEid")
@Html.Label("lblEid", eidmsg)
<br /><br />
            Enter New Salary:@Html.TextBox("txtNewSal")
@Html.Label("lblEsal", esalmsg)
<br />
<br />
<input type="submit" id="btnUpdate" value="Update" />
```

```
<br /><br />
@Html.Label("lblmsg", msg)
</div>
</form>
</body>
</html>
```

Example to Implement Crud Operations(Display,insert,update,delete) by using linq and Mvc

Step 1: Design front end and Back end



Model Code:-

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;

namespace MvcLinqCurdExample1.Models
{
    public class Employee
    {
        linqtosqlDataContext objlink = new linqtosqlDataContext();
        public int Eno  {   get; set; }
        public string Ename{get; set; }
        public decimal Esal{get; set; }
        public IQueryable Display()
        {
            var v1 = from empnew in objlink.emps select empnew;
            return v1;
        }
        public IQueryable GetDetails()
        {
 var v1 = from empnew in objlink.emps where empnew.empno==Eno select empnew;
            return v1;
        }
        public int Insert()
        {
            emp objemp = new emp();
            objemp.empno = Eno;
            objemp.empname = Ename;
```

```
            objemp.salary = Esal;
            try
            {
                objlink.emps.InsertOnSubmit(objemp);
                objlink.SubmitChanges();
                return 1;
            }
            catch
            {
                return 0;
            }
        }
        public int Delete()
        {
            emp objemp = objlink.emps.Single(empnew => empnew.empno == Eno);
            try
            {
                objlink.emps.DeleteOnSubmit(objemp);
                objlink.SubmitChanges();
                return 1;
            }
            catch
            {
                return 0;
            }
        }
        public int Update()
        {
            emp objemp = objlink.emps.Single(empnew => empnew.empno == Eno);
            objemp.salary = Esal;
            try
            {
                objlink.SubmitChanges();
                return 1;
            }
            catch
            {
                return 0;
            }
        }
    }
}
```

controller code:-

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using MvcLinqCurdExample1.Models;

namespace MvcLinqCurdExample1.Controllers
{
    public class EmpController : Controller
    {
        // GET: Emp
        internal void Display()
        {
            Employee obj = new Employee();
            ViewBag.emp = obj.Display();
        }
```

```
// GET: Home
public ActionResult Index()
{
    Display();
    return View();
}
[HttpPost]
public ActionResult Insertemp()
{
    Display();
    if (Request["txteno"] == "")
    {
        ViewBag.eno = "Pls enter eno";
    }

    else if (Request["txtename"] == "")
    {
        ViewBag.ename = "Pls enter ename";
    }

    else if (Request["txtesal"] == "")
    {
        ViewBag.esal = "Pls enter esal";
    }
    else
    {
        Employee obj = new Employee
        {
            Eno = int.Parse(Request["txteno"]),
            Ename = Request["txtename"],
            Esal = decimal.Parse(Request["txtesal"])
        };
        int i = obj.Insert();
        if (i == 1)
        {
            ViewBag.msg = "record is inserted";
        }
        else
        {
            ViewBag.msg = "record is not inserted";
        }
    }
    return View("Index");
}
public ActionResult EditIndex()
{
    Employee obj = new Employee
    {
        Eno = int.Parse(Request.QueryString["eno"]),
    };
    ViewBag.n = obj.GetDetails();
    return View("UpdateIndex");
}
public ActionResult UpdateIndex()
{
    Employee obj = new Employee
    {
        Eno = int.Parse(Request["txtempno"]),
        Esal = decimal.Parse(Request["txtsal"]),
    };

    int i = obj.Update();
```

```
            if (i == 1)
            {
                ViewBag.msg = "salary is updated";
            }
            else
            {
                ViewBag.msg = "salary is not updated";
            }
            return View("UpdateIndex");
        }
        public ActionResult Delete()
        {
            Display();
            Employee obj = new Employee
            {
                Eno = int.Parse(Request.QueryString["eno"])
            };
            int i = obj.Delete();
            if (i == 1)
            {
                ViewBag.msg = "record is deleted";
            }
            else
            {
                ViewBag.msg = "record is not deleted";
            }
            return View("Index");
        }
    }
}
```

Index.cshtml code:-

```
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form id="myform" method="post">
<div>
<table border="1">
<tr>
<th>EmpNo</th>
<th>EmpName</th>
<th>EmpSalary</th>
</tr>
@foreach (var item in ViewBag.emp)
                {
<tr>
<td>@item.empno</td>
<td>@item.empname</td>
<td>@item.salary</td>
<td>
@Html.ActionLink("Edit", "EditIndex", new { eno = item.empno })</td>
<td>
@Html.ActionLink("delete", "Delete", new { eno = item.empano })</td>
</tr>
```

```
                }
<tr>
<td>@Html.TextBox("txteno")</td>
<td>@Html.TextBox("txtename")</td>
<td>@Html.TextBox("txtesal")</td>
<td><input type="submit" id="btnadd" value="Add" formaction="Insertemp" /></td>
</tr>
<tr>
<td><span>@ViewBag.eno</span></td>
<td><span>@ViewBag.ename</span></td>
<td><span>@ViewBag.esal</span></td>
</tr>
</table>
<span>@ViewBag.msg</span>
</div>
</form>
</body>
</html>
```

UpdateIndex.cshtml:-

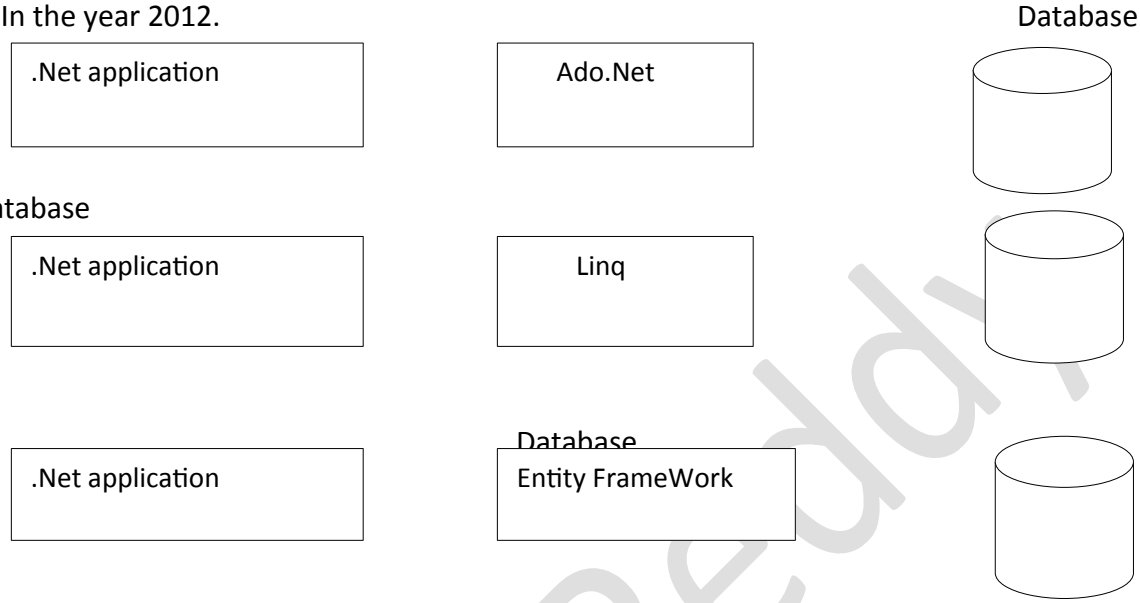
```
@{
    Layout = null;
    int eno = 0;
    decimal esal = 0;
    if (ViewBag.n != null)
    {
        foreach (var item in ViewBag.n)
        {
            eno = item.eno;
            esal = item.salry;
        }
    }

}
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>UpdateIndex</title>
</head>
<body>
<form id="myform" method="post" action="~/Emp/UpdateIndex">
<div>
Enter empNo:<input type="text" value="@eno" name="txtempno" readonly/><br/>
            Enter new salary:<input type="text" value="@esal" name="txtsal" />
<input type="submit" id="btn" value="Update" /><br />
<input type="submit" id="btngo" value="GoBack" formaction="~/Emp/Index" />
<span>@ViewBag.msg</span>
</div>
</form>

</body>
</html>
```

Entity FrameWork

Wat is Entity Framework?

Entity frame work is a .Net Advanced data access technology , it always .Net Application to Communicate to database.

Linq is Advanced for Ado.Net,Entity FrameWork is advanced for Linq.
Ado.Net is introduced in .Net 1.0 linq is introduced in .Net 3.5 as well as Entity Frame Work First version i.e. Entity Frame Work 3.5 was introduced with .Net 3.5 after releasing of linq (few months gap)for entity frame Work 3.5 is a basic version
Latest version of entity framework is entity framework 6.0 as released with .Net framework 4. In the year 2012.



Entity FrameWork is an ORM Framework.
ORM Stands for object Relational mapping.

What is an object relational mapping framework?

Object relational mapping frame work automatically creating class with are based on the database tables.

Advantages of entity frame work

1.Productivity

Entity frame work will reduce the coding part which make the programmer task easy which improves the productivity of the programmer.

2.maintainalibility:

In entity frame work data access code is very less which will makes the maintenances easy.

3.Improved Performance:

In entity frame work first request to fetch the details little bit slow but after that it is very fast to fetch data from database for post back request.

Programming by using entity frame work in Asp.Net

Example to display emp table table with in the Asp.Net web page by using entity frame Work.
Step:1design of front end And Back end.

Webform1.aspx[Design]

empno	empname	salary
111	Rama	1000.0000
222	sitha	2000.0000
		3000.000

Mydatabase

Emp

empno	empname	salary
111	Rama	1000.0000
222	Sitha	2000.000

Step:2 Create a new Web application
Step:3 Design webform1.aspx like above
Step:4 Adding Entity frame work ORM tool.

- Open solution explorer select project name right click add new item it will open add new item window. Here select left side “data” here select right side”Ado.Net Entity Data Model” rename it as “EntityModel.edmx” click Add.
- It will open EntityDataModelWizard.
- Here select “GenerateformDataBase” click Next in next window Click new connection it will open connection properties windows here write server Name[satya-pc], here select Sql Server Authentication user name[sa] password[abc] here select Database name as mydatabase click Ok.
- Here select a radio button called
  - yes,Include the sensitive data in the connection string
    click Next. Here select Entity Frame Work 5.0 or 6.0 click Next.
    In the Next Window Here select table as emp click finish.

With this process it will Add EntityModel.edmx file with its support files.
Step:5 WebForm1.aspx.cs

```
namespace EfAspEmpDisplayExample
{
    public partial class WebForm1 : System.Web.UI.Page
    {
       protected void Page_Load(object sender, EventArgs e)
        {
            mydbmvcEntities objdb = new mydbmvcEntities();
            GridEmpNo.DataSource = objdb.emps.ToList();
            GridEmpNo.DataBind();

}
 }
}
```

Example to Insert a Record into Emp Table:
Step1: Design of Fornt end and Back end

Enter Emp NO :
Enter EMp Name :
Enter Salary :
Insert

Step:2 Creating Asp.Net Mvc Application
Step:3 Adding ORM tool
Step:4 Adding Model[Employee.cs]

Model Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
namespace MvcAspInsertEmpExample.Models
{
    public class Employee
    {
        mydbmvcEntities objdb = new mydbmvcEntities();
        public int Eno { get; set; }
        public String Ename { get; set; }
        public decimal Esal { get; set; }
        public int InsertIntoEmp()
        {
            emp objemp = new emp();
            objemp.empno = Eno;
            objemp.empname = Ename;
            objemp.salary = Esal;
            try
            {
                objdb.emps.Add(objemp);
                objdb.SaveChanges();
                return 1;
            }
            catch
            {
                return 0;
            }
        }
    }
}
```

Step:4 Adding Controller **[EmpController.cs]**

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using MvcAspInsertEmpExample.Models;
namespace MvcAspInsertEmpExample.Controllers
```

```
{
    public class EmpController : Controller
    {// GET: Emp
        public ActionResult Index()
        {
            return View();
        }
        [HttpPost]
        public ActionResult Insert()
        {
            Employee objmodel = new Employee
            {
                Eno = int.Parse(Request["txtEno"]),
                Ename = Request["txtEname"],
                Esal = decimal.Parse(Request["txtEsal"])
            };
            ViewData["Insert"] = objmodel.InsertIntoEmp();
            return View("Index");
        }
    }
}
```

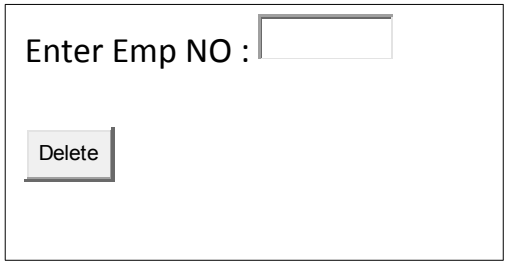
Step:6 Adding View: [Index.cshtml]

```
@{
    Layout = null;
    string msg = "";
    if (this.IsPost)
    {
        int i = (int)ViewData["Insert"];
        if (i == 1)
        {
            msg = "Record is Inserted";
        }
else
        {
            msg = "Record IS Not Inserted";
        }
    }
}
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form id="myform" method="post">
<div>
            Enter Emp NO : @Html.TextBox("txtEno")
<br /><br />
            Enter EMp Name : @Html.TextBox("txtEname")
<br /><br />
            Enter Salary : @Html.TextBox("txtEsal")
<br /><br />
```

```
<input type="submit" id="btmInsert" value="Insert"
formaction="~/Emp/Insert" />
<br /><br />
            @Html.Label("lblmsg",msg)
</div>
</form>
</body>
</html>
```

Example to Delete A Record in to Emp table:

Step : 1 Design of Front end and Back end



Step :2 Creating Asp.Net Mvc Application
Step:3 Adding ORM tool
Step:4 Adding Model[Employee.cs]

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
namespace MvcAspEmpDeleteExample.Models
{
    public class Employee
    {
        public int Eno { get; set; }
        public int DeleteEno()
        {
            mydbmvcEntities objdb = new mydbmvcEntities();
            emp objEmp = objdb.emps.Find(Eno);
            try
            {
                objdb.emps.Remove(objEmp);
                return 1;
            }
            catch
            {
                return 0;
            }
        }
    }
}
```

Controller Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using MvcAspEmpDeleteExample.Models;
```

```
namespace MvcAspEmpDeleteExample.Controllers
{
    public class EmpController : Controller
    {

        // GET: Emp
        public ActionResult Index()
        {
            return View();
        }
        [HttpPost]
        public ActionResult Delete()
        {
            Employee objModel = new Employee
            {
                Eno = int.Parse(Request["txtEno"])
            };
            ViewData["Delete"] = objModel.DeleteEno();
            return View("Index");
        }
    }
}
```

View Code :

```
@{
    Layout = null;
    string msg = "";
    if(this.IsPost)
    {
        int i = (int)ViewData["Delete"];
        if (i == 1)
        {
            msg = "Record is Delete";
        }
        else
        {
            msg = "record is not Deleted";
        }
    }
}
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form id="myform" method="post">
<div>
            Enter Emp NO : @Html.TextBox("txtEno")
<br /><br />
<input type="submit" name="btnDelete" value="Delete" formaction="~/Emp/Delete" />
<br /><br />
@Html.Label("lblmsg",msg)
</div>
</form>
</body>
</html>
```

Example to Update the employee Salary:

Step :1 Design front end and back end

Enter Emp No :

Enter Emp Name :

Step :2 Creating Asp.Net Mvc Application
Step:3 Adding ORM tool
Step:4 Adding Model[Employee.cs]

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
namespace MvcEfEmpUpdateExample.Models
{
    public class Employee
    {
        mydbmvcEntities objdb = new mydbmvcEntities();
        public int Eno { get; set; }
        public decimal Esal { get; set; }
        public int UpdateEmp()
        {
            emp objEmp = objdb.emps.Find(Eno);
            objEmp.salary =Esal;
            try
            {
                objdb.SaveChanges();
                return 1;
            }
            catch
            {
                return 0;
            }
        }
    }
}
```

Step :5 Controller Code:[EmpController]

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using MvcEfEmpUpdateExample.Models;

namespace MvcEfEmpUpdateExample.Controllers
{
    public class EmpController : Controller
    {
        // GET: Emp
        public ActionResult Index()
        {
            return View();
        }
        [HttpPost]
        public ActionResult Update()
```

```
        {
            Employee objmodel = new Employee
            {
                Eno = int.Parse(Request["txtEno"]),
                Esal = decimal.Parse(Request["txtEsal"])
            };
            ViewData["UpdateEval"] = objmodel.UpdateEmp();
            return View("Index");
        }
    }
}
```

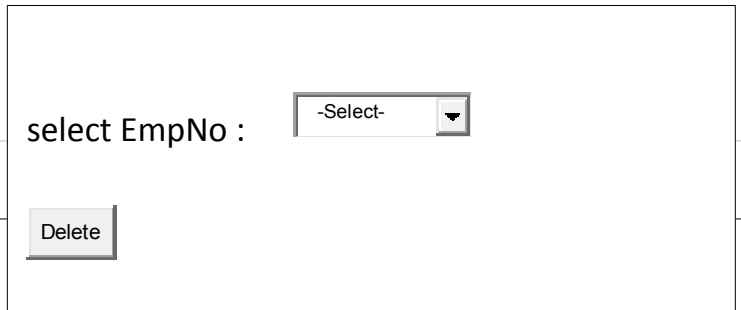
Step:6 View Code:[Index.cshtml]

```
@{
    Layout = null;
    string msg = "";
    if (this.IsPost)
    {
        int i = (int)ViewData["UpdateEval"];
        if (i == 1)
        {
            msg = "salary is Updated";
        }
        else
        {
            msg = "salary is not Updated";
        }
    }
}

<!DOCTYPE html>

<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form id="myform" method="post">
<div>
            Enter Emp No : @Html.TextBox("txtEno")
<br /><br />
            Enter Emp Name : @Html.TextBox("txtEsal")
<br /><br />
<input type="submit" id="txtUpdated" value="Update" formaction="~/Emp/Update" />
<br /><br />
@Html.Label("lblmsg",msg)
</div>
</form>
</body>
</html>
```

Update RouteConfig.cs Controller=”Emp”.

Example to Delete a record from Emp table by using drop down list

Step1:Design of front end and back end



step :2 Creating Asp.Net Mvc Application
Step:3 Adding ORM tool
Step:4 Adding Model[Employee.cs]

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
namespace MvcEFDropdownListDeleteExample.Models
{
    public class Employee
    {
        public int Eno { get; set; }
        mydbmvcEntities objdb = new mydbmvcEntities();
        public List<int> GetEno()
        {
            List<int> eno = objdb.emps.Select(e => e.empno).ToList();
            return eno;
        }
        public int DeleteEmp()
        {
            emp objemp = objdb.emps.Find(Eno);
            try
            {
                objdb.emps.Remove(objemp);
                objdb.SaveChanges();
                return 1;
            }
            catch
            {
                return 0;
            }
        }
    }
}
```

Controller Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using MvcEFDropdownListDeleteExample.Models;
namespace MvcEFDropdownListDeleteExample.Controllers
{
    public class EmpController : Controller
    {

        Employee objemp = new Employee();
        internal void ForwordEmpNo()
```

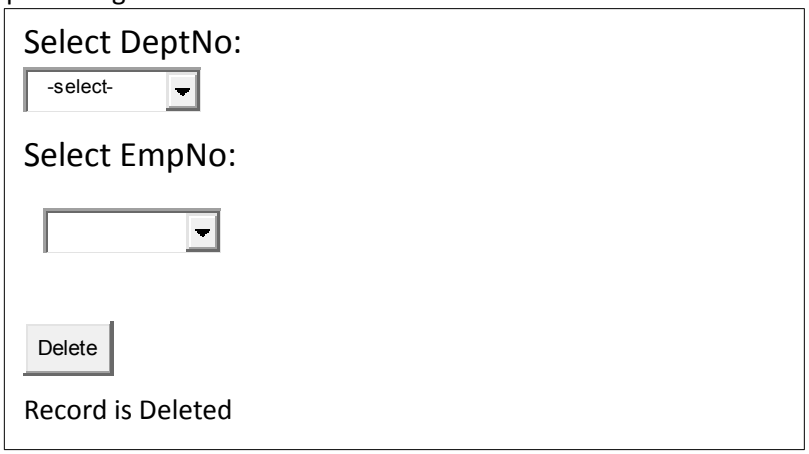
```
        {
            Employee objemp = new Employee();
            ViewData["dropEmpNoList"] = new SelectList(objemp.GetEno());
        }
        // GET: Emp
        public ActionResult Index()
        {
            ForwordEmpNo();
            return View();
        }
        [HttpPost]
        public ActionResult Delete()
        {
            Employee objemp = new Employee
            {
                Eno = int.Parse(Request["dropEmpNoList"])
            };
            ViewData["delvalue"] = objemp.DeleteEmp();
            ForwordEmpNo();
            return View("Index");
        }
    }
}
```

View Code:

```
@{
    Layout = null;
    string msg = "";
    if (this.IsPost)
    {
        int i = (int)ViewData["delvalue"];
        if (i == 1)
        {
            msg = "Record is Deleted";
        }
        else
        {
            msg = "Record is not Deleted";
        }
    }
}
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form id="myform" method="post">
<div>
            select EmpNo : @Html.DropDownList("dropEmpNoList","-Select-")
<br /><br />
<input type="submit" id="btnDelete" value="Delete"formaction="~/Emp/Delete" />
<br /><br />
@Html.Label("lblmsg",msg)
</div>
</form>
</body>
</html>
```

Example to Deleting Record using Two Drop Downs & Entity Framework

step :1 design front end and back end



Step-2: Creating MVC New Project

Step-3: Adding Entity Model

Step-4: Adding Model

Model Code: Employee.cs

```
public class Employee
    {
        companyEntities objcomp = new companyEntities();
        public int dno { get; set; }
        public int eno { get; set; }
        public List<int> GetDnos()
        {
            List<int> dnos = objcomp.depts.Select(d => d.deptno).ToList();
            return dnos;
        }
        public List<int> GetEnos()
        {
            List<int> enos = objcomp.emps.Where(d => d.deptno ==
dno).Select(e=>e.empno).ToList();
            return enos;
        }
```

```
        public int DeleteEmp()
        {
            emp objemp = objcomp.emps.Find(eno);
            try
            {
                objcomp.emps.Remove(objemp);
                objcomp.SaveChanges();
                return 1;
            }
            catch
            {
                return 0;
            }
        }
    }
```

Step-5: Adding Controller

Controller Code: EmpController.cs

```
public class EmpController : Controller
    {
        //
        // GET: /Emp/
        public void ForwardDeptno()
        {
            Employee obj = new Employee();
            ViewBag.dnos = obj.GetDnos();
        }


        public ActionResult Index()
        {
            ForwardDeptno();
            return View();
```

```
        }
        [HttpPost]
        public ActionResult Index(string s)
        {
            Employee obj = new Employee
            {
                dno = int.Parse(Request["DropDnosList"])
            };
            ViewBag.enos=obj.GetEnos();
            ForwardDeptno();
            return View("Index");
        }
        [HttpPost]
        public ActionResult Delete()
        {
            Employee obj = new Employee
            {
                eno = int.Parse(Request["DropEnosList"])
            };
            ViewData["delvalue"] = obj.DeleteEmp();
            ForwardDeptno();
            ViewBag.enos = obj.GetEnos();
            return View("Index");
        }
    }
```

Step-6: Adding View

View Code: Index.cshtml

```
@{
    Layout = null;
    string msg = "";
    if(this.IsPost)
```

```
    {
        if(Request["btnDelete"]=="Delete")
        {
            int i = (int)ViewData["delvalue"];
            if(i==1)
            {
                msg = "Record is Deleted";
            }
            else
            {
                msg = "Record is not Deleted";
            }
        }
    }
}
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form name="myform" method="post" action="~/Emp/Index">
<div>
            Select DeptNo:
<select name="DropDnosList" onchange="submit()">
<option>-select-</option>
                @foreach (int dno in ViewBag.dnos)
                {
<option>@dno</option>
                }
</select>
<br /><br />
            Select EmpNo:
<select name="DropEnosList">
<option>-select-</option>
                @{
                    if (this.IsPost)
                    {
                        foreach (int eno in ViewBag.enos)
                        {
<option>@eno</option>
                        }
                    }
                }
</select>
<br /><br />
<input name="btnDelete" type="submit" value="Delete"
formaction="~/Emp/Delete" />
<br /><br />
            @Html.Label("lblMsg", msg)
</div>
</form>
</body>
```

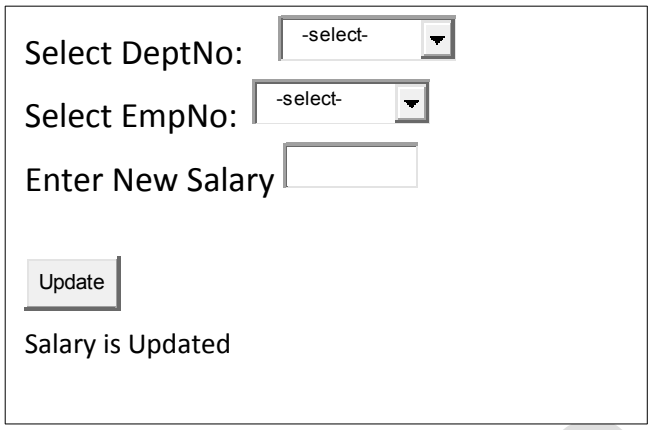
```
</html>
```

Step-7: Updating RouteConfig.cs

```
        Controller="Emp"
```

Example to update the record by using drop down list and Entity frame work

Step: 1 Design for front End and Back End



Step-2: Creating MVC New Project

Step-3: Adding Entity Model

Step-4: Adding Model

Model Code: Employee.cs

```
public class Employee
    {
        companyEntities objcomp = new companyEntities();
        public int Dno { get; set; }
        public int Eno { get; set; }
        public decimal Esal { get; set; }
        public List<int> GetDnos()
        {
            List<int> dnos = objcomp.depts.Select(d => d.deptno).ToList();
            return dnos;
        }
```

```
        public List<int> GetEnos()
        {
            List<int> enos = objcomp.emps.Where(d => d.deptno ==
Dno).Select(e=>e.empno).ToList();
            return enos;
        }
        public int UpdateEmp()
        {
            emp objemp = objcomp.emps.Find(Eno);
            objemp.empsal = Esal;
            try
            {
                objcomp.SaveChanges();
                return 1;
            }
            catch
            {
                return 0;
            }
        }
    }
```

Step-5: Adding Controller

Controller Code: EmpController.cs

```
public class EmpController : Controller


  {
        Employee objmodel = new Employee();
        public void ForwardDrops()
        {
```

```
            ViewData["DropDnosList"] = new SelectList(objmodel.GetDnos());
            ViewData["DropEnosList"] = new SelectList(objmodel.GetEnos());
        }
        public ActionResult Index()
        {
            ForwardDrops();
            return View();
        }
        [HttpPost]
        public ActionResult Index(string s)
        {
            objmodel.Dno = int.Parse(Request["DropDnosList"]);
            ForwardDrops();
            return View("Index");
        }
        [HttpPost]
        public ActionResult Update()
        {
            objmodel.Eno = int.Parse(Request["DropEnosList"]);
            objmodel.Esal = decimal.Parse(Request["txtNewSal"]);
            ViewData["updatevalue"] = objmodel.UpdateEmp();
            ForwardDrops();
            return View("Index");
        }
    }
```

Step-6: Adding View

```
@{
    Layout = null;
    string msg = "";
    if(this.IsPost)
    {
```

```
        if(Request["btnUpdate"]=="Update")
        {
            int i = (int)ViewData["updatevalue"];
            if(i==1)
            {
                msg = "Salary is Updated";
            }
            else
            {
                msg = "Salary is not Updated";
            }
        }
    }
}

<!DOCTYPE html>

<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form name="myform" method="post" action="~/Emp/Index">
<div>
            Select DeptNo:
            @Html.DropDownList("DropDnosList", null, "-select-",
htmlAttributes: new { onchange = "submit()" })
<br /><br />
            Select EmpNo:
            @Html.DropDownList("DropEnosList", "-select-")
<br /><br />
            Enter New Salary @Html.TextBox("txtNewSal")
<br/><br/>
<input name="btnUpdate" type="submit" value="Update"
formaction="~/Emp/Update" />
<br /><br />
            @Html.Label("lblMsg", msg)
</div>
</form>
</body>
</html>
```

Step-7: Updating RouteConfig.cs

        Controller=”Emp”

Example to delete a record from Emp table by using Html Table

EmployeeNo	EmployeeName	Employee salary	Delete
111	Rama	1000	Delete
222	Sitha	2000	Delete

Record is Deleted

Step-1: Creating MVC New Project

Step-2: Adding Entity Model

Step-3: Adding Model

Model Code: Employee.cs

```
public class Employee
    {
        public int Eno { get; set; }
        EdatabaseEntities objdb = new EdatabaseEntities();
        public IQueryable GetEmp()
        {
            return objdb.emps;
        }
        public int DeleteEno()
        {
            emp objeno = objdb.emps.Find(Eno);
            try
            {
                objdb.emps.Remove(objeno);
                objdb.SaveChanges();
                return 1;
            }
            catch
            {
                return 0;
            }
        }
    }
```

Step-4: Adding Controller

Controller Code: EmpController.cs

```
public class EmpController : Controller
    {
```

```
        Employee objmodel = new Employee();
        public void ForwardEmp()
        {
            ViewBag.emp = objmodel.GetEmp();
        }
        public ActionResult Index()
        {
            ForwardEmp();
            return View();
        }
        public ActionResult DeleteEmp()
        {
            objmodel.Eno = int.Parse(Request.QueryString["eno"]);
            ViewData["delvalue"] = objmodel.DeleteEno();
            ForwardEmp();
            return View("Index");
        }
    }
```

Step-5: Adding View

View Code: Index.cshtml

```
@{
    Layout = null;
    string msg = "";
    if (this.IsPost)
    {
        int i = (int)ViewData["delvalue"];
        if (i == 1)
        {
            msg = "Record is Deleted";
        }
        else
        {
            msg = "Record is not Deleted";
        }
    }
}
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form name="myform" method="post">
<div>
<table border="1">
<tr>
<th>Employee No</th>
<th>Employee Name</th>
<th>Employee Salary</th>
<th>Delete</th>
```

```
</tr>
@{
                    foreach (var item in ViewBag.emp)
                    {
<tr>
<td>@item.Eno</td>
<td>@item.Ename</td>
<td>@item.salary</td>
<td>@Html.ActionLink("Delete", "DeleteEmp", new {eno=item.Eno})</td>
</tr>
                    }
}
</table>
@Html.Label("lblMsg",msg)
</div>
</form>
</body>
</html>
```

Step-6: Updating RouteConfig.cs

Controller="Emp"

Example to Insert a record into EmpTable by using Html Table:

EmployeeNo	EmployeeName	EmployeeSalary	Insert
111	Rama	1000	
222	Sitha	2000	
333	Ravi	3000	
444	raju	4000	Insert

Record is Inserted

Step-1: Creating MVC New Project

Step-2: Adding Entity Model

Step-3: Adding Model

Model Code: Employee.cs

```
public class Employee
    {
        public int Eno { get; set; }
        public string Ename { get; set; }
```

```
        public decimal Esal { get; set; }

        EdatabaseEntities objdb = new EdatabaseEntities();
        public IQueryable GetEmp()
        {
            return objdb.emps;
        }
        public int InsertEmp()
        {
            emp objemp = new emp();
            objemp.Eno = Eno;
            objemp.Ename = Ename;
            objemp.salary = Esal;
            try
            {
                objdb.emps.Add(objemp);
                objdb.SaveChanges();
                return 1;
            }
            catch
            {
                return 0;
            }
        }
    }
```

Step-4: Adding Controller

Controller Code: EmpController.cs

```
public class EmpController : Controller
    {
        Employee objmodel = new Employee();
        public void ForwardEmp()
        {
            ViewBag.emp = objmodel.GetEmp();
        }

        public ActionResult Index()
        {
            ForwardEmp();
            return View();
        }

        [HttpPost]
        public ActionResult Insert()
        {
            objmodel.Eno = int.Parse(Request["txtEno"]);
            objmodel.Ename = Request["txtEname"];
            objmodel.Esal = decimal.Parse(Request["txtEsal"]);
            ViewData["insertvalue"] = objmodel.InsertEmp();
            ForwardEmp();
            return View("Index");
        }
```

```
    }
```

Step-5: Adding View

View Code: Index.cshtml

```
@{
    Layout = null;
    string msg = "";
    if (this.IsPost)
    {
        int i = (int)ViewData["insertvalue"];
        if (i == 1)
        {
            msg = "Record is Inserted";
        }
        else
        {
            msg = "Record is not Inserted";
        }
    }
}

<!DOCTYPE html>

<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form name="myform" method="post">
<div>
<table border="1">
<tr>
<th>Employee No</th>
<th>Employee Name</th>
<th>Employee Salary</th>
<th>Insert</th>
</tr>
@{
                    foreach (var item in ViewBag.emp)
                    {
<tr>
<td>@item.Eno</td>
<td>@item.Ename</td>
<td>@item.salary</td>
</tr>
                    }
}
<tr>
<td>@Html.TextBox("txtEno")</td>
<td>@Html.TextBox("txtEname")</td>
```

```
<td>@Html.TextBox("txtEsal")</td>

<td><input type="submit" name="btnInsert" value="Insert"
formaction="~/Emp/Insert"</td>
</tr>
</table>
@Html.Label("lblMsg", msg)
</div>
</form>
</body>
</html>
```

Step-6: Updating RouteConfig.cs

Controller=”Emp”

Example to update salary by using Html Table

Design of front end and Bcak End

Employee No	Employee Name	Employee Salary	Edit
111	Rama	10000	Edit
222	Sitha	20000	Edit
333	Ravi	3000 3500	Edit

Salary is Updated

Step-1: Creating MVC New Project

Step-2: Adding Entity Model

Step-3: Adding Model

Model Code: Employee.cs

```
public class Employee
    {
        public int Eno { get; set; }

        public decimal Esal { get; set; }

        EdatabaseEntities objdb = new EdatabaseEntities();
        public IQueryable GetEmp()
        {
            return objdb.emps;
        }
        public int UpdateEmp()
        {
```

```
            emp objemp = objdb.emps.Find(Eno);
            objemp.salary = Esal;
            try
            {
                objdb.SaveChanges();
                return 1;
            }
            catch
            {
                return 0;
            }
        }
    }
```

Step-4: Adding Controller

Controller Code: EmpController.cs

```
public class EmpController : Controller
    {
        Employee objmodel = new Employee();
        public void ForwardEmp()
        {
            ViewBag.emp = objmodel.GetEmp();
        }

        public ActionResult Index()
        {
            ForwardEmp();
            return View();
        }

        public ActionResult EditEmp()
        {
            ForwardEmp();
            return View("Update");
        }
        public ActionResult Cancel()
        {
            ForwardEmp();
            return View("Index");
        }

        [HttpPost]
        public ActionResult Update()
        {
            objmodel.Eno = int.Parse(Request["txtEno"]);
            objmodel.Esal=decimal.Parse(Request["txtNewSal"]);
            ViewData["updatevalue"] = objmodel.UpdateEmp();
            ForwardEmp();
            return View("Index");
        }

    }
```

Step-5: Adding View

View Code: Index.cshtml

```
@{
    Layout = null;
    string msg = "";
    if (this.IsPost)
    {
        if(Request["btnUpdate"]=="Update")
        {
        int i = (int)ViewData["updatevalue"];
        if (i == 1)
        {
            msg = "Salary is Updated";
        }
        else
        {
            msg = "Salary is not Updated";
        }
        }
    }
}

<!DOCTYPE html>

<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Index</title>
</head>
<body>
<form name="myform" method="post">
<div>
<table border="1">
<tr>
<th>Employee No</th>
<th>Employee Name</th>
<th>Employee Salary</th>
<th>Edit</th>
</tr>
@{
                    foreach (var item in ViewBag.emp)
                    {
<tr>
<td>@item.Eno</td>
<td>@item.Ename</td>
<td>@item.salary</td>
<td>@Html.ActionLink("Edit", "EditEmp", new { eno = item.Eno })</td>
</tr>
                    }
}
</table>
@Html.Label("lblMsg", msg)
```

```
</div>
</form>
</body>
</html>
```

Step-6: Adding View

View Code: Update.cshtml

```
@{
    Layout = null;
}

<!DOCTYPE html>

<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Update</title>
</head>
<body>
<form name="myform1" method="post">
<div>
<table border="1">
<tr>
<th>Employee No</th>
<th>Employee Name</th>
<th>Employee Salary</th>
<th>Update/Cancel</th>
</tr>
@{
                    foreach (var item in ViewBag.emp)
                    {

if(item.Eno.ToString()==Request.QueryString["eno"])
                        {
<tr>
<td><input type="text" name="txtEno" value=@item.Eno readonly /></td>
<td>@item.Ename</td>
<td><input type="text" name="txtNewSal" value=@item.salary /></td>
<td><input type="submit" name="btnUpdate" value="Update"
formaction="~/Emp/Update" />
@Html.ActionLink("Cancel", "Cancel")</td>
</tr>
                        }
                        else
                        {
<tr>
<td>@item.Eno</td>
<td>@item.Ename</td>
<td>@item.salary</td
<td>@Html.ActionLink("Edit", "EditEmp", new { eno = item.Eno })</td>
</tr>
                        }
```

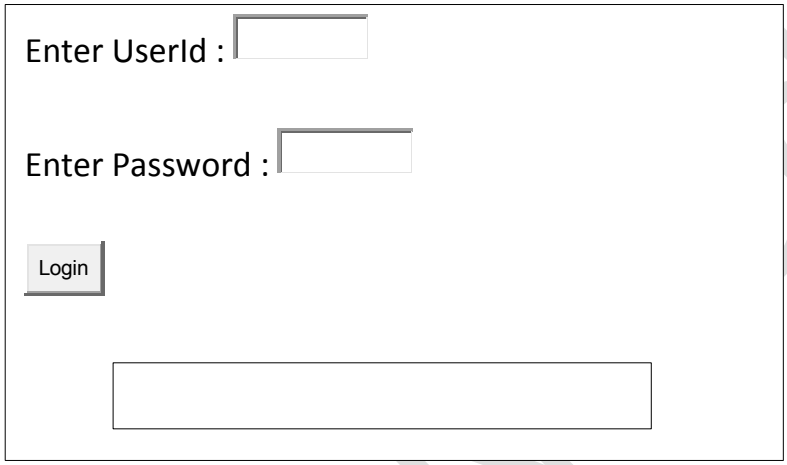
```
                    }
}
</table>
</div>
</form>
</body>
</html>
```

Step-7: Updating RouteConfig.cs

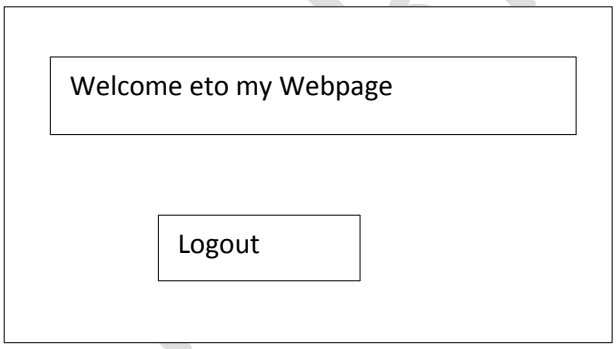
Controller=”Emp”

Example for login

Design of front end and back end:



Welcome.cshtml



Step-1: Creating MVC New Project

Step-2: Adding Entity Model

Step-3: Adding Model

Model Code: Login.cs

```
public class Login
    {
```

```
        public string UName { get; set; }
        public string UPwd { get; set; }

        UserDetailsEntities objdb = new UserDetailsEntities();

        public List<userlogin> GetUser()
        {
            //checking user is exist or not
            userlogin objcheck = objdb.userlogins.FirstOrDefault(u =>
u.username.Equals(UName) && u.userpwd.Equals(UPwd));

            //if user is exist getting that user record based on given
credentials
            if (objcheck != null)
            {
                List<userlogin> user = objdb.userlogins.Where(u =>
u.username.Equals(UName) && u.userpwd.Equals(UPwd)).ToList();
                return user;
            }
            else
            {
                return null;
            }
        }
    }
```

Step-4: Adding Controller

Controller Code: LoginController.cs

```
public class LoginController : Controller
    {
        public ActionResult Login()
        {
            return View();
        }

        public ActionResult LogoutUser()
        {
            return View("Login");
        }

        [HttpPost]
        public ActionResult Welcome()
        {
            Login objmodel = new Login
            {
                UName = Request["txtUname"],
                UPwd = Request["txtPwd"]
            };
            ViewBag.user = objmodel.GetUser();
            if (ViewBag.user != null)
            {
                return View("Welcome");
```

```
            }
            else
            {
                return View("Login");
            }
        }
    }
```

Step-5: Adding View

View Code: Login.cshtml

```
@model MVCLoginEFNormalTextBox.Models.Login
@{
    Layout = null;
    if (this.IsPost)
    {
        if (ViewBag.user == null)
        {
<script type="text/javascript">
            alert("Invalid User")
</script>
        }
    }

}

<!DOCTYPE html>

<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>Login</title>
</head>
<body>
<form name="myform" method="post">
<div>
            Enter Username @Html.TextBox("txtUname")
<br /><br />
            Enter Password @Html.TextBox("txtPwd")
<br /><br />
<input name="btnLogin" type="submit" value="Login"
formaction="~/Login/Welcome" />
</div>
</form>
</body>
</html>
```

Step-6: Adding View

View Code: Welcome.cshtml

```
<form name="myform" method="post">
```

```
<div>
<table border="1">
<tr>
<th>User ID</th>
<th>Name</th>
<th>User Name</th>
<th>User Pwd</th>
<th>User Hint</th>
</tr>
@foreach (var item in ViewBag.user)
                {
<tr>
<td>@item.userid</td>
<td>@item.name</td>
<td>@item.username</td>
<td>@item.userpwd</td>
<td>@item.userhint</td>
</tr>
                }
</table>
<br/><br/><br/>
@Html.ActionLink("Logout","LogoutUser")
</div>
</form>
```

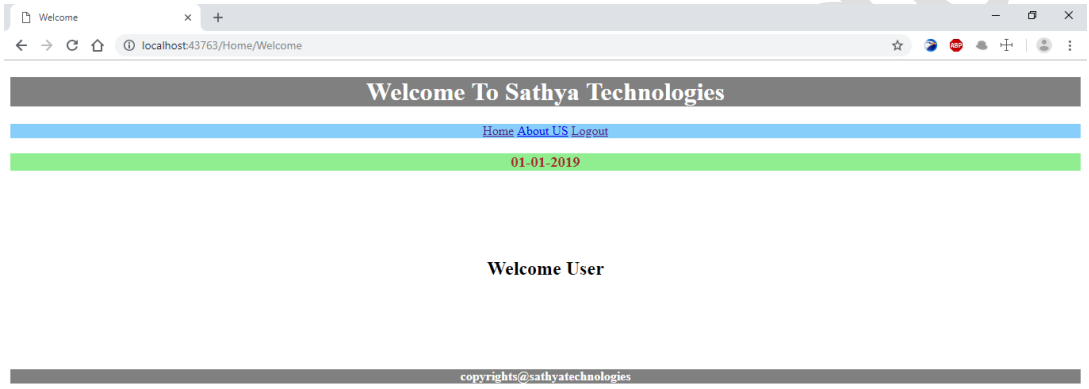
Step-7: Updating RouteConfig.cs

Controller=”Login”, action=Login”

MVC Master Page Example

Login Page: - (Design)

Welcome Page: - (Design)



Step-1: Creating MVC Project

Step-2: Adding Master Page

- Go to Solution Explorer
- Go to Views folder and expand Views folder
- Right click on Shared folder
- Add -> New Item
- It will open new item window
- Select MVC 4 Layout Page(Razor)
- Rename it as _MasterPage1.cshtml
- Click add
- By this process we added master page
- Write Code like below

```
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width" />
<title>@ViewBag.Title</title>
```

```
</head>
<body>
<header style=" background-color:gray;text-align:center">
<h1 style="color:white">Welcome To Sathya Technologies</h1>
</header>
<nav style="background-color:lightskyblue;text-align:center;color:green">
<div>
@Html.ActionLink("Home", "Login", "Home")
@Html.ActionLink("About US", "AboutUs", "Home")
@Html.ActionLink("Logout", "Logout", "Home")
</div>
</nav>
<div style="color:brown;background-color:lightgreen">
@RenderSection("DailyChangableContent",true)
</div>
<br /><br />
<br /><br />
<div>
@RenderBody()
</div>
<br/><br/>
<br/><br/>
<footer style="background-color:gray;text-align:center">
<h4 style="color:white">copyrights@sathyatechnologies</h4>
</footer>
</body>
</html>
```

Some important points (for MasterPage):-

Nav- Nav tags are generally used for navigation bar. We can add links to navigate between the pages.

@RenderSection- Render section is just like a place holder where we can place our code that we can inject from the content page. If we make render section property true, we must need to add this section in all the pages which are used in this layout page, otherwise it will go through as a not implemented error. If we want to make this an option, we need to set it to false.

@RenderBody- RenderBody is a place holder to display the content of the View.

Header & Footer- These are useful to display the header and footer information.

Layout

Step-3: Adding Controler(HomeController.cs)

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;

namespace MVCMasterPage.Controllers
```

```
{
    public class HomeController : Controller
    {
        public ActionResult Login()
        {
            return View();
        }
        public ActionResult Welcome()
        {
            return View("Welcome");
        }
        public ActionResult AboutUs()
        {
            return View("Login");
        }
        public ActionResult Logout()
        {
            return View("Login");
        }
    }
}
```

Step-4: Adding Views

- **Go to controller**
- **Right click on Login method**
- **Click on Add View**
- **Rename it as Login**
- **Check "use a Layout or MasterPage" at bottom**
- **Click on browse button**
- **Select MasterPage1 in Views Folder-> Shared Folder**
- **Click Add**
- **By this process we added Child page for MasterPage1**

Note: The below Index View is created by using _MasterPage1.cshtml page. In general, if you use _Layout.cshtml, it doesn't display here because it is set in the global default Master page. If we want to set our Layout page as default layout page, we need to set it in _ViewStart.html page, as shown below.

Login.cshtml:-

```
@{
    ViewBag.Title = "Login";
    Layout = "~/Views/Shared/_MasterPage1.cshtml";
}
@section DailyChangableContent{
<h3 style="text-align:center">@DateTime.Today.ToShortDateString()</h3>
}
<br /><br />
<form name="myform" method="post">
<div style="text-align:center;top:200px;border:double;background-
color:lightgray;width:350px;margin-left:500px;color:blue">
```

```
<h2>Login</h2>
        Enter Username @Html.TextBox("txtUName", null, new { style =
"color:red" })
<br /><br />
        Enter Password @Html.Password("txtUPwd", null, new { style =
"color:red" })
<br /><br />
<input type="submit" name="btnLogin" style="color:aqua;background-
color:blueviolet" formaction="~/Home/Welcome" />
<br/><br/>
</div>
</form>
```

Welcome.cshtml:-

```
@{
    ViewBag.Title = "Welcome";
    Layout = "~/Views/Shared/_MasterPage1.cshtml";
}

@section DailyChangableContent{
<h3 style="text-align:center">@DateTime.Today.Date.ToShortDateString()</
h3>
}
<div>
<h2 style="text-align:center">Welcome User</h2>
</div>
```

Step-5:- Update RouteConfig.cs

```
controller = "Home", action = "Login"
```

Bundling:

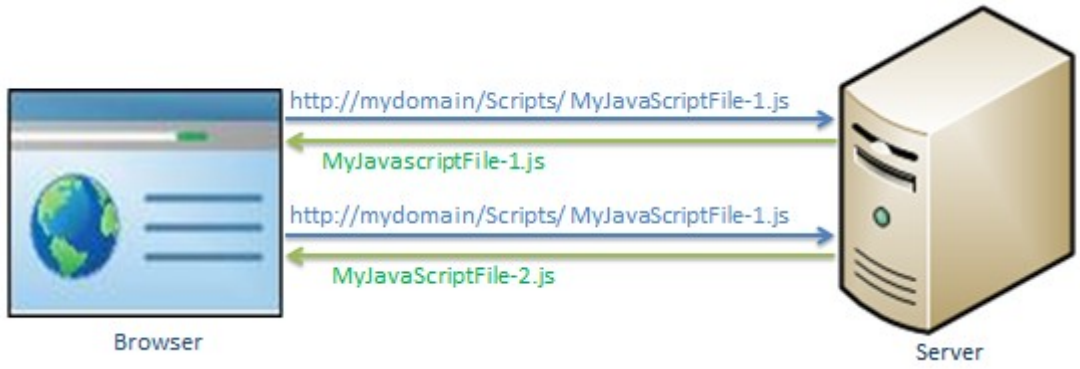
What is bundling?

Bundling is a technique were introduced in MVC4 to improve request load time.

Bundling allows us to load the bunch of static files from the server into one HTTPRequest

Diagram:1 Which explains before bundling.

LOAD SCRIPT FILES IN SEPARATE REQUESTS:

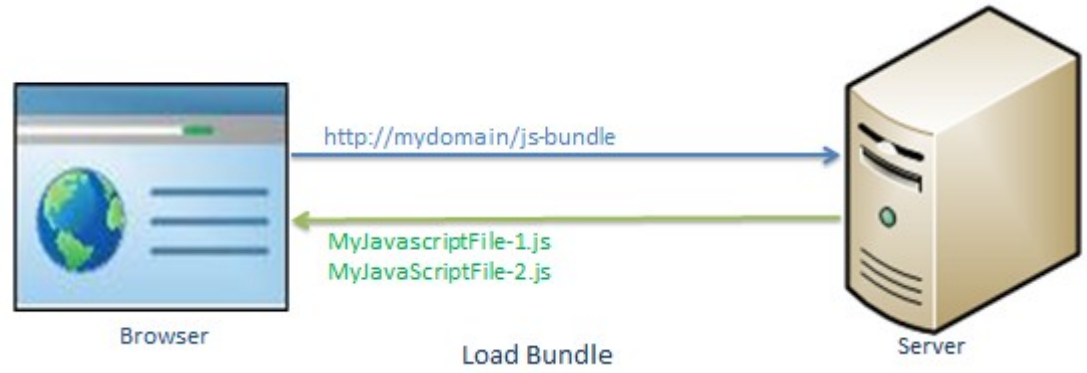


EXPLANATION FOR THE ABOVE DIAGRAM:

In the above diargarm browser sends 2 separate requests to load 2 different javascript files i.e Myjavascriptfile-1.js and Myjavascriptfile-2.js

Bundling technique in MVC 4 allows us to load more than one javascript file i.e Myjavascriptfile-1.js and Myjavascriptfile-2.js in one request as shown below

Diagram to represent Bundedling:



What is Minification:

Minification is a technique which optimises script or css file size by removing unneccesary

Whitespace &comments &shortening variable names toone character.

For eg:consider the following javascript function.

```
sayHello=function(name)
{
        //this is comment
        Var msg=”hello”+name;
```

```
    alert(msg);
}
```

The above javascript file will be optimiszedand minimiszedinto the following script.

Eg:for minimiszed javascript

sayHello=function(n){var t="hello"+n;alert(t)}

As in the above code,it has removed unnecessarywhite space,comments and also shortening variable names to reduce the characters which in turn will reduce the size of the javascript file.

Conclusion:

Bundling&Mining impacts on the loading of the page,it loads page faster by minimising size of the file and numbers of requests.

Bundle Types:

MVC5 includes following bundule classes in System.Web.Optimization base base class library.

ScriptBundle:

SciptBundle is responsible for javascript minfication of single or multiple script files.

StyleBundle:

StyleBundle is responsible for CSS minification of single or multiple style sheet files.

Dynamic Folder Bundle:

It represents a bundle that ASP.NET creates from a folder that contains the same type .

All the above bundle classes are included in System.Web.Optimization

All the Bundle classes are derived classes of Bundle class.

Routing:

Note :Routing is not a specific to Mvc Frame Work it can be used with Asp.Net Web form application (or) Asp.Net Mvc application

What is Routing?

Routing define executing flow for a particular request

It Asp.Net every request will contain like below Http://Domain/StudentInfo.aspx. means in Asp.Net request it is targeted the .aspx file due to that reason the URL are file based URL's . To that reason the Url are file based url's to avoid mapping each Url with a physical file in Asp.Net Mvc introduced a new concept called Routing,

In Mvc Routing enables us to define Url pattern that mapes to the request handler.

The request handler can be a file or class

In Asp.Net request handler is .Aspx file but In MVC request handler is Controller class and Action method.

For example : Http://Domain/Students/Index.

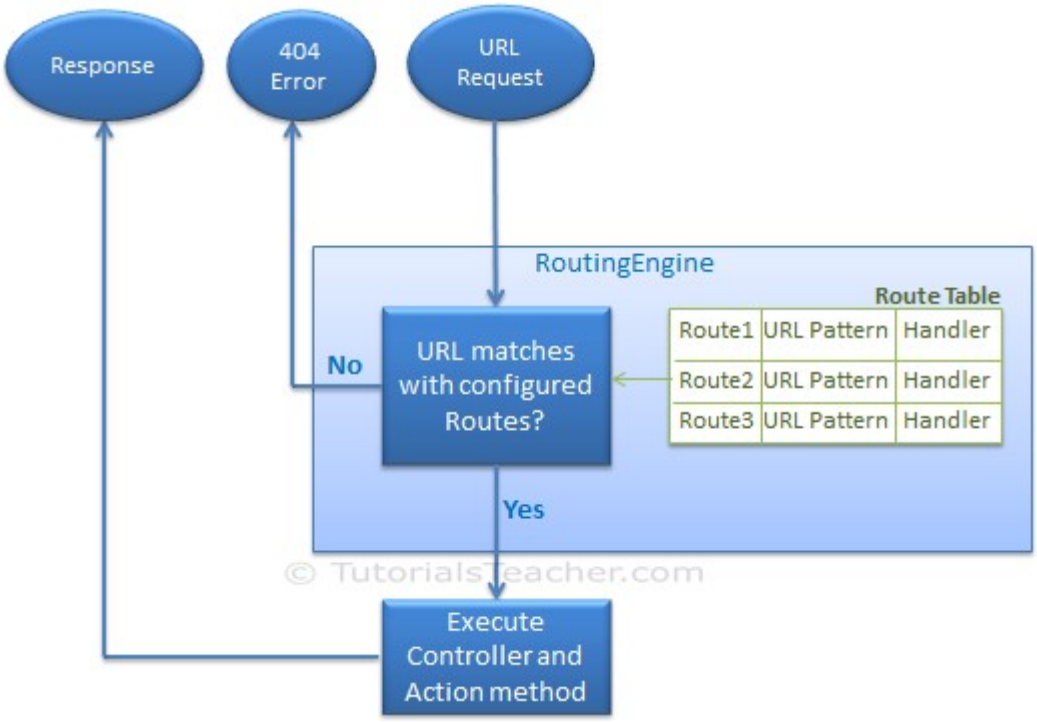
Here Students is Controller, Index is ActionMethod.

What is Route?

Route defines the Url pattern and handler information, All the configured routes of an application stored in internally route table and will be used by Routing engine to determine appropriate

handler class or file for an incoming request.

The following diagram illustrates (or) Explain the routing process.



Configured Route?

Every Mvc application must configured atlest one route which is configured by Mvc Frame Work by default.

You can registered a route with in routeConfiged class which is in RouteConfig.Cs under App.Start folder.

The following diagram represent How to Configure a route with in the RouteConfig class.

Structure of RouteConfig.Cs file:

```
Public class RouteConfig

{

 Public static void RegisterRoutes(RouteCollection Routes)

  {

 Routes.Ignore("{resourse}.axd/{*pathInfo}");

Routes.MapRoute(name:"Default",Url:"{Controller}/{action}/{id}"
defaults:new{Controller="Home",action="Index",id="urlparameter.optional"}

);

}//method

}//class
```

Filters in MVC:

There are 5 types of Filters in MVC.

1. Authentication Filters.
2. Authorization Filters.
3. Action Filters.
4. Result Filters.
5. Exception Filters.

1.Authentication Filters.

- Authentication Runes before any Other Runs(or) any action methods
- Authencation Confirms that you are a valid (or) invalid user
- Action Filter implements the authentication interface i.e." IAuthenticationFilter" interface.

2.Authorization Filters.

Authorization filters are responsible for checking user access these implements the "IAuthorizatuionFilter" interface with in the Frame Work

3.Action Filters.

Action Filters can Apply on Entire Controller or Single Action Methods. these filters will be called before after the action Start Execution and After the action has Executed.

These Action Filters contain logic i.e. executed before anf after a Controller Action Execute. you can use an action filters to modify the view data that a controller action returns. Inter Face of action Filters is "IActionFilter".

4.Result Filters:

These filters contain logic i.e. executed before and after a view result is Exected.

That means result filterwill execute before view result and after view result. Interface of result filter is :"IResultFilter".

5.Exeception Filter:

Exception Filter will execute if there are any unhandled exception throw during the execution.

These filters can be used to handled error rised by either your Controller action (or) Controller action result.

Interface of Exception Filter is "IExceptionFilter".

RN Reddy IT School

For Training's contact

Cell: +91 9966640779, 9885199122 &9866183094

E-Mail:rnreddyitschool@gmail.com

Website: www.rnreddyitschool.co

FaceBook Group:https://www.facebook.com/groups/rnreddytechsupports

FaceBook Page:https://www.facebook.com/groups/reddyitschool

Youtube Channel:-https://www.youtube.com/c/RNREDDYITSchool